

WHITEPAPER · SPECIFICATION · VERSION 2.0

Telegraph

Telegraph is the network that turns intelligence into a graded commodity.
Ranked by performance. Driven by demand.

Ahmed Ali

ahmed@telegraphprotocol.com

info@telegraphprotocol.com

[Telegraph](#)

Version 2.0

2nd October 2026

CONTENTS

PART I – WHITEPAPER

ABSTRACT	4
THESIS: HOW TELEGRAPH WORKS - END TO END (TL;DR)	5
1. TELEGRAPH: AN OVERVIEW	12
2. DESIGN OVERVIEW	15
3. NETWORK PARTICIPANTS	18
4. INTENTS AND INTELLIGENCE SUPPLY	21
5. RANKING AND EVALUATION	23
6. VALIDATORS AND CONSENSUS	31
7. ROUTING AND THE AUTONOMOUS ENGINE	36
8. PAYMENTS AND SETTLEMENT	40
9. TOKENOMICS AND INCENTIVES	45
10. BOOTSTRAPPING	51
11. GOVERNANCE	54
12. SECURITY MODEL	58

PART II – REFERENCE

A. PROTOCOL DEFINITIONS AND STATE	65
B. INTENT AND MINER REGISTRATION PROTOCOL	70
C. EVALUATOR PROTOCOL	74
D. RANKING PROTOCOL	80
E. VALIDATOR AND CONSENSUS PROTOCOL	85
F. ROUTING PROTOCOL	91
G. PAYMENT AND SETTLEMENT PROTOCOL	96
H. ECONOMIC PROTOCOL	102
I. AUTONOMOUS ENGINE AND BOOTSTRAP PROTOCOL	110
J. GOVERNANCE PROTOCOL	115
K. SECURITY AND FAILURE HANDLING	120

PART III – GENESIS, PARAMETERS & APPENDICES

A. GENESIS PARAMETER REGISTRY	129
B. CONSOLIDATED FORMULA REGISTRY	134
C. STATE AND COMMITMENT REGISTRY	136
D. REFERENCE BOUNDARY	137

ABSTRACT

Telegraph is the network that turns intelligence into a graded commodity. Ranked by performance. Driven by demand. Telegraph implements this through a peer-to-peer ranking protocol for machine intelligence. The protocol organizes intelligence around Intents, which define the tasks supported by the network. For each Intent, independent Miners provide machine-callable intelligence, independent Evaluators measure Miner performance, and Validators verify and finalize the resulting ranking state.

Each active Intent maintains a Canonical Evaluator and a Miner Ranking. Evaluators compete for Canonical status through a protocol-defined competition and verification process. The current Canonical Evaluator provides the evaluation method used to measure Miner performance for that Intent. Miner evaluations are aggregated into a deterministic ranking that is finalized by the Validator network and used to route Consumer demand.

Participation is permissionless. A new Miner may enter an Intent, outperform existing participants, rise in the Miner Ranking, receive a greater share of routed demand, and earn revenue from paid intelligence delivery. A new Evaluator may enter the same Intent, demonstrate stronger evaluation performance, pass protocol verification, and replace the current Canonical Evaluator. Intelligence supply and the mechanism used to evaluate that supply can therefore evolve independently over time.

Telegraph is agnostic to the technology used to produce intelligence. A Miner may expose an AI model, inference service, API, dataset, algorithm, search system, analytical service, tool, or another machine-callable capability compatible with an Intent. These systems participate through common protocol interfaces and are ranked according to the evaluation process defined for the Intent.

Evaluation is also open to independent participation. Evaluator developers construct the logic and task-specific material required by their methods. Telegraph provides common protocol machinery for registration, competition, commitments, bounded evaluation-material influence, hidden verification, ranking, and state finalization. This allows evaluation methods to improve without requiring Telegraph to operate a bespoke evaluation system for each Intent.

Paid Consumer activity is denominated in USDC. Higher-ranked Miners receive greater routing opportunity and are compensated from fulfilled Consumer demand. Canonical Evaluators and Validators receive protocol rewards under the MACHINA emission system. Real-time intelligence delivery operates independently from the cadence of economic settlement, while canonical token settlement occurs on Ethereum.

The protocol coordinates a continuous competitive process in which better intelligence can earn greater demand, stronger Evaluators can replace weaker Evaluators, and updated rankings can redirect demand as measured performance changes. This makes heterogeneous intelligence measurable, comparable, rankable, and economically connected to paid demand. Miner-attributable paid value is settled through open-market MACHINA acquisition, linking intelligence consumption directly to MACHINA demand. Telegraph defines the interfaces and rules through which intelligence supply, evaluation, ranking, routing, verification, accounting, and settlement operate as one system.

Thesis: How Telegraph Works - End to End

TL;DR

Telegraph is the network that turns intelligence into a graded commodity. Ranked by performance. Driven by demand. Technically, it operates as a peer-to-peer ranking protocol for machine intelligence.

Machine intelligence already exists across models, APIs, datasets, algorithms, search systems, analytical services, tools, and other machine-callable systems. Humans, applications, agents, smart contracts, machines, and autonomous systems increasingly depend on these capabilities to make decisions and perform work.

Telegraph provides a common protocol for organizing that intelligence around specific tasks, measuring its performance, producing verified rankings, routing demand through those rankings, and connecting useful intelligence to economic reward.

The intelligence, evaluation methods, and demand come from independent participants. Telegraph provides the shared coordination layer through which those participants operate as one system.

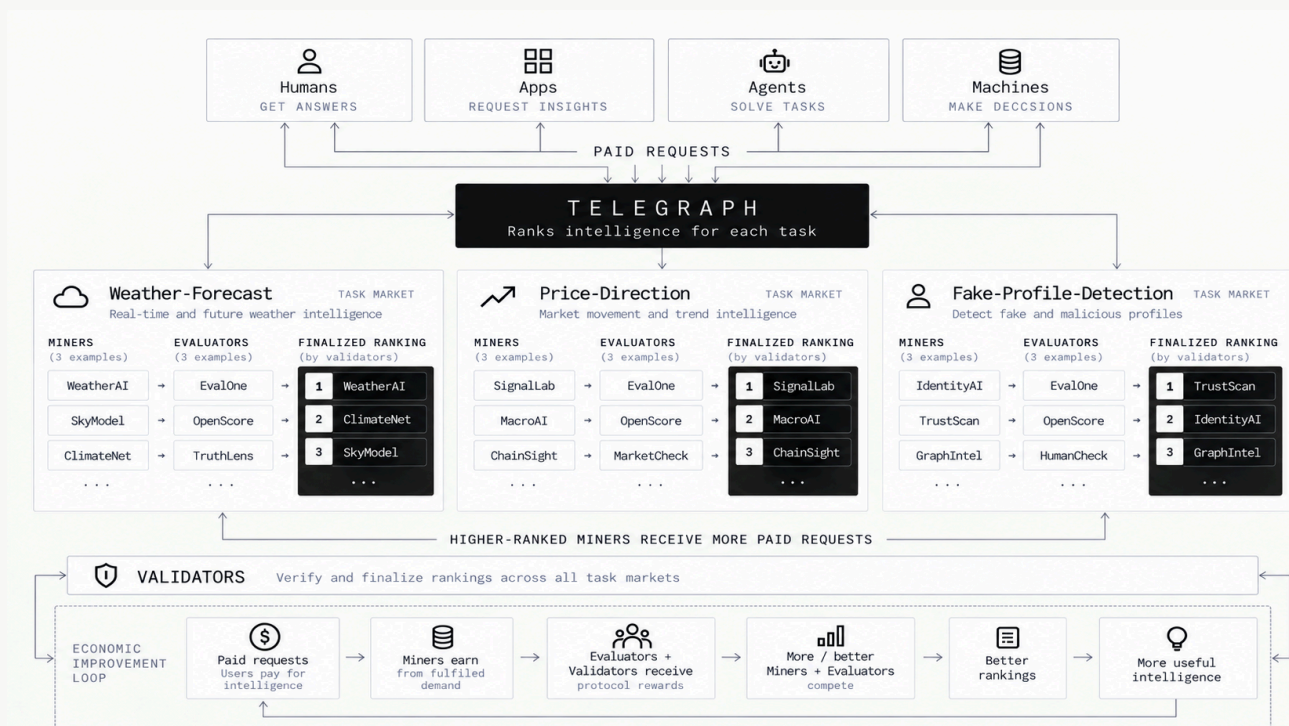


Figure 1. Telegraph connects task-specific intelligence markets, shared verification, real demand, and continuous competition.

Intelligence is organized around specific tasks

The fundamental organizing unit of Telegraph is the Intent.

An Intent defines a category of machine-intelligence task. Examples may include weather forecasting, deepfake detection, fraud analysis, search, classification, risk scoring, or another machine-readable job.

Each Intent creates its own competitive environment. It maintains the Miners capable of serving that task, the Evaluators capable of measuring them, one current Canonical Evaluator, and one current finalized Miner Ranking.

This keeps intelligence specific to the job being performed.

A weather system competes within weather-related Intents. A deepfake detector competes where deepfake detection is required. A specialist search system competes where its capabilities are relevant.

Different forms of intelligence can therefore coexist within one global protocol while being measured according to the tasks they actually perform.

Miners provide the intelligence

The intelligence supply is provided by Miners.

A Miner may expose an AI model, inference service, API, dataset, algorithm, search system, analytical service, tool, or another machine-callable capability compatible with an Intent. Participants register the Intents they can serve and enter the same evaluation and ranking process as the existing supply.

Performance determines ranking position. Ranking position influences routing opportunity.

A newly registered Miner can outperform existing participants, rise through the ranking, receive greater opportunity to serve Consumer demand, and earn revenue from fulfilled intelligence delivery.

This makes intelligence supply continuously competitive.

New technologies can enter existing Intents through the same protocol interfaces, allowing the available intelligence to change as better systems are developed.

Evaluators determine how intelligence is measured

Telegraph treats evaluation as a second competitive layer.

For every active Intent, independent Evaluators define methods for measuring Miner performance. An Evaluator combines evaluation logic with any task-specific material required by its method, such as examples, reference information, labels, rubrics, test cases, adversarial cases, or other evaluation inputs.

Candidate Evaluators compete through standardized workloads.

The leading eligible candidate then proceeds through independent hidden verification. Validators execute its committed implementation against protocol-selected material that was unavailable during ordinary candidate competition.

A qualifying candidate becomes the Canonical Evaluator for that Intent.

Canonical status remains open to future competition. A stronger Evaluator can enter, demonstrate better performance, satisfy the qualification process, and become the new Canonical Evaluator.

The system therefore contains two independent competitive processes:

Miners compete to provide stronger intelligence.

Evaluators compete to provide stronger methods for measuring that intelligence.

Improvement can occur on either side without requiring the other side to be redesigned.

Validators turn evaluation into finalized ranking state

The current Canonical Evaluator provides the measurement method used during Miner-ranking rounds.

A ranking round establishes a common evaluation context for the eligible Miners of an Intent. Participating Validators independently execute the Canonical Evaluator against the committed evaluation basis.

Their Local Scores move through the protocol-defined aggregation process to produce Canonical Scores. Miners are then ordered deterministically.

Byzantine fault-tolerant consensus finalizes the resulting Miner Ranking.

Each Intent therefore has a current finalized representation of measured Miner performance.

When performance changes, subsequent ranking rounds can change that ordering. Improving Miners can rise, degrading Miners can fall, and newly registered Miners enter under the same applicable rules.

Finalized rankings also remain subject to protocol monitoring and spot checks between ordinary ranking rounds.

Verifiable ranking and provenance

Telegraph makes the ranking and delivery path auditable through finalized protocol state and authenticated evidence.

Miner performance is measured through the applicable evaluation process, Validator consensus finalizes the resulting ranking state, and provenance evidence can bind the Miner identity, Intent, request, response, and protocol context where authenticated transcripts are required.

These commitments allow Consumers and downstream systems to reconstruct how a Signal was produced and which finalized state governed its routing.

Semantic quality is represented through the applicable evaluation method and the requirements of the consuming application. Provenance, deterministic execution, and finality provide a verifiable basis for relying on the process that selected and delivered the intelligence.

Finalized rankings drive real-time consumption

Consumers use the finalized ranking state produced by this process.

A Consumer may be a human-facing application, an autonomous agent, a smart contract, a service, or another machine system.

When a request enters Telegraph, it is resolved to the relevant Intent. The routing layer reads the current finalized Miner Ranking and the eligibility state for that request.

Routing is ranking-based and probabilistic. Higher-ranked eligible Miners receive greater routing opportunity under the current routing policy.

The selected Miner performs the task and produces the resulting machine-consumable intelligence as a Signal.

Ranking production and live intelligence delivery operate as separate processes.

Ranking continuously establishes an authoritative state in the background. Consumer requests use the latest finalized state when intelligence is needed.

This architecture allows the same ranking system to support synchronous requests, persistent streams, subscriptions, continuous intelligence production, and autonomous machine activity.

Consumption remains simple and task-specific

Applications and agents consume the intelligence produced by the network.

They can rely on the current ranking for a specific Intent rather than independently discovering every provider, integrating every service, constructing their own evaluation infrastructure, and continuously monitoring the entire market.

A weather agent can consume ranked weather intelligence.

A security agent can consume ranked deepfake-detection intelligence.

A financial system can consume ranked risk intelligence.

A robot or autonomous system can consume different Intents as its tasks change.

Each decision remains specific to the capability being requested while the underlying coordination occurs through the same Telegraph protocol.

This creates a simple relationship between global coordination and local consumption:

many task-specific intelligence markets, one shared coordination layer.

Consumers participate through request-response interfaces, persistent streams, subscriptions, and other supported delivery mechanisms while the ranking system evolves independently underneath them.

The system is produced by complementary peers

Four primary roles create the complete protocol cycle.

Miners supply intelligence.

Evaluators measure that intelligence and compete over the measurement process.

Validators verify protocol execution and finalize ranking state.

Consumers use the resulting intelligence and create demand.

Each role contributes a different part of the system.

Supply gives the protocol intelligence to rank.

Evaluation makes heterogeneous supply measurable and comparable.

Validation converts the ranking process into finalized shared state.

Consumption converts that state into useful activity and economic demand.

Telegraph connects these roles through common interfaces, commitments, ranking rules, routing rules, accounting, and settlement.

The quality of the resulting network therefore emerges from participation across these complementary roles.

A participant that develops stronger intelligence can introduce a better Miner.

A participant that develops a stronger way to measure intelligence can introduce a better Evaluator.

Consumers automatically benefit from later finalized rankings through the same protocol interfaces.

The protocol remains the coordination mechanism while the participant network supplies the intelligence, evaluation methods, verification work, and demand.

Economic incentives connect quality to demand

Telegraph connects measured performance to economic opportunity.

Paid Consumer activity is denominated in USDC.

Miner compensation comes from fulfilled paid Consumer demand. Higher ranking provides greater routing opportunity, creating greater opportunity to serve paid requests and earn revenue.

At genesis, 2 percent of successfully paid Consumer activity is allocated to the Protocol Treasury and 98 percent is attributed to Miner settlement.

Canonical Evaluators receive the applicable protocol rewards associated with evaluation activity. Validators receive rewards for verification, consensus, and state finalization.

The economic system therefore connects participation to useful activity:

Consumers create demand.

Ranking directs that demand across measured supply.

Useful supply gains greater opportunity to earn.

Evaluation and verification receive incentives for maintaining the system that produces trusted ranking state.

The economic flywheel is anchored to paid Consumer demand. Miner revenue comes from fulfilled paid Consumer activity, while protocol-generated evaluation, ranking, spot-check, bootstrap, grant, and free activity remain outside Qualified Consumer Demand.

Evaluator rewards are additionally bounded by the irreversible protocol fees generated by the corresponding paid demand. This keeps demand-dependent evaluation incentives tied to external economic activity, while Validators are rewarded for verification, consensus, and finalization under the protocol emission model.

As useful intelligence attracts demand, demand creates revenue opportunity, revenue opportunity can attract additional supply, and stronger supply increases the value of accurate evaluation. Because fulfilled intelligence is priced in USDC and Miner-attributable settlement value is used to acquire MACHINA on the open market, paid consumption creates direct settlement demand for MACHINA. The economic layer closes the loop between measured performance, commodity value, MACHINA, and sustained participation.

Real-time intelligence and economic settlement operate on separate cadences

Telegraph separates intelligence delivery from economic settlement.

A Signal can be delivered as soon as the applicable request, session, or job conditions are satisfied.

The protocol supports synchronous x402 requests, pre-funded high-frequency sessions, managed access, and direct on-chain jobs. These delivery paths converge into the same accounting and canonical Ethereum settlement architecture.

High-frequency activity can be accounted for separately from final settlement, allowing intelligence delivery to continue without requiring an Ethereum transaction for every request, response, stream event, or acknowledgement.

Economic activity is aggregated and settled through the canonical Ethereum settlement system.

This separation allows the system to combine real-time machine activity with verifiable economic settlement at a cadence appropriate to each.

The intelligence network continuously evolves

Telegraph maintains an evolving competitive state.

Miner Rankings change as Miner performance changes.

Canonical Evaluators change as stronger evaluation methods enter and qualify.

Evaluation material can expand.

New Miners can introduce new forms of intelligence.

New Intents can create markets for new categories of machine capability.

Each finalized ranking represents the current measured state of its Intent under the applicable evaluation process.

Later ranking rounds measure the market again.

The protocol therefore creates an environment in which both core competitive surfaces remain open:

the intelligence can improve, and the method used to measure the intelligence can improve.

Telegraph continues to provide the coordination layer while the underlying participant network evolves.

The unified flywheel

These mechanisms form one unified intelligence and economic flywheel.

- 1. More and better intelligence supply enters an Intent.**
- 2. Evaluators measure performance, making heterogeneous supply comparable.**
- 3. Finalized rankings order that supply by measured performance.**
- 4. Validators verify execution and finalize the ranking state.**
- 5. Paid Consumer demand is routed through the ranked supply.**

6. Fulfilled intelligence gains economic value through paid consumption.

7. Miner-attributable USDC is used to acquire MACHINA through settlement, linking intelligence consumption to MACHINA demand.

8. Economic opportunity attracts stronger supply and stronger evaluation.

Later ranking rounds measure the changed market, rankings update, demand redirects, and the cycle repeats.

The cycle therefore compounds rather than terminating at delivery.

Better supply increases the value of accurate evaluation.

Better evaluation improves the protocol's ability to distinguish available supply.

Better rankings improve the state used to route demand.

Paid demand rewards useful intelligence and creates settlement demand for MACHINA.

New competition begins from that updated economic state.

This is the core mechanism through which Telegraph turns independently produced intelligence into an evolving commodity market ranked by performance and driven by demand.

From specific Intents to global machine intelligence

The same architecture can operate across many independent Intents.

Each Intent remains specific enough to measure intelligence meaningfully while the protocol provides a shared mechanism for participation, evaluation, verification, ranking, routing, and settlement.

As new tasks emerge, new Intents can organize intelligence around them.

As new providers appear, they can enter the relevant markets.

As evaluation improves, Canonical Evaluators can change.

As Consumer demand expands, more applications, agents, machines, smart contracts, and autonomous systems can consume the resulting ranked intelligence.

The scalability of Telegraph comes from repeating the same coordination architecture across an expanding set of specific intelligence markets rather than requiring a central operator to construct every intelligence source or evaluation system.

The resulting network can therefore grow along several dimensions at once:

more Intents,

more intelligence providers,

more evaluation methods,

more Consumers,

more autonomous machine activity,

while preserving task-specific ranking at the point where intelligence is actually consumed.

Telegraph remains the common protocol connecting them.

The complete system

Telegraph coordinates the full lifecycle of ranked machine intelligence:

Intents organize tasks.

Miners supply intelligence.

Evaluators measure it.

Validators verify and finalize the ranking.

Routing applies that finalized state to real demand.

Consumers receive task-specific Signals.

Paid demand rewards useful supply.

New supply and new evaluation methods enter.

The rankings evolve.

Demand is routed again.

The cycle continuously connects intelligence, measurement, verification, consumption, and economics.

As participation expands, Telegraph can coordinate an increasingly broad universe of machine-callable intelligence while preserving competition at the level where quality actually matters: the specific task being performed.

Many intelligence providers.

Many evaluation methods.

Many task-specific rankings.

Many applications, agents, machines, and autonomous systems.

One shared protocol connecting them.

An intelligence commodity emerges from many task-specific markets operating through one continuously evolving peer-to-peer protocol, with performance measured locally and economic value connected globally.

1. TELEGRAPH: AN OVERVIEW

1.1 A Peer-to-Peer Ranking Protocol

Machine intelligence is distributed across independently operated models, APIs, datasets, algorithms, search systems, analytical services, tools, and other machine-callable systems. Performance varies by task and changes as new systems enter the market and existing systems improve.

Applications and autonomous agents using these systems must decide which providers to use, integrate with them, measure their performance, monitor changes, and replace weaker providers. Telegraph provides a common protocol for this process.

For each Intent, Miners compete to supply intelligence and Evaluators compete to measure Miner performance. The current Canonical Evaluator produces the evaluation results used by the Miner-ranking mechanism. Validators verify the required protocol execution and finalize the resulting ranking state. Consumers use that state to access intelligence through Telegraph.

Telegraph produces verified rankings. Ranking state is the finalized result of the protocol-defined evaluation, aggregation, verification, and consensus process.

1.2 Intents

An Intent defines a category of machine-intelligence task supported by the protocol. Intents allow intelligence supply and evaluation to be organized around a specific task rather than around a particular model, vendor, or implementation.

Each active Intent maintains the state required to organize competition for that task, including an eligible Miner Set, an eligible Evaluator Set, one current Canonical Evaluator, one current finalized Miner Ranking, and the commitments required by the applicable evaluation and ranking procedures.

Miners register for the Intents they are able to serve. Evaluators register for the Intents they are able to evaluate. A participant may support more than one Intent, and several independent participants may compete within the same Intent.

1.3 Intelligence Supply

An Intelligence Provider, referred to in the protocol as a Miner, registers one or more Intents and supplies machine-consumable intelligence.

The protocol does not define the internal architecture of a Miner. A Miner may expose an AI model, inference service, API, dataset, algorithm, search system, analytical service, tool, or another machine-callable capability compatible with an Intent. Miner participation is governed by the applicable registration, interface, bond, and eligibility requirements.

This design allows new forms of intelligence to enter existing Intents without changing the core protocol. A newly registered Miner enters the same evaluation and ranking process as existing participants and receives routing opportunity according to measured performance.

1.4 Evaluation

The quality of a Miner Ranking depends on the evaluation mechanism used to measure the Miners. Telegraph therefore treats evaluation as a competitive protocol layer.

An Evaluator defines how Miner responses are processed and how standardized evaluation results are produced for an Intent. Evaluation may also use task-specific material such as examples, reference information, expected outputs, rubrics, test cases, labels, adversarial cases, or other evaluation inputs.

Evaluator developers are responsible for constructing their evaluation logic and obtaining or producing the task-specific material required by that logic. Dataset discovery, collection, cleaning, labeling, synthetic generation, ground-truth construction, rubric design, fine-tuning, and equivalent Evaluator-development processes are performed by Evaluator participants.

Telegraph provides the common mechanisms through which Evaluators register, compete, commit executable artifacts and evaluation material, undergo hidden verification, become Canonical, and later be replaced. Evaluation can therefore evolve without Telegraph operating a bespoke evaluation pipeline for each Intent.

1.5 Canonical Evaluators

Each active Intent maintains one current Canonical Evaluator. Registered Evaluators first participate in a standardized candidate-ranking process using public, mock, historical, synthetic, or otherwise non-sensitive evaluation workloads. Candidate execution remains self-hosted during this stage, and the resulting standardized outputs produce a candidate ordering.

The highest-ranked eligible candidate becomes the presumptive Canonical Evaluator and enters a separate hidden-verification stage. Validators retrieve the candidate's committed executable, verify the relevant artifact and execution metadata, and execute the candidate against protocol-selected hidden evaluation material.

A candidate that satisfies the applicable qualification requirements becomes or remains Canonical. If it fails, verification advances to the next-ranked eligible candidate. If no challenger passes, the previously verified Canonical Evaluator remains active. New Evaluators may enter later competitions and replace the incumbent.

1.6 Miner Rankings and Routing

The current Canonical Evaluator is used to measure Miner performance for its Intent. Miner-ranking rounds establish a common evaluation basis for eligible Miners and commit the information required to reproduce the applicable ranking computation.

Participating Validators independently execute the current Canonical Evaluator against the committed evaluation basis. Valid results are aggregated according to the protocol's ranking rules to produce Canonical Scores and a deterministic Miner Ranking. Byzantine fault-tolerant consensus finalizes the resulting ranking state.

Consumer requests are resolved to Intents and routed using the current finalized Miner Ranking for the relevant Intent. Higher-ranked eligible Miners receive greater routing opportunity under the applicable routing policy. Until a new ranking is finalized, the previous finalized ranking remains authoritative.

Consumer requests are not individually scored by Validators before delivery. Rankings are produced separately and continuously refreshed, requests are routed using the current finalized ranking.

1.7 Continuous Intelligence Delivery

Telegraph supports request-driven and continuous intelligence delivery. A Consumer may submit a request directly, or the Autonomous Engine may generate questions from configured information sources. Each request or generated question is resolved to an Intent and routed according to the current finalized Miner Ranking.

The selected Miner returns the machine-consumable intelligence delivered as a Signal. Signals may be returned through request-response interfaces, WebSocket streams, subscriptions, or other supported delivery mechanisms.

The Autonomous Engine consumes ranking state produced by the evaluation and consensus layers. It does not determine Miner Rankings or Canonical Evaluator status.

1.8 Economic Competition and Continuous Improvement

Telegraph connects measured performance to economic opportunity. Higher-ranked Miners receive greater opportunity to serve paid Consumer demand. Canonical Evaluators receive the protocol rewards associated with eligible evaluation activity. Validators receive protocol rewards for verification, consensus, and state finalization.

A stronger Miner may enter an Intent, rise in the ranking, receive more routed demand, and earn more revenue. A stronger Evaluator may enter the same Intent, outperform the current Canonical Evaluator under the evaluator-selection mechanism, pass hidden verification, and become Canonical.

These processes allow intelligence supply and evaluation quality to change independently. Consumer demand, Miner competition, Evaluator competition, and updated ranking state form a continuing economic cycle in which protocol participation can improve the quality of ranked intelligence available to Consumers and make heterogeneous supply economically comparable as a commodity.

1.9 Protocol Agnosticism

Telegraph defines the interfaces and rules through which independently operated systems participate in a common ranking network. The protocol does not require a particular model architecture, AI provider, programming language, training method, data source, hosting provider, or computational technique for Miner participation.

The protocol also does not prescribe one permanent evaluation methodology for an Intent. Evaluators are independently developed and compete for Canonical status. New intelligence systems and new evaluation methods can therefore enter the protocol through the same architecture.

Telegraph coordinates registration, competition, commitments, evaluation, ranking, verification, consensus, routing, accounting, and settlement. The participating systems provide the intelligence, evaluation methods, and demand.

1.10 Peer Coordination and Intent-Local Intelligence

Telegraph is peer-to-peer because the protocol outcome is produced through the coordinated participation of independent roles rather than by Telegraph operating the intelligence system itself. For each Intent, Miners provide task-specific intelligence, Evaluators provide the methods used to measure that intelligence, Validators verify the required execution and finalize protocol state, and Consumers create the demand to which finalized rankings are applied. These roles are independent but complementary: intelligence supply requires a measurement process to become ranked; evaluation requires compatible supply to measure; finalized rankings provide the state from which demand can be routed; and paid Consumer demand creates the economic opportunity that rewards useful supply and supports continued protocol participation.

Telegraph coordinates these participants through common interfaces and protocol rules. It does not need to build every model, API, dataset, evaluation method, or consuming application itself. New supply, new evaluation methods, and new demand can enter independently, while the protocol provides the shared mechanism through which they become comparable, verifiable, routable, and economically connected.

This coordination remains specific to each Intent. A weather-forecasting request is resolved to an Intent containing supply and evaluation appropriate to weather forecasting, a deepfake-detection request is resolved to its own compatible Intent, another agent may depend on an entirely different task. Each Consumer therefore receives intelligence specific to the job it needs, even while Telegraph coordinates ranking across many Intents through the same global protocol.

The resulting system can scale globally without collapsing intelligence into one universal leaderboard. **Global ranked intelligence emerges from many task-specific markets operating through one shared protocol.**

2. DESIGN OVERVIEW

2.1 Protocol Layers

Telegraph protocol can be divided into several functional layers.

The Intent layer defines the tasks around which intelligence supply and evaluation are organized. The Miner layer contains independently operated systems that provide intelligence for those Intents. The Evaluator layer contains independently operated evaluation mechanisms that measure Miner performance and compete for Canonical status.

The ranking layer applies the current Canonical Evaluator to the eligible Miner Set and produces the Miner Ranking. The Validator layer verifies the required protocol execution and finalizes ranking and related state through Byzantine fault-tolerant consensus.

The routing and delivery layers use finalized ranking state to select eligible Miners and deliver Signals to Consumers. The economic layer records Consumer demand, compensates participants, and aggregates settlement to the canonical Ethereum settlement domain.

2.2 Intent and Miner Layer

For each active Intent, registered Miners form the supply set from which intelligence can be routed. Miner participation is permissionless subject to the applicable protocol requirements.

A Miner registration binds the participant's supported Intents, service endpoint, settlement identity, pricing information, credential commitments where required, and integration configuration to protocol state. The detailed registration fields and compatibility rules are defined in the protocol reference.

Miner performance determines ranking position and routing opportunity. New Miners enter the same ranking process as existing Miners and can displace weaker participants through measured performance.

2.3 Evaluator Layer

For each active Intent, registered Evaluators form an Evaluator Set. Each Evaluator combines evaluation logic with any task-specific evaluation material required by its method.

Candidate Evaluators remain independently operated during the initial competition stage. They process standardized non-sensitive workloads through their registered interfaces and return the results required by the candidate-ranking mechanism.

The candidate ordering identifies the presumptive winner. Canonical promotion then requires independent hidden verification of the candidate's committed executable. Candidate ranking and Canonical verification are separate protocol stages.

2.4 Evaluation Material and Hidden Verification

Evaluator developers and other eligible participants may contribute task-specific evaluation material for an Intent. Eligible material is committed before use and is subject

to protocol admission, versioning, duplicate, contribution-frequency, and contribution-influence rules.

Hidden Canonical-verification material is selected from committed evaluation state using protocol randomness fixed after the applicable candidate and evaluation-state commitments. The hidden sample is unavailable to candidate-operated infrastructure before verification.

The protocol bounds the effective sampled influence of individual contributors. During early participation the allowed share adapts to the number of eligible independent contributors and converges to a maximum effective share of 2 percent once at least 50 independent contribution domains participate. The detailed rule is defined in the protocol reference.

2.5 Shared Foundation Model and Evaluator Lifecycle

Where required by the applicable Evaluator architecture, Telegraph may use a shared Foundation Model as a common computational backbone for Intent-specific Evaluators.

An Evaluator that depends on this backbone is defined by the combination of the active Foundation Model version, the Evaluator artifact, and the applicable evaluation input. Evaluator competition and Foundation Model training are separate protocol lifecycles.

A new Evaluator may replace the current Canonical Evaluator without retraining the Foundation Model. A successful Foundation Model retraining event creates a new Foundation Model version. Because changing the shared backbone may alter the behavior of an unchanged Evaluator, affected Canonical Evaluators are revalidated before the new Foundation Model version becomes authoritative for those Intents.

The Foundation Model does not determine which Evaluator becomes Canonical. Canonical selection remains external to the backbone and follows the protocol-defined competition and verification process.

2.6 Miner Evaluation and Ranking

Each Miner-ranking round fixes the protocol context required to compare eligible Miners under equivalent conditions. This context includes the Intent, round identifier, eligible Miner Set, evaluation request or batch, response-collection rules, current Canonical Evaluator, and deterministic execution configuration.

A ranking-round Leader coordinates the evaluation request, response collection, required provenance evidence, and construction of the committed evaluation package. The Leader does not select the Canonical Evaluator and does not determine Miner scores at its discretion.

Participating Validators execute the current Canonical Evaluator against the same committed evaluation basis. Local Scores proceed through the protocol's commit-reveal procedure and are aggregated using the finalized Validator stake snapshot and the protocol-defined stake-weighted median. Miners are then ordered deterministically to produce the candidate Miner Ranking.

Byzantine fault-tolerant finalization converts the candidate ranking into protocol state. If the required quorum is unavailable, the new ranking does not finalize and the previous finalized ranking remains authoritative.

2.7 Validator and Consensus Layer

Validators are staked protocol nodes responsible for verification, consensus, and state finalization. Their responsibilities include reproducing required ranking computations, verifying candidate Evaluator behavior where required, validating protocol state transitions, signing protocol commitments, and participating in Byzantine fault-tolerant consensus.

The Active Validator Set is determined by stake competition. At genesis, the Validator Cap is 64. Protocol finality requires a quorum of $\text{floor}(2 \times |V| / 3) + 1$ Validators, which is 43 when the Active Validator Set contains 64 Validators.

The quorum requirement is not reduced during network partitions or periods of reduced Validator availability. If the required quorum is unavailable, new protocol state does not finalize and the previous finalized state remains authoritative.

2.8 Routing and Autonomous Engine

Finalized Miner Rankings determine routing opportunity. Before an Intent has a finalized Miner Ranking, routing is distributed uniformly among eligible registered Miners. After ranking state exists, routing becomes probabilistic and ranking-based.

Routing can operate within a restricted eligible set where a Consumer requires particular confidentiality, execution, security, or policy characteristics. Operational failover may select another eligible Miner when the selected Miner cannot complete a request, without modifying the finalized Miner Ranking.

The Autonomous Engine supports both direct requests and continuous intelligence production. It resolves requests or generated questions to Intents, reads current ranking and eligibility state, selects an eligible Miner under the routing policy, receives the response, and produces the resulting Signal.

2.9 Economic and Settlement Layer

Consumer-denominated protocol activity is accounted for in USDC. Miners receive no MACHINA from protocol emissions and are compensated exclusively from fulfilled paid Consumer demand. Validators, Evaluators, and the Protocol Treasury receive the fixed emission allocations defined by the protocol.

The protocol applies a 2 percent fee to paid Consumer activity. The remaining 98 percent is allocated to Miner settlement. USDC allocated to Miner settlement is processed through the protocol's economic mechanism on Ethereum, and the resulting MACHINA forms the basis for Miner entitlements. Because each fulfilled delivery carries a USDC-denominated price and Miner-settlement USDC is used to acquire MACHINA on the open market, paid intelligence consumption creates protocol-mediated demand for MACHINA. MACHINA is therefore the protocol-native economic unit into which the value of consumed intelligence is settled.

Real-time intelligence delivery does not wait for Ethereum settlement. Requests, Signal delivery, usage accounting, and high-frequency streaming can occur off-chain while economic activity is aggregated and committed periodically.

Participant entitlements use cumulative accounting. Finalized entitlement commitments allow Miners, Validators, and Evaluators to claim the positive difference between their latest cumulative entitlement and amounts previously withdrawn.

2.10 Continuous Protocol Evolution

The principal competitive state of Telegraph is designed to change as participation changes. Miner Rankings can change as Miner performance changes. Canonical Evaluators can change as stronger evaluation methods enter and qualify. Foundation Model versions can change through retraining and trigger revalidation of dependent Evaluators.

Evaluation material can expand through independent contribution, new Intents can introduce new categories of intelligence, and new forms of Miner technology can participate through the same interfaces.

Telegraph remains the common coordination layer through which independently operated participants compete, produce evaluation and ranking state, route demand, and settle value.

3. NETWORK PARTICIPANTS

Telegraph has four primary protocol roles: Miners, Evaluators, Validators, and Consumers. MACHINA holders may also participate through delegated staking. Miners and Evaluators form the two competitive layers that determine the intelligence available for each Intent and the method used to measure it. Validators verify the required protocol execution and finalize state. Consumers create demand for the resulting intelligence.

The roles are independent. A participant may perform more than one role where the applicable protocol requirements permit it, but authority in one role does not grant authority in another. A Miner does not determine its own score, an Evaluator does not finalize protocol state, a Delegator does not control Validator execution, and a Consumer does not participate in ranking simply by purchasing intelligence.

Role	Primary function	Economic relationship
Miner	Supplies machine-callable intelligence for one or more Intents.	Receives routed paid demand according to ranking and eligibility.
Evaluator	Measures Miner performance and competes for Canonical status.	Eligible Canonical Evaluators receive protocol rewards.
Validator	Verifies protocol execution, participates in consensus, and finalizes state.	Receives Validator emissions for protocol work.
Consumer	Requests or subscribes to intelligence through Telegraph.	Pays for fulfilled intelligence in USDC.
Delegator	Delegates MACHINA to an active Validator.	Receives a proportional share of Validator rewards after commission.

3.1 Miners

A Miner is an Intelligence Provider that registers one or more Intents and supplies machine-consumable intelligence. The protocol does not require a Miner to use a particular model, data source, software architecture, or hosting environment.

A Miner may expose an AI model, inference service, API, dataset, algorithm, search system, analytical service, tool, or another machine-callable capability compatible with an Intent. This allows the same protocol to rank heterogeneous sources of intelligence according to their measured performance for a defined task.

Miner participation is permissionless subject to the applicable registration, bond, interface, and security requirements. A Miner declares the Intents it supports and the operational information required for Telegraph to call its service and settle fulfilled demand.

For each Intent, registered Miners form the supply set from which ranking and routing operate. Miner responses are evaluated through the current Canonical Evaluator for that Intent. Measured performance determines ranking position, and ranking position influences routing opportunity.

Miners do not participate in Validator consensus and do not determine their own ranking scores. New Miners may enter an existing Intent and compete under the same protocol rules as existing participants.

Miner compensation is funded by paid Consumer demand. The protocol does not allocate MACHINA emissions directly to Miners. USDC attributable to fulfilled Consumer activity is processed through the settlement mechanism, and MACHINA acquired through that process forms the basis for Miner entitlements.

3.2 Evaluators

An Evaluator measures Miner performance for one or more Intents. Evaluators form a separate competitive layer from Miners: Miners compete to supply intelligence, while Evaluators compete to determine how the performance of that intelligence is measured.

For each Intent, registered Evaluators form an Evaluator Set. One Evaluator is recorded as the current Canonical Evaluator. Canonical status is obtained through the protocol's Evaluator-competition and verification process and can be replaced when a qualified challenger performs better and passes the required verification.

An Evaluator consists of evaluation logic and, where required by that logic, task-specific evaluation material. Evaluation material may include examples, reference information, expected outputs, rubrics, test cases, labels, adversarial cases, ground truth, or other inputs used to measure performance.

Evaluator development is performed by the participant operating the Evaluator. Dataset discovery, collection, cleaning, labeling, synthetic generation, ground-truth construction, rubric design, fine-tuning, and comparable development work are not centrally performed by Telegraph.

The current execution architecture represents Evaluator logic through a committed WASM artifact. Candidate Evaluators remain self-hosted during the initial competition stage. Canonical promotion requires a separate hidden-verification stage in which the committed implementation is independently executed by the assigned Validator subset against protocol-selected hidden evaluation material.

Canonical Evaluators receive rewards from the Evaluator emission allocation according to the economic rules of the protocol.

3.3 Validators

A Validator is a staked protocol node responsible for verification, consensus, and state finalization. Validators verify the computations, commitments, and state transitions required by the protocol and participate in Byzantine fault-tolerant consensus over the resulting state.

Validator responsibilities include reproducing required Miner-ranking computations, verifying candidate Evaluator behavior where assigned, validating routing-revocation and other protocol state transitions, signing protocol messages and commitments, and participating in finalization.

Validators verify the execution of the ranking system. Miner quality is measured through the Evaluator and ranking mechanisms, it is not independently judged by Validators.

The right to operate an active Validator is established through a Validator Profile. Validators compete for membership in the Active Validator Set by staked MACHINA. At genesis, the Validator Cap is 64.

Each Validator Profile separates ownership and online operation through a Manager Key and an Operator Key. The Manager Key controls the Validator Profile and staked MACHINA and is used for staking, unbonding, reward claims, and Operator Key authorization. The Operator Key performs the online signing, verification, and consensus functions of the Validator and cannot withdraw the Validator's staked MACHINA.

Validator penalties affect emissions, reward eligibility, and participation status according to the applicable protocol rules. Validator principal is not reduced as a protocol penalty.

3.4 Consumers and Agents

A Consumer is an application, autonomous agent, smart contract, service, or human-facing interface that requests or subscribes to intelligence through Telegraph. An Agent is a Consumer implemented as autonomous or programmatic software.

Consumer requests are resolved to Intents and routed according to the current finalized Miner Ranking and the applicable eligibility and routing rules. Consumers may interact with Telegraph through request-response interfaces, persistent streams, subscriptions, and other supported delivery mechanisms.

Consumers are not required to participate in Miner or Evaluator competition. They consume the intelligence produced by the network and create the paid demand that connects ranking performance with economic opportunity.

Paid Consumer activity is denominated and accounted for in USDC. Supported payment paths may include x402 request-response payments, pre-funded escrow, and session or streaming accounting. The payment and settlement mechanisms are described later in this document.

3.5 Delegators

A MACHINA holder may delegate stake to an active Validator. Delegated stake contributes to the Validator's stake weight and allows Validators to aggregate stake when competing for membership in the Active Validator Set.

Validator rewards attributable to delegated stake are accounted for proportionally under the applicable commission rules. The Validator retains its configured commission and the remaining delegated reward becomes claimable by the Delegator.

Delegator rewards use pull-based claiming so the protocol does not need to distribute individual payments to every Delegator during each reward cycle.

Delegation does not grant authority over Validator execution, Evaluator selection, ranking computation, or consensus. It is an economic participation mechanism within the Validator staking system.

3.6 Participation Across Roles

The protocol permits a participant to operate in more than one role where the applicable registration, eligibility, security, and conflict requirements are satisfied. The roles remain logically separate even when they are operated by the same participant.

Authority is derived from the rules of the role being exercised. Operating a Miner does not grant ranking authority. Operating an Evaluator does not grant consensus authority. Delegated stake does not grant control over Validator execution. Consumer demand does not determine ranking outside the protocol-defined evaluation and routing mechanisms.

This separation allows participation to remain open while preserving the boundaries required for competition, verification, and finalization.

4. INTENTS AND INTELLIGENCE SUPPLY

Intents are the organizing unit of Telegraph. An Intent identifies a machine-intelligence task for which independent Miners may supply compatible services and for which the protocol can maintain evaluation, ranking, routing, and settlement state. The Intent interface allows heterogeneous intelligence systems to compete under a common task definition without requiring the protocol to prescribe how those systems are built.

Miner participation is permissionless. A Miner registers the Intents it supports, the operational configuration required to call its service, its settlement identity, and the commitments required by the protocol. Registration creates a stable relationship between the Miner's on-chain identity and the off-chain service that supplies intelligence.

4.1 Intents

An Intent is a uniquely identified category of machine-intelligence task. It defines the interface and execution expectations under which Miners can provide intelligence and Evaluators can measure Miner performance.

The protocol does not restrict an Intent to a particular technology or application domain. An Intent may be served by models, APIs, datasets, algorithms, search systems, analytical services, tools, or other compatible machine-callable systems. The same Intent can therefore contain heterogeneous providers whose implementations differ while their outputs remain comparable under the applicable evaluation process.

Each active Intent maintains the protocol state required for competition and routing, including its registered Miner Set, registered Evaluator Set, current Canonical Evaluator, current finalized Miner Ranking, and the commitments required by the applicable evaluation and ranking procedures.

4.2 Intent Identity and Versioning

Intent identity is versioned. Telegraph uses semantic versioning so that integrations can rely on a stable task definition and interface for the lifetime of a compatible Intent version.

A material change to an Intent's interface, output structure, execution requirements, or other compatibility-sensitive behavior produces a new Intent version. Existing registrations remain bound to the version under which they were created unless they are explicitly updated according to the applicable registration rules.

This versioning model allows the protocol to evolve an Intent without silently changing the assumptions made by Miners, Evaluators, Consumers, or downstream smart contracts.

4.3 Miner Registration

To serve an Intent, a Miner submits an on-chain registration and the required registration bond. Any address satisfying the protocol requirements and providing a valid configuration may register for an active Intent.

Registration binds the Miner's operational service configuration to its on-chain identity and economic settlement state. The registration records the supported Intents, the Miner settlement address, the minimum Signal price declared for each supported Intent, any

required credential commitment, and the cryptographic commitment and reference for the Miner's declarative configuration.

The Miner settlement address is the EVM address to which the Miner's MACHINA settlement entitlement is assigned. The minimum Signal price establishes the lowest USDC price the Miner is willing to accept for a supported Intent, subject to the protocol pricing rules. At genesis, the protocol-wide minimum floor is 0.01 USDC, and a Miner may declare a higher floor.

Where a Miner requires authenticated access, the registration may include a content-addressed hash of the encrypted credential package. The commitment binds the credential package to the Miner's registered identity without exposing the credential itself through registration state.

4.4 Miner Configuration and Output Mapping

A Miner describes its operational interface through the protocol's unified YAML configuration. The configuration contains the information required for Telegraph to call the Miner, submit requests, interpret responses, and apply any deterministic output transformation required by the supported Intent.

Where a Miner response must be consumed by an EVM contract, the configuration can map selected response fields into the protocol's canonical OnChainData structure. The structure separates addresses, unsigned integers, strings, and booleans into deterministic slot arrays.

The mapping specifies the array type, slot index, field name, and source path used to extract each committed output. Slot positions remain stable for a registration version. A change that alters the committed slot layout requires an updated registration or a new Intent version according to the applicable compatibility rules.

This gives downstream contracts a deterministic decoding interface while leaving the Miner free to use any internal architecture capable of producing a compatible response.

```
OnChainData = { addresses[], integers[], strings[], bools[] }
```

4.5 Miner Eligibility and Pricing

Registration makes a Miner eligible to participate subject to the protocol's current registration, bond, interface, security, and operational requirements. Eligibility for a specific request may be further constrained by the routing policy, Consumer requirements, confidentiality requirements, or other protocol-defined conditions.

The price ultimately paid for a request is determined by the protocol pricing mechanism. A Miner's registered minimum price acts as a lower bound for the applicable service while allowing the protocol to apply the demand, routing, and payment rules defined later in this document.

Miner eligibility and ranking are separate concepts. Registration permits a Miner to compete. The finalized Miner Ranking determines its relative routing opportunity once ranking state exists.

4.6 Miner Deregistration

A Miner may exit an Intent through the protocol's deregistration procedure. Deregistration begins a seven-day unbonding period for the Miner registration bond, subject to unresolved protocol obligations.

A deregistering Miner receives no new routed Consumer demand during the unbonding period. Obligations arising from the previously active registration remain enforceable, including evaluation, accounting, and settlement obligations associated with requests that were already accepted or fulfilled.

Settlement arising from activity completed before deregistration continues through the normal settlement process. Deregistration therefore stops new routing without invalidating economic obligations already created by the Miner's prior participation.

4.7 Protocol Flows and Application Workflows

Telegraph distinguishes a protocol-level Flow from a higher-level Workflow.

A Flow is a single intelligence operation. It resolves a request to an Intent, selects an eligible Miner using the current finalized Miner Ranking and routing policy, obtains the Miner response, records the usage required for economic accounting, and delivers the resulting Signal to the Consumer.

Payment authorization for a Flow may use x402, a session, escrow, or another supported payment mechanism. Signal delivery occurs independently from the cadence of final economic settlement, allowing the real-time path to remain fast while usage and settlement are aggregated separately.

A Workflow is a multi-step orchestration built above the protocol. It may contain state, memory, conditional execution, or several Telegraph Flows. Applications, Agents, and the Autonomous Engine may compose Flows into Workflows according to their own application logic.

The protocol therefore remains focused on Intent resolution, ranking-based routing, intelligence delivery, verification, accounting, and settlement. The interpretation, composition, storage, and downstream use of delivered Signals belong to the Consumer or application layer.

5. RANKING AND EVALUATION

Ranking and evaluation form the core intelligence-selection system of Telegraph. For each active Intent, Miners compete to supply intelligence and Evaluators compete to measure Miner performance. The protocol maintains one current Canonical Evaluator and one current finalized Miner Ranking for each Intent.

Telegraph produces verified rankings. Evaluators define the measurement process, Validators verify the required execution and finalize state, and Consumers use the resulting ranking for routing. The protocol does not require Telegraph to judge Miner quality manually or operate a custom evaluation pipeline for every Intent.

Both sides remain open to replacement. A stronger Miner can gain ranking position and routed demand. A stronger Evaluator can become Canonical after competition, qualification, and independent verification. The ranking system can therefore change as intelligence supply and evaluation methods improve.

Stage	Primary participants	Protocol result
Evaluator development	Evaluator participants	Evaluation logic and task-specific material
Candidate competition	Registered Evaluators	Candidate ordering
Hidden verification	Assigned Validator subset	Qualified Canonical Evaluator
Miner-ranking round	Miners and Validators	Canonical Scores
Ranking finalization	Active Validator Set	Finalized Miner Ranking
Routing and monitoring	Routing layer and Validators	Demand distribution and routing eligibility

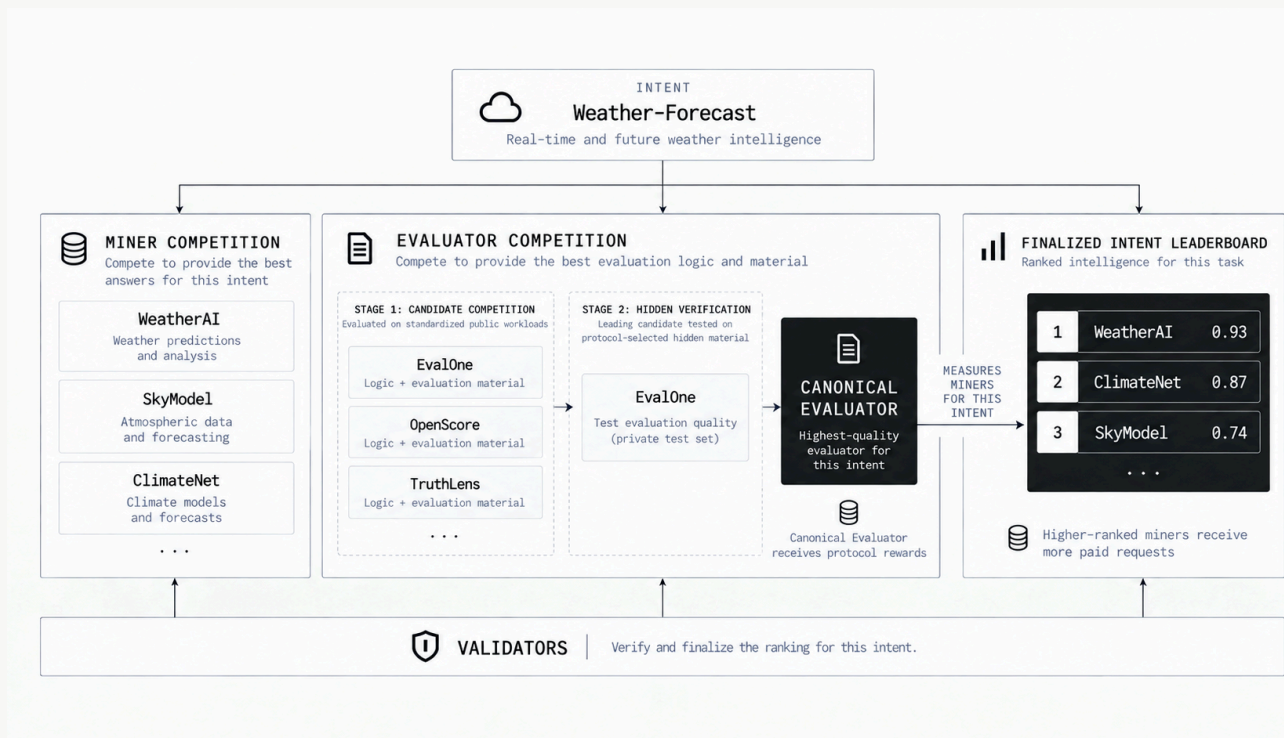


Figure 2. Miners and Evaluators compete independently within each Intent; the Canonical Evaluator measures Miner performance and Validators finalize the resulting leaderboard.

5.1 Evaluator Architecture

Evaluator participation is permissionless and specific to an Intent. A registered Evaluator commits the identity, endpoint, executable artifact, executable hash, version, deterministic runtime metadata, standardized interface, applicable evaluation-material commitments, economic recipient, registration bond, and eligibility state required for competition and independent verification.

WASM is the committed executable format used by the current verification architecture. Candidate Evaluators operate their own infrastructure during ordinary competition. A material change to Evaluator behavior requires a new executable commitment and version.

Evaluator registration creates participation friction through a bond, but the bond remains the registrant's principal. Failed qualification or other protocol penalties may affect unfinalized rewards, status, or eligibility, Evaluator registration principal is not reduced as a protocol penalty.

5.2 Evaluator Logic and Evaluation Material

An Evaluator has two related components. Evaluator logic defines how Miner performance is measured. Evaluation material provides any task-specific information required by that logic.

Evaluation material may contain task examples, reference information, known outputs, labels, rubrics, test cases, adversarial cases, structured evaluation inputs, ground truth, or other Intent-specific measurement material.

The Evaluator developer is responsible for constructing the Evaluator and obtaining or producing the material required by it. Dataset discovery, scraping, cleaning, labeling,

synthetic generation, ground-truth construction, rubric design, fine-tuning, and equivalent development work occur outside Telegraph.

Telegraph uses a common protocol interface across Intents. The protocol supplies generic mechanisms for registration, commitments, admission, competition, sampling, verification, ranking, and state finalization, it does not require a separately engineered Telegraph corpus or ground-truth pipeline for each Intent.

5.3 Evaluation Contributions

Evaluation material intended to participate in Canonical Evaluator selection is committed before use. Modifying committed material creates a new version and commitment.

Eligible contributions for an Intent form its committed evaluation-material state. The protocol applies generic mechanical admission controls such as schema validation, malformed-input rejection, size limits, duplicate detection, contribution-frequency limits, versioning, and minimum contribution requirements.

The protocol does not select a subjectively preferred contributor dataset. Canonical selection is driven by bounded contribution influence, unpredictable sampling, candidate competition, qualification rules, independent verification, and the continuing ability of stronger Evaluators to replace weaker ones.

$$\text{Evaluation_Commitment} = H(\text{Intent_ID} \parallel \text{Material_Hash} \parallel \text{Metadata_Hash} \parallel \text{Version})$$

5.4 Bounded Contribution Influence

No individual evaluation contributor may dominate the hidden material used to qualify a Canonical Evaluator. Community-created Intents use an adaptive maximum contribution share that supports bootstrapping with a small contributor set and converges to a mature-pool cap as contributor diversity increases.

Let C_i denote the number of eligible independent evaluation contributors for Intent i . The maximum effective sampled share is:

$$\text{MaxShare}_i = \max(0.02, 1 / C_i)$$

With four eligible contributors the maximum share is approximately 25 percent each, with ten it is 10 percent, with twenty-five it is 4 percent, and with fifty or more it is 2 percent. The limit applies to effective sampled influence, not simply to the raw amount of material submitted.

Genesis or protocol-bootstrap Intents may begin with predefined evaluation material before sufficient permissionless contributions exist. Bootstrap material receives no permanent preference once ordinary contribution participation becomes available. Contribution controls also account, where practical, for economically linked contribution domains so trivial identity splitting cannot multiply influence.

5.5 Candidate Ranking

The first stage of Evaluator competition ranks candidate implementations without requiring Validators to execute every candidate. Candidate-ranking workloads may use public, mock, historical, synthetic, or other non-sensitive material appropriate to the Intent.

The workload is standardized for the competition event so candidate results are comparable. Candidate Evaluators process the workload on their own infrastructure and return the standardized results required by the evaluation specification.

The candidate results produce an ordering of registered Evaluators. This ordering determines which candidate is considered first for Canonical verification. Candidate-ranking success alone does not grant Canonical status.

$$\text{Candidate_Score_e} = \text{Aggregate}(\text{Result_e}, 1, \dots, \text{Result_e}, N)$$

5.6 Hidden Qualification and Canonical Verification

The highest-ranked eligible candidate becomes the presumptive Canonical Evaluator. Promotion requires performance on hidden evaluation material that was unavailable during the self-hosted ranking stage.

The hidden verification workload is selected from committed evaluation state using finalized protocol randomness after the applicable candidate executable and evaluation state have been committed. Qualification may require minimum hidden evaluation volume, minimum performance, minimum confidence, cross-batch stability, or another deterministic Intent-specific rule defined by the protocol.

A bounded rotating subset of the Active Validator Set performs the verification. At genesis, the verification subset contains 10 percent of the Active Validator Set, rounded upward. With 64 active Validators, seven Validators are assigned.

$$\begin{aligned} \text{VerificationSubsetSize} &= \text{ceil}(0.10 \times |V|) \\ \text{Verification_Seed} &= H(\text{Protocol_Randomness} || \text{Intent_ID} || \text{Evaluator_Round_ID}) \end{aligned}$$

The assigned Validators retrieve the candidate's committed executable, verify its artifact hash, version, and runtime metadata, derive the hidden sample, and execute the candidate locally inside the deterministic verification environment. The candidate operator does not receive the hidden sample before or during this verification.

A candidate that satisfies the qualification requirements becomes or remains Canonical. If it fails, verification proceeds to the next-ranked eligible candidate. If no challenger passes, the previously verified Canonical Evaluator remains active.

5.7 Canonical Evaluator State

Each active Intent maintains one current Canonical Evaluator. Canonical state binds the Intent, Evaluator identity, executable hash, Evaluator version, applicable Foundation Model version, evaluation-pool version, evaluation-engine version, evaluation specification, activation state, and the qualification metadata required to reproduce the Canonical decision.

A material change to the Evaluator artifact, Foundation Model version, evaluation-pool version, or evaluation-engine version creates a new execution context and is handled through the applicable validation or revalidation process.

Canonical status creates no permanent incumbency advantage. A new Evaluator may register, outperform the incumbent in candidate competition, pass hidden verification, and replace it.

5.8 Shared Foundation Model and Evaluator Lifecycle

Telegraph may use a shared Foundation Model as a common computational backbone for Intent-specific Evaluators. Where an Evaluator depends on this backbone, its behavior is determined by the active Foundation Model version, the Evaluator artifact, and the applicable evaluation input.

$$\text{Evaluator_Output} = f(\text{Foundation_Model_Version}, \text{Evaluator_Artifact}, \text{Evaluation_Input})$$

Evaluator competition and Foundation Model training are separate lifecycles. A new Evaluator can compete for Canonical status without retraining the Foundation Model, so a better Evaluator may replace the incumbent while the shared backbone remains unchanged.

Foundation Model retraining may occur separately using accumulated eligible training material and the applicable training process. A successful retraining creates a new Foundation_Model_Version.

Changing the shared backbone may change the behavior of an unchanged Evaluator. Activation of a new Foundation Model version therefore triggers revalidation of affected Canonical Evaluators before the new version becomes authoritative for those Intents.

An Evaluator that still satisfies the Canonical qualification requirements may remain Canonical under the new Foundation Model version. An Evaluator that no longer qualifies is replaced through the ordinary competition and Canonical-selection process. The Foundation Model does not determine which Evaluator becomes Canonical.

5.9 Miner-Ranking Rounds

Miner ranking is a separate process that uses the current Canonical Evaluator. For each ranking round, the protocol fixes the Intent, Round ID, eligible Miner Set, evaluation request or request batch, response-collection window, current Canonical Evaluator, applicable evaluation configuration, and deterministic execution context.

The eligible Miner Set is committed before ranking results are known. Eligible Miners are evaluated under equivalent conditions. Implementations may use batching, bounded concurrency, or distributed scheduling as long as equivalent evaluation inputs and collection rules are preserved.

$$\text{Eligible_Miner_Set_Root} = \text{MerkleRoot}(\text{Eligible_Miners})$$

5.10 Ranking-Round Leader and Response Integrity

One active Validator coordinates each Miner-ranking round. Leader selection follows a strict round-robin schedule over the ordered Active Validator Set. Each successive ranking round advances the Leader turn to the next active Validator in that order.

$$\text{LeaderIndex} = \text{Round_ID} \bmod |V|$$

The Leader coordinates the protocol-defined evaluation request, response collection, required provenance evidence, and construction of the committed evaluation package. The Leader does not select the Canonical Evaluator and does not determine Miner scores.

Where a Miner endpoint does not independently produce transferable authenticated evidence, the Leader produces zkTLS or another protocol-approved authenticated transcript for the ranking interaction. The evidence binds the authenticated Miner endpoint, Miner identity, Intent, ranking round, exact request, exact response, and applicable protocol context.

The ranking process commits both the eligible Miner Set and the valid responses received within the collection window. A timely valid response from an eligible Miner cannot be selectively omitted.

$$\text{Miner_Response_Root} = \text{MerkleRoot}(\text{Valid_Miner_Responses})$$

5.11 Ranking Evaluation Package

All Validators participating in a ranking round operate from the same committed evaluation basis. The Evaluation Package binds the Intent, Round ID, eligible Miner Set, request or request-batch commitment, valid Miner responses, provenance commitments, Canonical Evaluator identity and executable commitment, applicable evaluation configuration or material commitment, protocol version, and deterministic execution inputs.

Full evaluation data may remain off-chain where appropriate. The commitments required to reproduce and verify the ranking computation become part of the protocol state.

5.12 Independent Validator Evaluation and Commit-Reveal

Assigned Validators independently execute the current Canonical Evaluator against the same committed Miner evaluation inputs. A Local Score is the result of protocol execution, not a discretionary Validator judgment. Validators do not substitute alternative evaluation logic or modify the authenticated Miner response before execution.

$$\text{Local_Score}_{(v,m)} = E_i * (\text{Evaluation_Input}_m)$$

$$\text{Local_Score} \in [0,1]$$

Validators commit to their independently produced scores before observing the plaintext scores of other Validators. Each commitment binds the Validator, Intent, ranking round, Miner, Evaluation Package, Canonical Evaluator executable, Local Score, and a nonce. After the commit phase closes, the Validator reveals the Local Score and nonce, and the network verifies that the reveal reconstructs the earlier commitment.

Missing or invalid participation is handled by the applicable Validator liveness and incorrect-execution rules.

5.13 Canonical Scores and Deterministic Miner Ranking

A Miner requires at least the Byzantine quorum of distinct valid Validator score reveals before its Canonical Score may finalize. At the genesis Active Validator Set of 64, this threshold is 43.

$$\tau = \text{floor}(2 \times |V| / 3) + 1$$

Aggregation uses the finalized Validator stake snapshot associated with the ranking round. Valid Local Scores are ordered with their corresponding Validator stake weights. The Canonical Score is the first score for which cumulative represented stake exceeds one half of the stake represented by valid revealers.

$$\text{Canonical_Score_m} = \text{StakeWeightedMedian}(\text{Local_Scores_m})$$

Canonical Scores are converted into the Miner Ranking deterministically. Miners are ordered by descending Canonical Score, with ascending Miner ID used as the tie-break for equal scores. The ranking contains no discretionary Leader decision.

$$R_i = \text{Sort}(M_i, \text{Canonical_Score DESC}, \text{Miner_ID ASC})$$

5.14 Finalization and Continuous Ranking

The resulting Miner Ranking becomes active only after Byzantine fault-tolerant finalization. Finalized state commits the applicable Miner Ranking, Canonical Evaluator state, evaluation state, routing state, protocol sequence, and associated state roots.

If a replacement ranking round fails to finalize, the preceding finalized ranking remains active subject to the normal routing-eligibility rules.

Miner Ranking represents current measured performance. Each subsequent ranking round measures performance again and may change the ordering. Historical ranking position provides no scoring advantage in a later round. Improving Miners may rise, degrading Miners may fall, and newly registered Miners compete under the same applicable evaluation rules.

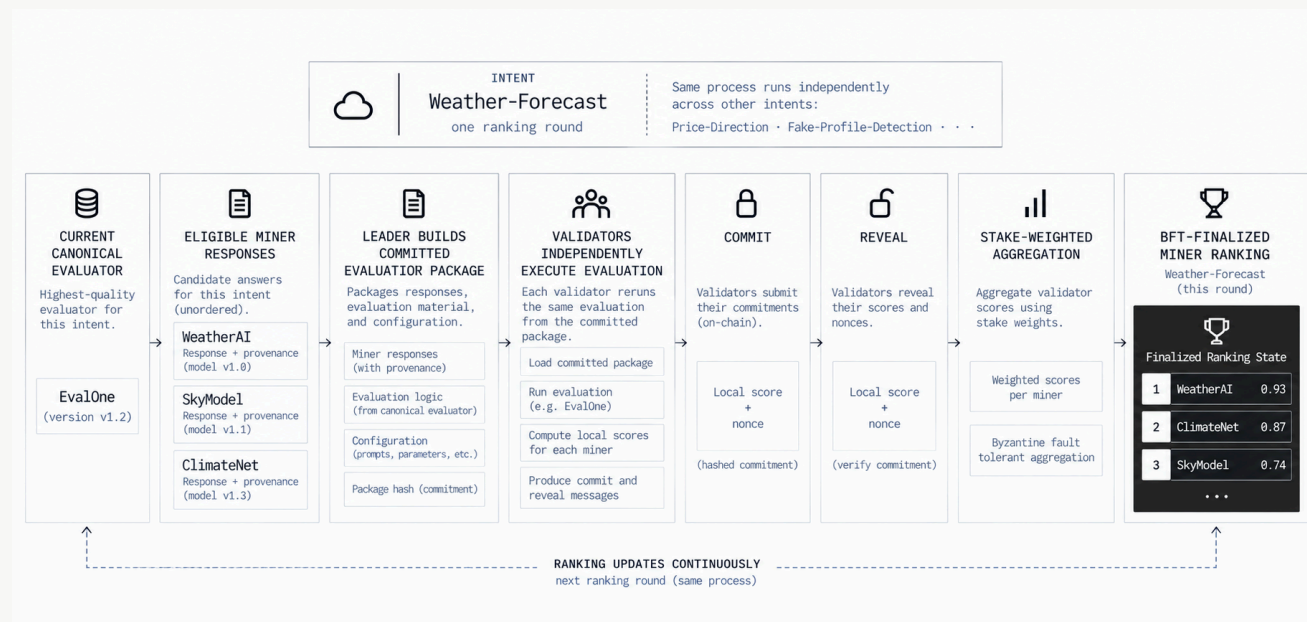


Figure 3. Each Intent repeatedly converts committed Miner responses into a BFT-finalized Miner Ranking through independent Validator execution, commit-reveal, stake-weighted aggregation, and deterministic ordering.

5.15 Ranking-Based Routing and Spot Checks

Consumer demand uses the current finalized Miner Ranking. Before the first finalized ranking for an Intent, routing is uniform among eligible registered Miners. Once ranking

state exists, routing becomes probabilistic and ranking-based, with higher-ranked eligible Miners receiving greater routing opportunity under the current routing policy.

Routing weights are protocol parameters and do not modify the underlying ranking. Operational delivery failure may cause request-level failover to another eligible Miner without changing ranking state.

Finalized ranking state remains subject to protocol spot checks. At genesis, the spot-check frequency parameter is $k_{\text{spot}} = 10$. Selection derives from finalized protocol randomness.

During a spot check, the selected Miner receives an Intent-compatible evaluation request and is measured using the current Canonical Evaluator. Spot-check traffic is protocol evaluation activity and is not classified as paid Consumer demand.

At genesis, a relative performance decline greater than 20 percent may produce a `Routing_Revocation` state transition. After Validator verification and finalization, the affected Miner is removed from active paid routing for the applicable period and its routing opportunity is redistributed among the remaining eligible Miners. Miner principal remains intact.

5.16 Continuous Evaluator Competition

Evaluator competition continues throughout the lifetime of an Intent. A new Evaluator may register, contribute its evaluation logic and material, compete, satisfy the applicable qualification requirements, pass independent verification, and become Canonical.

Subsequent Miner-ranking rounds then use the newly finalized Canonical Evaluator. The intelligence supply and the method used to evaluate that supply can therefore improve independently.

This creates two continuing competitive processes: Miner competition improves the available intelligence supply, while Evaluator competition improves the mechanism used to measure that supply. The ranking system connects both processes through finalized state.

5.17 Economic and Autonomous Feedback Loop

Miner revenue is generated by paid Consumer demand. Higher ranking produces greater routing opportunity and therefore greater opportunity to earn from Consumer usage. Canonical Evaluators receive the applicable Evaluator rewards defined by the economic model.

The resulting cycle connects demand, competition, and quality: Consumer demand creates Miner revenue opportunity, Miner competition can improve the intelligence supply, stronger intelligence increases the value of accurate evaluation, Evaluator competition can improve the measurement process, improved evaluation can produce better Miner Rankings, and the resulting rankings direct demand across the available intelligence supply.

The Autonomous Engine and external Consumers use the finalized ranking produced by this process. Ranking production, intelligence delivery, and economic settlement remain separate protocol processes connected through finalized state and authenticated accounting.

6. VALIDATORS AND CONSENSUS

Validators provide the verification, consensus, and state-finalization layer of Telegraph. They reproduce or verify protocol computations assigned to them, validate the commitments and evidence required by those computations, sign protocol state, and finalize state transitions through Byzantine fault-tolerant consensus.

Validator consensus finalizes ranking state, Canonical Evaluator state, routing state, protocol penalties, accounting commitments, and other consensus-relevant protocol state. Miner quality remains determined by the applicable Canonical Evaluator and ranking process. Validators verify that the required protocol computation was performed against the committed inputs.

The Validator architecture separates economic principal from protocol participation rights. Validators may lose emissions, routing or verification assignments, active-set membership, or permanent eligibility after severe violations, but their staked MACHINA principal is not burned, transferred, confiscated, or reduced as a protocol penalty.

6.1 Active Validator Set

The Active Validator Set is determined by stake competition. At each epoch boundary, the top N eligible Validator Profiles ranked by staked MACHINA form the Active Validator Set for the following epoch.

N is the Validator Cap. At genesis, the Validator Cap is 64. The cap may be modified through the governance process subject to the protocol's immutable consensus constraints.

Stake determines competition for Validator-set membership and the stake weights used by applicable ranking computations. It does not create a right to weaken protocol verification requirements or bypass the consensus threshold.

Each Validator Profile separates ownership and online protocol operation through a Manager Key and an authorized Operator Key. The Manager Key controls stake and profile administration. The Operator Key performs online signing, verification, and consensus functions and cannot withdraw the Validator's staked MACHINA.

$$V_e = \text{top } N \text{ eligible Validator Profiles by staked MACHINA at epoch } e$$

6.2 Validator Assignments

Validator assignments depend on the protocol operation being executed. An assignment may require a consensus signature, ranking computation, candidate-Evaluator verification, routing-revocation verification, state commitment, or another protocol message required for finalization.

Candidate Evaluators normally execute on their own infrastructure during the competition stage. Validator computation is concentrated on the independent verification and consensus operations required to make a candidate or ranking result authoritative.

Failure to complete a required assignment affects the Validator under the liveness and penalty rules described below.

6.3 Leader Rotation and Failover

One active Validator coordinates each Miner-ranking round. Leader selection follows a strict round-robin schedule over the ordered Active Validator Set. Each successive ranking round advances the Leader turn to the next active Validator in that order.

$$\text{LeaderIndex} = \text{Round_ID} \bmod |V|$$

The Leader coordinates the protocol-defined Miner evaluation request, response collection, required response-provenance evidence, and construction of the committed Evaluation Package. The Leader does not select the Canonical Evaluator and does not determine Miner scores.

If the scheduled Leader fails to complete its assignment within the applicable protocol window, the next Validator in the same round-robin order becomes eligible to continue the round.

Leader replacement does not change the Canonical Evaluator, evaluation request, eligible Miner Set, timely valid Miner responses already received, or other committed ranking inputs. A failed Leader assignment is treated as a missed Validator assignment under the liveness rules.

6.4 Consensus and Finality

A ranking or other consensus-relevant state transition becomes finalized only after the required quorum of the Active Validator Set signs the corresponding state commitment.

The Byzantine quorum is defined by the size of the Active Validator Set. At the genesis Validator Cap of 64, 43 distinct active Validators are required for finality.

$$\tau = \text{floor}(2 \times |V| / 3) + 1$$

Each Validator signs only after completing the verification required for the applicable state transition. Signatures are bound to the exact protocol state and consensus context, including the chain, protocol version, Validator Set, state sequence, and Protocol State Root.

Eligible signatures may be aggregated through BLS or another protocol-supported aggregate-signature mechanism. The resulting consensus proof establishes that the required Validator quorum authorized the committed protocol state.

If fewer than the required number of valid signatures are obtained, the proposed transition does not finalize and the previously finalized state remains canonical.

6.5 Network Partitions and Quorum Preservation

Telegraph does not reduce its Byzantine consensus threshold when the network cannot reach quorum.

If a network partition, Validator outage, or communication failure prevents the required signatures from being obtained, the affected transition remains unfinalized. Consensus resumes from the last finalized state when the required quorum can again communicate and verify a valid successor transition.

The protocol does not create competing finalized successor states by automatically weakening quorum requirements during degraded operation.

6.6 Protocol State Commitments

Finalized consensus state is represented by a top-level Protocol State Root. The root authenticates the consensus-relevant state associated with the applicable state sequence while preserving logical separation between protocol domains through authenticated subroots.

These subroots may commit finalized Miner Rankings, Canonical Evaluator state, evaluation state, routing eligibility and Routing Revocations, Miner, Validator, and Evaluator entitlements, penalty status, the Active Validator Set, and other state required by the implementation.

Evaluation state may additionally commit the data required for deterministic audit of a ranking round, including Evaluation Package roots, eligible Miner-set roots, Miner-response roots, Validator score commitments and reveals, the stake-snapshot identifier, Canonical Scores, and round metadata.

6.7 Data Availability and Deterministic Audit

Consensus-relevant data required to reproduce a finalized ranking transition must satisfy the protocol's data-availability requirements before the transition can finalize.

A historical ranking round retains, or makes retrievable, the authenticated inputs required to reconstruct the ranking process. An auditor can verify the request and response provenance, the committed Canonical Evaluator artifact, Validator Local Scores and commitments, Canonical Score aggregation, deterministic Miner Ranking, and the resulting finalized state.

This replay process verifies that ranking state was produced consistently with the committed protocol inputs and rules. Authenticated transcript evidence establishes the provenance and integrity of the interaction used for ranking, the Canonical Evaluator remains the mechanism that measures Miner performance.

Where evaluation material or Miner responses are confidential, replay access may be restricted to authorized verification or attested execution environments while the corresponding commitments remain available for protocol verification.

6.8 Validator Exit and Unbonding

A Validator Profile enters unbonding when it exits voluntarily, is displaced from the Active Validator Set by stake competition, or is removed through an applicable protocol-ejection rule.

At genesis, the Validator unbonding period is 21 days. During unbonding, the stake remains locked and does not contribute to Active Validator Set competition.

The unbonding period allows active-set transitions and outstanding reward and protocol-accounting state associated with the Validator's previous active period to finalize. Validator principal is returned in full after the applicable unbonding process.

6.9 Validator Penalty Model

Validator penalties enforce liveness, deterministic protocol execution, and cryptographic integrity. Penalties affect emissions, reward eligibility, Validator status, and participation rights. They do not reduce Validator principal.

The protocol separates violations into three categories so that failure to perform, incorrect execution, and objectively provable fraud have distinct consequences.

Category	Condition	Primary consequence	Principal
A	Liveness failure	Assignment emission forfeiture, ejection after sustained or repeated misses.	Returned after unbonding.
B	Cryptographically provable fraud	Epoch emission forfeiture, immediate removal, permanent Validator Profile revocation.	Returned after unbonding.
C	Incorrect deterministic execution	Assignment emission forfeiture, strike system, ejection after threshold.	Returned after unbonding.

6.10 Category A: Liveness Failure

A Validator must complete the protocol assignments allocated to it while active. A missed required assignment forfeits the Validator emission associated with that assignment, and the forfeited amount is redistributed among eligible Validators that completed the applicable work.

At genesis, a Validator is ejected for liveness failure if it fails required assignments continuously for seven days or misses more than 50 percent of its required assignments within any rolling seven-day window. A successful required assignment resets the continuous-failure counter.

A liveness-ejected Validator is removed from the Active Validator Set and enters the standard 21-day unbonding process. It may later compete for Validator-set membership again after satisfying the applicable eligibility requirements.

6.11 Category B: Cryptographically Provable Fraud

Category B applies when protocol data or attestations can be objectively demonstrated through cryptographic evidence to be fraudulent, fabricated, substituted, altered, or inconsistent with the committed inputs.

Examples include fabricated source data, falsified protocol payloads, false attestation of an Evaluator artifact or verification result, signing a state transition that violates the applicable cryptographic commitments, or forging or substituting protocol evidence.

Subjective evaluation disagreement, ordinary model error, and execution differences inside an explicitly permitted tolerance are not Category B fraud. The category requires objective evidence of protocol-level dishonesty.

When Category B fraud is finalized, the Validator forfeits unfinalized Validator emissions attributable to the affected epoch, becomes ineligible for further Validator emissions, is removed from the Active Validator Set, and has its Validator Profile permanently revoked.

The associated Manager and Operator Keys are permanently ineligible to register or operate another Validator Profile.

The Validator's staked MACHINA enters the standard unbonding process and is returned in full. Category B therefore removes protocol participation rights without confiscating economic principal.

6.12 Category C: Incorrect Protocol Execution

Category C applies when a Validator repeatedly produces outputs inconsistent with the deterministic computation, committed state, or verification result required by its assignment and the behavior does not meet the evidentiary standard for Category B fraud.

Examples include an incorrect deterministic Miner-ranking result, an incorrect state commitment, a candidate-Evaluator verification result inconsistent with required execution, use of obsolete protocol state, or repeated malformed or invalid consensus messages.

Each qualifying Category C event causes forfeiture of the emission associated with the incorrect assignment and produces one strike. At genesis, five strikes within any rolling 30-day window cause removal from the Active Validator Set and entry into the standard unbonding process.

Category C ejection is not permanent. An operator may correct the software, configuration, synchronization, or execution problem and later compete for Active Validator Set membership again. A result inside an explicitly defined mechanism-specific verification tolerance is treated as valid.

6.13 Fraud Evidence and Classification

Telegraph distinguishes cryptographically provable fraud from liveness failure, ordinary model error, statistical uncertainty, evaluation disagreement, and non-fraudulent deterministic execution error.

The classification determines the applicable effect on emissions, eligibility, and Validator status. It also prevents subjective disagreement about machine intelligence from being treated as protocol fraud.

The common invariant remains unchanged across all categories: finalized participant principal is not confiscated as a protocol penalty, and previously finalized cumulative entitlements are not clawed back.

7. ROUTING AND THE AUTONOMOUS ENGINE

Routing converts finalized ranking state into intelligence delivery. For each request, Telegraph resolves the applicable Intent, derives the set of eligible Miners, and selects a Miner according to the current routing policy. The selected Miner returns the machine-consumable result delivered as a Signal.

The Autonomous Engine is the common request-processing and routing layer used by direct Consumer activity and continuously generated intelligence. It consumes finalized protocol state. It does not create Miner Rankings, select Canonical Evaluators, or perform consensus.

This separation keeps intelligence selection tied to the protocol's evaluation and ranking mechanisms while allowing applications, agents, and automated systems to consume the resulting intelligence through several delivery modes.

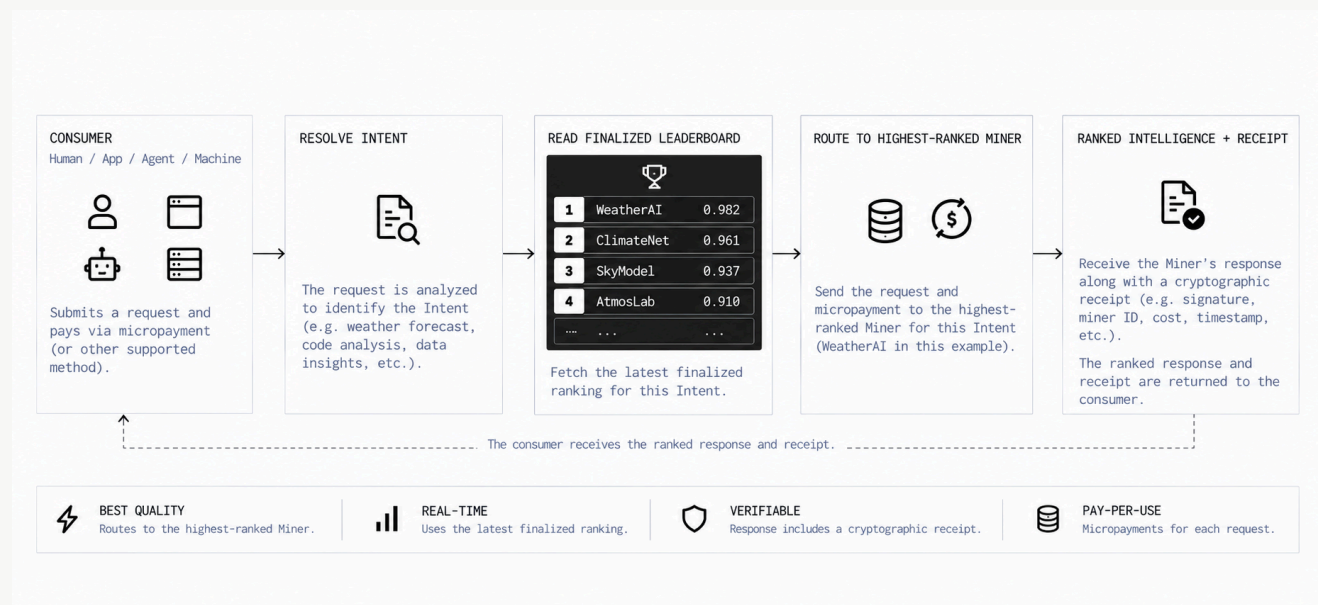


Figure 4. A live request resolves to an Intent, reads the latest finalized leaderboard, routes paid demand to the highest-ranked Miner, and returns ranked intelligence with a cryptographic receipt.

7.1 Finalized Ranking as Routing State

Consumer demand is routed using the current finalized Miner Ranking for the applicable Intent. Before the first Miner Ranking for an Intent is finalized, routing is distributed uniformly among eligible registered Miners.

Once ranking state exists, routing becomes probabilistic and ranking-based. Higher-ranked eligible Miners receive greater routing opportunity. The routing policy distributes demand across the qualifying ranked set so delivery does not depend on a single top-ranked Miner.

Routing weights are protocol parameters and do not modify the underlying Miner Ranking. A change in delivery outcome caused by availability or request-level failover does not alter ranking state.

A newly registered Miner obtains normal routing opportunity through evaluation and ranking. The protocol does not reserve a fixed Consumer-traffic share or mandatory routing grace period for new Miners.

7.2 Eligibility and Policy-Constrained Routing

Ranking is applied within the set of Miners eligible for the request. Eligibility may depend on active registration, current ranking state, availability, pricing compatibility, Routing_Revocation status, Consumer routing constraints, confidentiality requirements, jurisdictional restrictions, or other protocol-defined conditions.

Consumers may apply supported restrictions such as explicit Miner allowlists, required execution-environment attestations, required security capabilities, geographic or jurisdictional metadata where an applicable verification mechanism exists, organizational policies, or other Intent-compatible routing constraints.

The protocol first derives the eligible Miner set and then applies the current ranking-based routing policy within that set. Self-declared Miner metadata does not by itself establish legal or regulatory compliance, applications remain responsible for policies specific to their use case.

7.3 The Autonomous Engine

The Autonomous Engine contains three logical layers: Collectors, autonomous orchestration and question generation, and the Engine with its Miner router. These layers may run within one node binary or as separate services.

The Engine is the common request-processing layer. It receives a Consumer request or generated question, resolves one or more compatible Intents, reads the current ranking and eligibility state, selects an eligible Miner, constructs the Miner-specific request from registered integration metadata, dispatches the request, performs operational failover when required, receives the response, and produces the resulting Signal and delivery metadata.

Miner selection is controlled by finalized ranking and eligibility state. A language model or other orchestration component cannot select a preferred Miner outside those protocol rules.

7.4 Collectors

Collectors connect Telegraph to external information and trigger sources. A Collector may read from an API, market feed, event stream, blockchain state, database, scheduled source, or another machine-readable system.

Each Collector is a self-contained registered integration that defines how its source is accessed, triggered, and normalized. Collector output is input data for the orchestration layer, it is not routing authority.

A Collector cannot directly determine the Intent, select a Miner, change a Miner Ranking, set a routing weight, select a Canonical Evaluator, or determine an evaluation result.

7.5 Autonomous Orchestration

The orchestration layer executes Collectors according to their schedules or trigger conditions and converts relevant trigger data into one or more machine questions or tasks.

Question generation may use an LLM or another compatible mechanism. No specific commercial model provider is a protocol requirement. A generated question does not carry authoritative Miner-selection or ranking instructions, it enters the same Intent-resolution and routing process as other Engine requests.

7.6 Request-Driven and Autonomous Execution

Direct Consumer requests enter the Engine without requiring a Collector. The Engine resolves the request to an Intent, consults the finalized Miner Ranking and eligibility state, dispatches the request to a selected Miner, and returns the resulting Signal to the Consumer.

Autonomous execution begins with an external source. A Collector produces trigger data, the orchestration layer generates a question or task, and the Engine then follows the same Intent-resolution and ranking-based routing process used for direct requests.

Signals may be distributed through API, WebSocket, Intelligence Terminal, subscriptions, or other supported interfaces. Continuous intelligence streams may repeat this process as new external information or trigger events arrive.

Mode	Execution path
Direct	Consumer Request -> Engine -> Intent Resolution -> Finalized Miner Ranking -> Eligible Miner -> Miner Response -> Signal -> Consumer
Autonomous	External Source -> Collector -> Trigger Data -> Question / Task -> Engine -> Intent Resolution -> Finalized Miner Ranking -> Eligible Miner -> Miner Response -> Signal -> Subscriber

7.7 Intent Resolution and Operational Failover

Intent resolution occurs after a request reaches the Engine. The resolver maps the request to one or more compatible registered Intents. Miner selection then uses the finalized ranking and eligibility state for the resolved Intent.

If the selected Miner cannot complete the request because of unavailability, quota exhaustion, transport failure, or another supported operational failure, the Engine may attempt another eligible Miner according to the routing-failover policy.

Operational failover does not modify the finalized Miner Ranking. Persistent performance or availability failures are handled through subsequent ranking rounds, spot checks, Routing_Revocation, or the other eligibility mechanisms defined by the protocol.

7.8 Separation from Ranking and Evaluation

The Autonomous Engine consumes ranking state, it does not create it. Evaluation and ranking determine which Miners perform best for an Intent. The Engine uses the finalized result to deliver intelligence.

Candidate Evaluator competition is independent from Autonomous Engine routing. The Engine does not select the Canonical Evaluator, execute candidate qualification, determine Canonical Scores, or grade Miner responses.

This boundary also applies to Collectors and question-generation systems. External data and generated prompts can initiate requests, but they cannot override protocol ranking, eligibility, or evaluation state.

7.9 Collector and Internal-Request Security

Collector inputs are treated as untrusted external data. Collector content cannot directly alter Miner Ranking, select an Evaluator, modify an Intent registration, or change protocol economic state.

Requests sent from internal orchestration components to the Engine must be authenticated. A plaintext internal header is not sufficient authority to bypass payment or access controls. Daemon-to-Engine communication uses cryptographic authentication or an equivalent authenticated internal-request mechanism.

7.10 Signal Identity and Auditability

Signals carry enough metadata to reconstruct the execution path that produced them. Depending on the delivery mode, this may include a Signal ID, Signal hash, Intent ID, Miner ID, request and response commitments, Collector identity, trigger commitment, routing metadata, and payment or protocol-generation classification.

An autonomous Signal can therefore be traced from Collector and trigger through question generation, Intent resolution, Miner selection, response, and delivery. A direct Signal can be traced from the Consumer request through Intent resolution, Miner selection, response, and delivery.

This metadata supports auditability of the routing and delivery process without changing the role of the Canonical Evaluator in measuring Miner performance.

7.11 Delivery and Settlement Boundary

Real-time Signal delivery is operationally separate from economic settlement. A request can be fulfilled and a Signal delivered before the corresponding economic activity is aggregated and finalized on Ethereum.

Consumer usage is recorded through the applicable request-response, streaming, session, or other supported accounting mechanism. Settlement operates on the authenticated accounting state produced by those delivery paths.

This separation allows synchronous APIs, persistent machine streams, subscriptions, and autonomous delivery to share the same ranking and economic system without requiring each intelligence request to wait for on-chain settlement.

8. PAYMENTS AND SETTLEMENT

Telegraph separates intelligence delivery from economic settlement. A Consumer can receive a Signal as soon as the applicable request, session, or job conditions are satisfied, while USDC accounting and MACHINA settlement are finalized on their own cadence.

The protocol supports several payment rails for different forms of machine activity: synchronous x402 requests, pre-funded high-frequency sessions, managed access, and direct on-chain jobs. These rails use different authorization and delivery mechanics but converge into the same accounting and canonical Ethereum settlement system.

Paid Consumer activity is denominated in USDC. For every successfully paid intelligence delivery, 2 percent is allocated to the Protocol Treasury and 98 percent is attributed to Miner settlement. Miner settlement is subsequently converted into MACHINA through the aggregated Ethereum settlement process.

8.1 Payment Architecture

Ethereum is the canonical economic settlement domain of Telegraph. Canonical contracts maintain or authenticate MACHINA settlement state, Protocol Treasury state, participant entitlement roots, finalized protocol-state commitments, active Validator-set state, settlement sequencing, and reward claims.

Consumer payments may originate through supported payment networks or pre-funded escrow mechanisms. Telegraph does not require every request, response, streaming delivery, or acknowledgement to create an Ethereum transaction.

The payment architecture separates four operations: payment authorization, intelligence delivery, USDC accounting and Miner attribution, and Ethereum economic settlement. This allows high-frequency activity to be accounted for off-chain and settled in aggregate without delaying the delivery of intelligence.

Consumer rail	Execution model	Economic convergence
Synchronous request	x402-compatible authorization followed by ranked Miner delivery.	USDC accounting -> 2% Treasury / 98% Miner settlement
Streaming / session	Pre-funded escrow with cumulative acknowledgements and bounded in-flight exposure.	USDC accounting -> 2% Treasury / 98% Miner settlement
Direct on-chain job	Escrow-backed one-shot job to an explicitly selected Miner with Validator authorization.	Immediate job finality, later joins the same aggregated Miner settlement

8.2 x402 Synchronous Requests

Telegraph supports x402-compatible request-response payments for synchronous intelligence delivery. A Consumer submits a request for an Intent and Telegraph determines the applicable Signal price.

If valid payment authorization is absent, Telegraph returns the applicable HTTP 402 payment requirements. The Consumer then supplies authorization for the required USDC payment.

Telegraph verifies the amount, asset, destination, expiry, and replay-protection conditions before the request proceeds.

Once payment is authorized, the request is routed to an eligible Miner according to the applicable routing rules. Telegraph records the fulfilled paid delivery and the exact Miner attribution, then returns the result and the applicable protocol receipt to the Consumer.

Payment authorization and delivery records carry sufficient context to prevent the same authorization or delivery from being replayed or counted more than once. The Miner does not wait for an immediate USDC-to-MACHINA conversion.

$$\begin{aligned}\text{ProtocolFee} &= 0.02 \times \text{SignalPrice} \\ \text{MinerSettlementUSDC} &= 0.98 \times \text{SignalPrice}\end{aligned}$$

8.3 Pre-Funded Escrow and Streaming

Consumers requiring repeated or high-frequency access may pre-fund a USDC escrow. The escrow allows intelligence to be delivered continuously without requiring a blockchain transaction for every individual message.

The spendable capacity of a session accounts for funds already consumed and for delivery that has been transmitted but not yet acknowledged.

$$\text{EffectiveBalance} = \text{EscrowBalance} - \text{AccountedSpend} - \text{ReservedInFlightExposure}$$

During a WebSocket or comparable session, the Consumer periodically signs a cumulative acknowledgement containing the latest accepted delivery sequence and cumulative USDC tab. A valid later acknowledgement supersedes earlier acknowledgements for the same session, and the cumulative tab cannot decrease previously acknowledged spend.

$$\text{ACK}_k = \text{Sign}(\text{Session_ID} \parallel \text{Sequence}_k \parallel \text{CumulativeTab}_k \parallel \text{Context})$$

Telegraph also maintains a sliding in-flight window so acknowledgement latency cannot create unlimited unsecured exposure. Additional paid delivery is permitted only while the acknowledged spend and maximum permitted in-flight exposure remain within the authorized session limit.

$$\begin{aligned}\text{AcknowledgedSpend} + \text{MaximumInFlightExposure} &\leq \text{AuthorizedSessionLimit} \\ \text{AggregateInFlightExposure} &\leq \text{AvailableEscrow}\end{aligned}$$

If acknowledgements stop, the session becomes invalid, or the permitted window is exhausted, further paid delivery is suspended until sufficient acknowledgement or funding is available. Protocol nodes replicate the session-accounting state required to prevent concurrent sessions from collectively exceeding the Consumer's funded capacity.

Authenticated cumulative acknowledgements and synchronous payment receipts ultimately enter the same USDC accounting system. For each paid delivery the protocol records the Consumer, Intent, Miner, applicable price, protocol fee, and Miner-attributable USDC.

8.4 Managed and Web2 Access

Telegraph may provide managed-wallet, fiat-onboarding, or equivalent integrations for applications that do not want to manage blockchain payment operations directly.

A managed integration may acquire or deposit USDC, maintain a Consumer payment account, process x402 authorization, maintain pre-funded escrow, operate streaming sessions, or create direct on-chain jobs on behalf of the application.

No particular wallet, fiat processor, or custody provider is a consensus-critical dependency of Telegraph. Supported integrations ultimately produce USDC-denominated authorization and accounting state compatible with the same protocol settlement rules.

8.5 On-Chain Callbacks

A smart contract may request an on-chain callback for a direct job or another explicitly callback-enabled request. Callback execution is an optional delivery action and its gas cost is separate from the intelligence price.

A callback can receive compact result information such as the Job ID, Intent ID, Miner ID, output hash, and bounded OnChainData. Large Miner responses remain available through the authenticated off-chain result path rather than being placed directly on-chain.

The requesting application supplies the gas funding required by the callback mechanism. An optional callback failure does not invalidate an otherwise successfully finalized job unless callback success was explicitly included in the job conditions.

Callback execution is bounded against excessive gas consumption, reentrancy, and repeated execution. If callback funding is insufficient, the callback remains pending or follows the applicable expiry rule. Validators are not required to finance third-party callback execution.

8.6 Direct On-Chain Jobs

Telegraph provides a direct on-chain job rail for autonomous agents and smart contracts that require a one-shot, escrow-backed request to a specific Miner. This rail is separate from ordinary HTTP and streaming settlement.

A funded job identifies the Intent, selected Miner, request or request commitment, compatible input parameters, USDC budget, expiry, and any optional callback and callback-gas funding. The selected Miner must be active for the requested Intent and satisfy the applicable availability, pricing, routing-revocation, and eligibility requirements.

When a valid Miner ID is explicitly specified, normal probabilistic routing does not replace that selection. The request is sent directly to the selected Miner.

After the Miner responds, Telegraph authenticates the response and computes an output commitment. The raw response remains available through the authenticated result-availability layer.

```
outputHash = H(Miner_Response)
```

Where the request-response interaction requires transferable provenance, zkTLS or an equivalent authenticated transcript mechanism binds the exact job request and Miner response.

Validators verify the job-resolution package, including the funded state, Intent, selected Miner, request commitment, response provenance, output hash, payment state, fee calculation, Miner attribution, and callback commitment where applicable.

```
JobResolutionRoot = H(Job_ID || Intent_ID || Miner_ID || Request_Commitment || outputHash ||
Payment_State || Protocol_Context)
```

8.7 Job Finality and Validator Authorization

A successful direct job requires the same Byzantine quorum formula used by Telegraph consensus. At the genesis Active Validator Set of 64, 43 distinct active Validators are required.

$$\tau = \text{floor}(2 \times |V| / 3) + 1$$

Once the required authorization has been obtained, the resolution is submitted to the job contract. The contract verifies the job state, quorum authorization, replay protection, and committed result before completing the job.

A Job ID cannot be completed twice, and authorization for one job cannot be reused for another. Validator authorization establishes that the Telegraph job was executed and recorded according to protocol rules, it does not independently establish the semantic truth of the Miner response.

The job becomes final immediately after successful job-specific on-chain resolution. It does not wait for the next ordinary epoch or cumulative settlement submission. The resulting accounting can later be incorporated into the normal global protocol-state commitment.

8.8 ERC-8183 Agentic Commerce

Telegraph exposes the direct-job rail through an ERC-8183-compatible Agentic Commerce interface. The job lifecycle maps to Open, Funded, Submitted, and Completed, together with the applicable rejection and expiry paths.

The Agent creating and funding the job acts as the Client. The selected Telegraph Miner provides the requested intelligence, and the corresponding outputHash is submitted as the job deliverable commitment.

The protocol-controlled ERC-8183 Evaluator accepts completion only after the required Telegraph Validator quorum authorizes the corresponding job-resolution commitment. The ERC-8183 Evaluator therefore verifies Telegraph completion evidence. It is separate from the Canonical Evaluator used by Telegraph to rank Miner performance.

8.9 Direct-Job Economics

A successful direct job uses the same economic split as other paid Consumer activity.

$$\begin{aligned}\text{JobProtocolFee} &= 0.02 \times \text{JobPriceUSDC} \\ \text{JobMinerSettlementUSDC} &= 0.98 \times \text{JobPriceUSDC}\end{aligned}$$

The protocol fee is allocated to the Protocol Treasury and the Miner-settlement amount is attributed to the explicitly selected Miner. Completing the job does not trigger a separate per-job MACHINA purchase.

The Miner-attributable USDC joins the standard aggregated settlement ledger and is processed through the canonical Ethereum USDC-to-MACHINA settlement mechanism. This preserves one economic settlement system across synchronous requests, streaming activity, and direct jobs.

8.10 Aggregated Ethereum Miner Settlement

USDC allocated to Miner compensation is consolidated into the canonical Ethereum settlement domain. Telegraph acquires MACHINA using aggregated Miner-settlement USDC through time-distributed execution under the applicable settlement parameters.

The execution may be divided across multiple transactions and times to avoid concentrating predictable settlement volume. Settlement parameters govern matters such as cadence, minimum and maximum execution size, liquidity and price-impact limits, scheduling variation, failed execution, and rollover handling.

For settlement period N , U_N denotes the finalized USDC allocated to Miner settlement. After the Ethereum settlement execution completes, M_N denotes the exact amount of MACHINA acquired for that period. Final Miner entitlements use the amount actually acquired rather than an estimated future exchange rate.

$$\text{PeriodMinerEntitlement}_{(m,N)} = \text{floor}(\text{MinerUSDC}_{(m,N)} \times M_N / U_N)$$

Settlement arithmetic uses deterministic integer token units. Any deterministic remainder created by integer division is handled by the protocol settlement-remainder rule.

Each period entitlement is added to the Miner's cumulative lifetime entitlement and committed through the Miner Entitlement Root. Withdrawal changes the amount already claimed, not the cumulative gross entitlement previously finalized by the protocol.

$$\text{Claimable}_m = \text{LifetimeMinerEntitlement}_m - \text{WithdrawnAmount}_m$$

8.11 Unified Payment and Settlement Flow

The supported payment rails differ in authorization and delivery cadence, but all paid Consumer activity converges on the same economic accounting: 2 percent to the Protocol Treasury and 98 percent to Miner settlement.

Synchronous x402 requests may settle one request at a time from the Consumer's perspective. Streaming sessions compress many deliveries into cumulative authenticated accounting. Direct on-chain jobs obtain immediate job-specific finality. None of these delivery paths requires a separate economic market or a separate Miner-compensation system.

This separation allows Telegraph to support synchronous applications, high-frequency machine streams, managed integrations, and autonomous smart-contract jobs while retaining one canonical economic settlement domain and one cumulative entitlement model.

9. TOKENOMICS AND INCENTIVES

Telegraph separates payment for intelligence from rewards for protocol infrastructure. Miners are compensated from paid Consumer demand. Validators and Evaluators are compensated from the fixed MACHINA emission schedule. The Protocol Treasury receives its fixed emission allocation, protocol fees, and allocations redirected to it under the economic rules.

This separation is a core economic design choice. Intelligence Providers are paid when Consumers purchase intelligence. Protocol emissions reward the infrastructure that evaluates, verifies, finalizes, and secures the network. Demand therefore determines Miner revenue, while the fixed emission schedule finances protocol operation.

Consumer-denominated activity is accounted for in USDC. MACHINA is the protocol-native economic unit of Telegraph's intelligence commodity and the native asset used by the protocol's security, reward, and settlement mechanisms, with canonical economic settlement on Ethereum. This is an economic relationship rather than a fixed physical conversion: one MACHINA does not equal one request, Signal, inference, or unit of computation. Fulfilled intelligence is priced and accounted for in USDC; Miner-attributable settlement value is then used to acquire MACHINA on the open market.

9.1 MACHINA Supply

MACHINA has a fixed maximum supply of 21,000,000 tokens. The supply schedule is fixed at genesis and is an immutable property of the protocol.

The genesis emission rate is 7,200 MACHINA per 24-hour emission period. The daily emission halves every 1,461 emission days, approximately four years. There is no premine and no Team or VC allocation.

$$E(d) = E_0 \times 2^{(-\text{floor}(d / 1461))}$$

$E_0 = 7,200 \text{ MACHINA/day}$ $S_{\text{max}} = 21,000,000 \text{ MACHINA}$

Emission stops permanently when cumulative minting reaches the maximum supply. If the remaining mintable supply is smaller than the scheduled emission for a period, only the remaining amount is minted.

9.2 Fixed Emission Allocation

Each emission period is divided among three recipients using fixed percentages established at genesis.

Recipient	Share	Purpose
Validators	60%	Verification, consensus, state finalization, and other assigned protocol work
Evaluators	20%	Rewards for eligible Canonical Evaluators under the demand-weighted, fee-backed allocation mechanism
Protocol Treasury	20%	Protocol-controlled resources for authorized expenditure and residual allocations

Miners receive zero MACHINA from protocol emissions. Their compensation is derived exclusively from fulfilled paid Consumer demand. This prevents protocol inflation from

rewarding unused intelligence supply and connects Miner earnings directly to actual network usage.

9.3 Miner Demand-Funded Compensation

For each fulfilled paid delivery, Telegraph records the gross USDC value attributable to the Miner that served the request. The protocol applies the same economic split across supported paid Consumer interfaces.

$$\begin{aligned}\text{ProtocolFee} &= 0.02 \times \text{GrossPaidUSDC} \\ \text{MinerSettlementUSDC} &= 0.98 \times \text{GrossPaidUSDC}\end{aligned}$$

The 2 percent protocol fee is directed to the Protocol Treasury. The remaining 98 percent enters the Miner-settlement ledger.

Miner-settlement USDC is consolidated into the canonical Ethereum settlement domain and used to acquire MACHINA through time-distributed execution. The amount of MACHINA actually acquired for the settlement period determines Miner entitlements. The protocol does not use an estimated future exchange rate to calculate final Miner compensation.

Protocol evaluation traffic, ranking requests, Evaluator competition, spot checks, grants, Treasury-funded traffic, bootstrap activity, and other non-Consumer protocol operations do not count as paid Consumer demand for Miner revenue accounting.

9.4 Validator Rewards

Validators receive 60 percent of scheduled MACHINA emissions. Validator emissions are not dependent on Consumer demand, a period with no Consumer activity does not eliminate the Validator emission pool.

$$\text{ValidatorEmissionPool}(d) = 0.60 \times E(d)$$

Validator rewards are allocated according to successfully completed protocol work. Reward-bearing assignments may include Miner-ranking verification, Local Score computation, commit-reveal participation, BFT consensus, routing-state verification, protocol-state verification, canonical submission duties, and other protocol-defined assignments.

At genesis, 2 percent of the applicable Validator emission pool is allocated to the Internal Audit Fee for Validators that successfully perform assigned candidate-Evaluator verification.

A Validator that fails a required assignment does not earn the emission associated with that assignment. Category A, B, and C penalties are applied to unfinalized Validator rewards before cumulative entitlement is updated. Forfeited emissions may be redistributed under the applicable finalized reward rules.

Validator principal is never reduced as a protocol penalty.

9.5 Evaluator Emission Pool

Evaluators receive 20 percent of scheduled MACHINA emissions. The Evaluator pool is distributed across active Intents according to real paid Consumer activity and the fee-backed reward constraints described below.

$$\text{EvaluatorEmissionPool}(d) = 0.20 \times E(d)$$

The allocation mechanism uses economic demand rather than request count or a subjective measure of Intent difficulty. One unit of finalized Consumer-paid USDC receives the same initial weight regardless of the Intent on which it was spent.

Differences in task complexity, compute cost, scarcity, and Consumer value are expressed through Miner pricing and actual Consumer willingness to pay rather than through a protocol-assigned difficulty coefficient.

9.6 Qualified Consumer Demand

Qualified Consumer Demand is finalized gross USDC paid by Consumers for successfully fulfilled intelligence. For Intent i during emission day d , this amount is denoted $D_i(d)$.

Only protocol-recognized paid Consumer activity qualifies. Miner-ranking traffic, Evaluator competition, spot checks, protocol testing, Treasury-funded requests, protocol-funded Autonomous Engine bootstrap activity, grants, free requests, Intent Bounties, and other internally generated protocol activity are excluded. Qualification under this rule establishes that the activity is paid and protocol-recognized, it does not by itself prove that the Consumer, Miner, Evaluator, or other participating addresses are economically independent.

Demand and fee accounting use finalized protocol records and the same economic activity cannot be counted more than once. Splitting equivalent paid demand among multiple wallets, Miners, or Intents does not create additional aggregate fee-backed reward capacity.

$$F_i(d) = 0.02 \times D_i(d)$$

9.7 Demand-Weighted Evaluator Allocation

The first step in Evaluator allocation distributes the daily Evaluator pool across Intents in proportion to Qualified Consumer Demand.

$$D_{\text{total}}(d) = \sum D_i(d)$$

$$\text{RawEvaluatorAllocation}_i(d) = \text{floor}(\text{EvaluatorEmissionPool}(d) \times D_i(d) / D_{\text{total}}(d))$$

The allocation is linear in finalized paid USDC. Request count is not part of the formula. Dividing the same total economic demand among several Intents does not create more aggregate raw emission weight, apart from deterministic integer rounding.

9.8 Fee-Backed Evaluator Reward Cap

Demand weighting determines the relative share of the Evaluator pool, but it does not determine the final reward. Each Intent's allocation is additionally limited by the irreversible protocol fees generated by that Intent's Consumer activity.

The reward-backing safety factor is denoted β . At genesis, β is 0.80.

$$\text{RewardBackingUSDC}_i(d) = \beta \times F_i(d)$$

$$\beta = 0.80 \quad \Rightarrow \quad \text{RewardBackingUSDC}_i(d) = 0.016 \times D_i(d)$$

At the genesis parameters, the USDC-equivalent value of the Evaluator reward attributable to an Intent cannot exceed 1.6 percent of its Qualified Consumer Demand, while the same activity generates an irreversible protocol fee of 2 percent.

The reward backing applies only to the current allocation period. Unused backing does not accumulate, move between Intents, or become future reward capacity.

9.9 MACHINA Reference Price and Final Evaluator Allocation

Because the fee-backed limit is denominated in USDC while Evaluator rewards are denominated in MACHINA, the protocol uses a manipulation-resistant MACHINA reference price expressed in USDC per MACHINA.

$$\text{FeeBackedCap}_i(d) = \text{floor}(\text{RewardBackingUSDC}_i(d) / P_{\text{ref}}(d))$$

$$\text{FinalEvaluatorAllocation}_i(d) = \min(\text{RawEvaluatorAllocation}_i(d), \text{FeeBackedCap}_i(d))$$

The reference-price mechanism must resist short-duration market manipulation and does not rely on a single current-block spot price. It may incorporate sufficiently long Ethereum TWAP observations, realized settlement execution data, minimum-liquidity requirements, multiple approved observations or venues, deviation limits, circuit breakers, and other protocol-defined oracle protections.

An understated MACHINA reference price is the dangerous direction for this mechanism because it would permit a larger token allocation for the same USDC backing. The reference-price construction therefore uses conservative rules against temporary downward manipulation.

If the protocol cannot establish a valid reference price under the applicable oracle rules, Evaluator distribution for the affected period does not proceed and the undistributed allocation is transferred to the Protocol Treasury.

9.10 Canonical Evaluator Rewards

For each Intent, the finalized Evaluator allocation is paid only to the economic recipient of the current verified Canonical Evaluator, provided the Evaluator remains reward-eligible at the applicable snapshot.

Reward eligibility requires valid registration, finalized Canonical status, successful protocol verification, and satisfaction of the applicable availability and eligibility rules. Candidate Evaluators do not receive Canonical Evaluator emissions merely for participating in competition.

When a new Evaluator becomes Canonical, allocations finalized after that transition accrue to the new Canonical Evaluator.

Evaluator principal and registration bonds are not reduced as protocol penalties. Penalties may affect unfinalized emissions, Canonical status, eligibility, registration status, or future participation.

9.11 Protocol Treasury

The Protocol Treasury receives three principal inflows: 20 percent of scheduled MACHINA emissions, the 2 percent protocol fee on paid Consumer activity, and protocol emissions or economic allocations redirected to the Treasury under the protocol rules.

Evaluator emissions that cannot be validly distributed are transferred to the Treasury. This includes allocations above a fee-backed cap, allocations for an Intent without an

eligible Canonical Evaluator, deterministic remainder amounts, and the entire Evaluator pool when no Qualified Consumer Demand exists for the period.

Unused Evaluator emissions are not redistributed to other Intents and are not carried forward as later reward capacity.

Treasury expenditure may support protocol-authorized costs such as Ethereum transaction fees, Validator reimbursements, bootstrap activity, Miner evaluation-query reimbursement, development, security audits, infrastructure, incident response, ecosystem grants, and new-Intent bootstrap incentives under the applicable governance rules.

Treasury-funded activity remains separate from Qualified Consumer Demand and does not modify the fixed 60/20/20 emission split.

9.12 Signal Pricing

Each Miner declares a minimum USDC price for each supported Intent. At genesis, the protocol-wide minimum permitted price floor is 0.01 USDC, and a Miner may declare a higher minimum. The price charged for paid intelligence combines the Miner's registered minimum price with the applicable protocol Demand Multiplier.

$$\text{SignalPrice}_{(m,i)} = \text{min_price_usdc}_{(m,i)} \times \text{DemandMultiplier}_i$$

The Demand Multiplier is derived from protocol-recognized paid request volume for the applicable Intent using finalized or authenticated protocol accounting state from the applicable preceding pricing window. At genesis, the schedule is: 0-999 qualified requests = 1.0x, 1,000-9,999 = 1.5x, 10,000-99,999 = 2.5x, 100,000-999,999 = 5.0x, and 1,000,000 or more = 10.0x.

Only protocol-recognized paid Consumer requests count toward the pricing metric. Protocol evaluation traffic, free traffic, Treasury-funded requests, ranking activity, and other non-Consumer protocol traffic do not contribute to the Demand Multiplier request count.

A Miner's registered minimum price may be changed only through an authorized registration update. The protocol minimum price floor and Demand Multiplier schedule are governable economic parameters subject to the applicable timelock. The Demand Multiplier affects the Consumer price and is not applied again to the Evaluator emission-allocation formula.

9.13 Cumulative Reward Entitlements

Miner, Validator, and Evaluator rewards are maintained as cumulative lifetime entitlements. Each newly finalized period adds to the participant's previously finalized gross entitlement.

Claims use the difference between the latest finalized cumulative entitlement and the amount already withdrawn. Withdrawal does not reduce the historical cumulative entitlement recorded by the protocol.

Previously finalized cumulative entitlement is not clawed back by a later penalty, loss of Canonical status, or replacement by another participant.

$$\text{Claimable} = \text{Finalized_Cumulative_Entitlement} - \text{Previously_Withdrawn}$$

9.14 Economic Security Invariants

The economic model maintains several protocol-level invariants.

Miners receive no protocol emissions, their MACHINA entitlement originates from open-market MACHINA acquired with USDC generated by paid Consumer activity.

Validators receive the fixed 60 percent emission allocation subject to participation and penalty accounting, while Validator principal remains intact.

Evaluator reward value is bounded by the irreversible fees associated with the Consumer demand supporting that reward. At genesis, Evaluator reward backing is 80 percent of the corresponding 2 percent protocol fee.

Equivalent total paid demand produces equivalent aggregate raw Evaluator-emission weight regardless of how the demand is divided among wallets, Miners, or Intents, subject to deterministic rounding.

Reward backing is period-specific and cannot be reused in future periods. The protocol assigns no subjective difficulty multiplier to an Intent. Evaluator emissions that cannot be distributed under the economic-security rules become Treasury resources.

9.15 Adversarial Incentives and Multi-Role Participation

Telegraph assumes that economically motivated participants may control multiple protocol roles, multiple addresses, and multiple Intents. A Consumer, Miner, Evaluator, evaluation-material contributor, and Validator may therefore be economically related where the applicable role rules permit participation. Address separation is not treated as proof of economic independence.

The protocol does not depend on participants having no incentive to improve their own economic position. Miners benefit from routed paid demand and Canonical Evaluators may receive protocol rewards. The security objective is to prevent a participant from creating a profitable protocol-level shortcut around the competition, verification, accounting, and finalization rules.

Under the genesis fee-backing parameters, a participant that simultaneously acts as the paying Consumer, the serving Miner, and the Canonical Evaluator does not create new protocol value merely by recycling its own demand. The 2 percent protocol fee is irreversible. The corresponding Evaluator reward backing is limited to 1.6 percent of qualified paid demand, provided the protocol reference price remains valid. The Miner-settlement portion represents conversion of the paid USDC into MACHINA for the serving Miner.

Self-funded accounting margin = 2.0% protocol fee - 1.6% maximum Evaluator backing = 0.4%
before execution, liquidity, gas, slippage, and market risk

Splitting the same economic demand across additional wallets, Miners, or Intents does not increase aggregate raw Evaluator-emission weight or aggregate fee-backed reward capacity, apart from deterministic rounding. Creating additional addresses therefore does not increase the value supported by a fixed amount of finalized paid USDC under the Evaluator allocation formula.

These accounting constraints operate together with the protocol's ranking, contribution, provenance, oracle, routing, settlement, and consensus controls. Paid activity is treated

as an economic signal with explicit costs and verification requirements, not as proof that participating counterparties are economically independent.

9.16 Incentive Structure

Telegraph links economic opportunity to measured performance at both competitive layers. Higher-ranked Miners receive greater opportunity to serve paid Consumer demand. Stronger Evaluators can replace weaker Canonical Evaluators and become eligible for the rewards associated with the Intents they evaluate.

Paid demand therefore creates revenue opportunity for intelligence providers, while protocol emissions create an incentive to improve evaluation. Better intelligence can rise through Miner competition. Better evaluation can rise through Evaluator competition. Updated rankings can then redirect demand toward the strongest measured intelligence available under the current protocol state.

The economic loop is designed to reinforce the ranking system rather than substitute for it: demand finances intelligence supply, emissions finance protocol verification and evaluation, and finalized ranking state determines how demand is distributed.

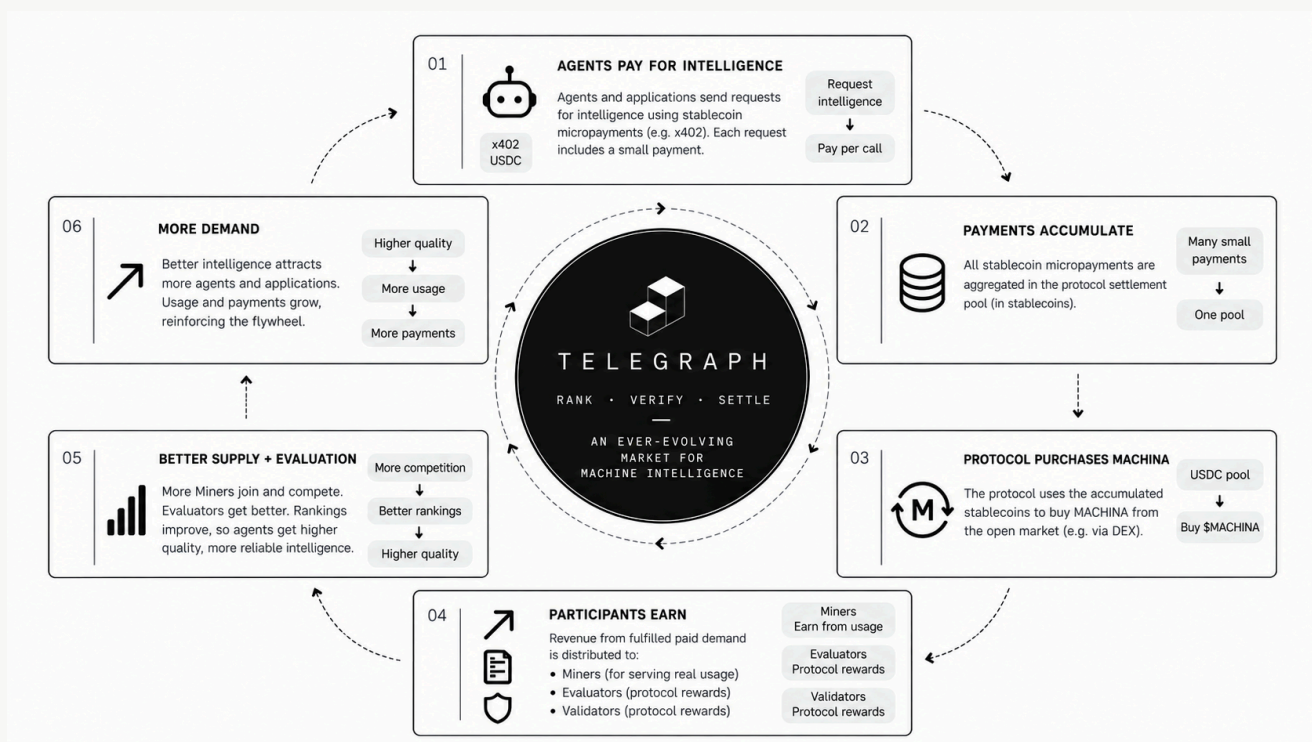


Figure 5. Paid Consumer demand creates open-market MACHINA demand and participant incentives, attracting stronger supply and evaluation, improving rankings, and reinforcing future demand.

10. BOOTSTRAPPING

Telegraph is designed to begin operating before a fully permissionless participant population necessarily exists. The bootstrap model allows the network to launch with the minimum Intents, Evaluators, Miners, Collectors, Validators, Autonomous Engine infrastructure, and public interfaces required to demonstrate the complete protocol.

Bootstrap infrastructure initializes operation, it does not create permanent economic or protocol privilege. Once the network is operating, the ordinary ranking, routing, evaluation, payment, and verification rules apply equally to bootstrap participants and later external participants.

The transition from bootstrap operation to open operation is therefore competitive and state-driven. External participants do not wait for a manual Company cutover to displace bootstrap infrastructure.

10.1 Bootstrap Model

At launch, the Telegraph Operating Company may operate bootstrap infrastructure including Foundation Miners, Collectors, Autonomous Engine nodes, Validators, bootstrap applications, and other services required to make the initial network usable.

Bootstrap or Treasury-funded activity is explicitly classified as protocol-funded activity. It remains separate from protocol-recognized paid Consumer demand for mechanisms that depend on independent economic usage, including Qualified Consumer Demand, Evaluator demand allocation, demand-based statistics, and other demand-weighted economic calculations.

A one-time bootstrap contribution creates no MACHINA allocation, repayment right, ranking privilege, routing privilege, Canonical Evaluator privilege, or governance privilege.

10.2 Genesis Intent Library

At genesis, ordinary MACHINA-denominated registration requirements may be temporarily impossible because circulating MACHINA is initially unavailable. Telegraph may therefore encode a limited Genesis Intent Library and the bootstrap participants required to make those Intents operational directly into genesis state.

Genesis state may include Intent configuration, initial Canonical Evaluators, evaluation material, Foundation Miners, Collectors, Validators and other required bootstrap state, together with the initial Foundation Model and training state where the applicable Intent architecture uses that subsystem.

The genesis exception applies only to initialization. Subsequent participant registrations, replacements, updates, and new Intents follow the normal permissionless protocol rules.

10.3 Foundation Miners

Foundation Miners are bootstrap intelligence providers used to ensure that a genesis Intent can produce useful Signals before sufficient external Miner participation exists. They are publicly identifiable as bootstrap infrastructure.

Foundation Miners receive no permanent routing preference or reserved market share. Before the first finalized Miner Ranking exists for an Intent, the applicable pre-ranking routing

rule is used. Once ranking state exists, Foundation Miners compete under the same ranking and routing rules as external Miners.

A better-performing external Miner can therefore gain ranking position and greater routing opportunity without Company approval or an explicit bootstrap-removal transaction.

10.4 Bootstrap Evaluators and Foundation Model State

A genesis Intent may require an initial Canonical Evaluator and the evaluation state needed to make ranking operational. Where applicable, genesis may therefore include evaluation material, initial Evaluator artifacts, Evaluator-generation or training artifacts, evaluation configuration, an initial Foundation Model version, and the training material required to initialize that shared backbone.

These artifacts initialize the evaluation system and do not create permanent control over an Intent. Subsequent Evaluator competition, permissionless evaluation contributions, Foundation Model updates, Canonical revalidation, and Canonical replacement follow the ordinary protocol mechanisms.

An external Evaluator that outperforms the bootstrap Evaluator and passes the required hidden verification can become Canonical through the same competitive process used after launch.

10.5 Bootstrap Validators and Finality

Telegraph may also begin with a bootstrap Validator population. A deployment operating below the Validator participation required for Byzantine fault-tolerant consensus is explicitly considered bootstrap operation.

Operation below the normal quorum-backed participation level does not provide the same decentralization or Byzantine-finality guarantees as normal protocol operation. The protocol does not represent bootstrap operation as equivalent to full quorum-backed finality.

As sufficient Validator participation becomes available, protocol operation transitions to the normal consensus and finality rules described in Section 6. The BFT threshold itself is not weakened to preserve liveness during the transition.

10.6 Bootstrap Collectors and Public Interfaces

Bootstrap operation may include Collectors, Autonomous Engine nodes, and public applications required to demonstrate request-driven and continuously generated intelligence. These components use the same separation of authority described in Section 7.

Collectors may supply external trigger data and bootstrap applications may generate protocol requests, but they do not control Intent identity, Miner Ranking, Canonical Evaluator selection, routing weights, or consensus state.

As external Collectors, applications, and Consumers enter, no protocol migration is required. They interact with the same registered Intents, finalized rankings, routing state, and payment interfaces.

10.7 Protocol-Funded Activity Classification

Bootstrap traffic, Treasury-funded requests, protocol evaluation traffic, and other protocol-generated activity remain separately classified from paid Consumer activity.

This separation is economically significant. Protocol-funded activity does not manufacture Qualified Consumer Demand, Evaluator demand-weighted emissions, or other economic statistics intended to measure external paid usage.

The same separation applies in public reporting so protocol-generated activity cannot be presented as ordinary paid Consumer demand merely because it traverses the same delivery infrastructure.

10.8 Intent Bounties

A new Intent may initially lack enough activity to establish or refresh useful Miner Ranking state. Telegraph may support USDC-denominated Intent Bounties or another explicitly funded bootstrap mechanism for this purpose.

An Intent Bounty funds the protocol evaluation activity required to obtain Miner responses, execute the current Canonical Evaluator, perform Validator verification, and produce a Miner Ranking.

Intent Bounties are protocol evaluation expenditure. They are not Consumer demand and do not increase Demand Multiplier request volume, Qualified Consumer Demand, Evaluator demand-weighted emissions, or organic-demand statistics. Reimbursable Miner calls made for a bounty remain classified as protocol evaluation expenditure.

10.9 Signal Dashboard and Market Visibility

Telegraph exposes public information about network activity through the Signal Dashboard or equivalent public interfaces. The objective is to make demand, ranking, routing, and competitive opportunity visible to prospective Miners, Evaluators, Consumers, and developers.

The public interface may expose current Miner Rankings, the leading Miner, active Miner count, current Canonical Evaluator, request volume, paid Consumer volume, settled USDC volume, routing status, Foundation Miner status, historical ranking information, and Miner revenue metrics.

Protocol-recognized paid Consumer activity is displayed separately from protocol evaluation traffic, bootstrap traffic, Treasury-funded activity, Intent Bounties, and free traffic. This distinction allows market participants to identify genuine usage signals, underserved Intents, and opportunities for new intelligence or evaluation supply without conflating protocol-funded activity with paid demand.

10.10 Automatic Transition to Open Operation

Telegraph does not require a centralized manual transition from bootstrap operation to open operation. Bootstrap participants remain subject to the same competition and verification mechanisms as equivalent external participants.

As external Miners outperform Foundation Miners, finalized Miner Rankings redirect routing opportunity. As external Evaluators outperform bootstrap Evaluators and pass verification,

Canonical Evaluator status changes through the normal Canonical-selection process. As paid Consumer activity grows, it becomes the demand measured by the economic system while bootstrap and Treasury-funded activity remain separately classified.

The bootstrap design therefore requires no permanent Company routing share, permanent Foundation Miner allocation, Agent Zero reduction schedule, or manual network cutover. Competition and finalized protocol state perform the transition.

11. GOVERNANCE

Telegraph governance allows the Active Validator Set to modify bounded protocol parameters while preserving the protocol's immutable economic, competitive, and consensus invariants.

Governance is deliberately narrower than protocol control. It may tune operating parameters as the network evolves, but it cannot appoint winners, weaken the Byzantine finalization rule, confiscate participant principal, rewrite finalized state, or alter the fixed MACHINA supply and emission structure.

Each active Validator has one governance vote independent of stake weight. Stake determines competition for Validator-set membership, once a Validator is active, governance voting power is one Validator, one vote.

11.1 Governance Activation

On-chain governance becomes active only when the Active Validator Set contains at least 43 Validators. Before that threshold is reached, all governable parameters remain fixed at their genesis values.

After activation, passage requires the Byzantine governance threshold derived from the Active Validator Set applicable to the proposal.

$$\text{Governance_Threshold} = \text{floor}(2 \times |V| / 3) + 1$$

At the genesis Validator Cap of 64, the threshold is 43 affirmative Validator votes. If the Validator Cap changes, governance does not retain a fixed 43-vote requirement, the threshold is recomputed from the applicable Active Validator Set.

Changing the Validator Cap therefore changes the number of votes required, while the minimum two-thirds-plus-one rule remains intact.

11.2 Governable Parameters

Governance may modify operational, economic, ranking, routing, participation, and security parameters that the protocol explicitly designates as governable. Changes apply prospectively after the applicable timelock and cannot alter finalized historical accounting or cumulative entitlements.

Parameter class	Governable examples
Validator and consensus	Validator Cap, assignment and liveness thresholds, Category A and C penalty thresholds, Validator unbonding, epoch or accounting-period duration where applicable, commit and reveal durations, Evaluator-verification subset size, Internal Audit Fee.
Registration and participation	Miner and Evaluator registration bonds, Intent registration or activation bond where applicable, Miner deregistration and Evaluator unbonding, maximum active Miner count per Intent where enabled, deterministic execution resource limits.
Evaluator and ranking	Minimum competition sample volume, statistical confidence threshold, Canonical replacement margin, cross-batch stability, hidden-batch count or size, evaluation-material admission limits, mature contribution-influence cap, bootstrap contribution-share parameters, contributor-diversity requirements, bounded execution tolerance, Evaluator competition cadence, Miner-ranking cadence.

Parameter class	Governable examples
Routing	Ranking-based routing weights, qualifying routing range, spot-check frequency, routing-revocation threshold, routing-recovery rules, protocol-defined operational failover parameters.
Economics	Protocol fee, minimum Signal price, Demand Multiplier thresholds and multipliers, Evaluator fee-backing coefficient beta, settlement cadence, oracle and reference-price parameters such as P_ref, subject to the protocol security requirements.

Governance cannot use these powers to select a particular Canonical Evaluator or to modify a finalized Miner Ranking. Canonical status and Miner Ranking continue to emerge from the protocol-defined competition, verification, aggregation, and finalization processes.

Governance also cannot grant a specific Miner routing preference outside the protocol-defined ranking and eligibility rules. Registration-bond principal remains protected by the no-principal-slashing invariant.

The Byzantine finalization threshold is derived from the Active Validator Set and is not an independently governable weaker quorum.

$$\tau = \text{floor}(2 \times |V| / 3) + 1$$

11.3 Governance Timelocks

Governance changes are delayed by parameter-specific timelocks so participants can observe an approved change before it becomes executable. The timelock begins only after the proposal has passed.

At genesis, structural changes use a 30-day timelock and ordinary operational, economic, statistical, routing, and security-parameter changes use a 14-day timelock.

Parameter class	Genesis timelock
Validator Cap	30 days
Validator unbonding	30 days
Miner / Evaluator unbonding	30 days
Ranking or Evaluator competition cadence	30 days
Evaluator statistical qualification parameters	14 days
Evaluation contribution and sampling parameters	14 days
Penalty and liveness thresholds	14 days
Registration bonds	14 days
Protocol fee	14 days
Minimum Signal price	14 days
Demand Multiplier schedule	14 days
Routing distribution	14 days
Spot-check parameters	14 days

Parameter class	Genesis timelock
Routing-revocation threshold	14 days
Internal Audit Fee	14 days
Other ordinary operational parameters	14 days

A proposal that changes several parameters uses the longest timelock applicable to any parameter in the proposal.

11.4 Immutable Protocol Invariants

Some properties are outside governance entirely. A proposal that attempts to violate an immutable invariant is invalid and cannot enter executable state.

Invariant	Governance restriction
MACHINA monetary policy	Maximum supply of 21,000,000, genesis emission schedule and halving rule, 60% Validator / 20% Evaluator / 20% Treasury emission split, zero premine, zero Team or VC allocation, zero Miner protocol emissions.
Principal protection	Validator, Miner, and Evaluator principal cannot be slashed, burned, confiscated, transferred, or reduced as a protocol penalty.
Finalized entitlements	Finalized cumulative economic entitlements cannot be retroactively clawed back.
BFT safety	Finalization must require at least $\text{floor}(2 \times V / 3) + 1$ of the applicable Active Validator Set.
Competitive selection	Canonical Evaluator status and Miner Rankings must continue to result from protocol-defined competition, verification, aggregation, and finalization.
No direct appointments	Governance cannot directly appoint a Miner, Evaluator, or Validator to a competitive protocol position.
Finalized state	Governance cannot retroactively rewrite finalized ranking, accounting, settlement, or consensus state.

11.5 Proposal Lifecycle

Any active Validator may submit a proposal that targets one or more explicitly governable parameters. The proposal records the proposer, the current and proposed values, activation conditions, the applicable timelock, and a rationale commitment or content-addressed reference.

Every proposal then passes through a public discussion period, a Validator vote, and, if approved, its required timelock before execution.

Stage	Duration	Rule
Submission	-	Any active Validator may submit an eligible proposal with the required parameter and rationale metadata.
Discussion	7 days	Public discussion period. Governance votes are not counted during this stage.

Stage	Duration	Rule
Voting	7 days	Only the Active Validator Set applicable to the proposal may vote. Each active Validator has one vote. Passage requires the governance threshold.
Timelock	14 or 30 days	A passed proposal waits for its applicable timelock. Multi-parameter proposals use the longest applicable delay.
Execution	Next applicable boundary	After the timelock expires, the change becomes executable and takes effect at the next protocol boundary applicable to that parameter. No retroactive effect is permitted.
Proposal passes when affirmative votes $\geq \text{floor}(2 \times V / 3) + 1$		

A proposal that fails to reach the required threshold expires without execution.

11.6 Emergency Veto

A proposal that has passed but has not yet executed may be cancelled through an emergency supermajority veto.

$$\text{Emergency_Veto_Threshold} = \text{ceil}(5 \times |V| / 6)$$

At an Active Validator Set of 64, the emergency veto threshold is 54 Validators.

A successful veto cancels the pending proposal. A materially equivalent proposal cannot be resubmitted for 90 days.

Emergency veto authority is defensive only. It cannot directly modify protocol state, execute a different parameter value, bypass the normal proposal process, or rewrite finalized state.

11.7 Governance Boundary

Governance controls parameter evolution, it does not control competitive outcomes. Miner ranking positions, Canonical Evaluator status, routing opportunity, reward eligibility, and Validator-set membership continue to follow their respective protocol mechanisms.

This boundary prevents governance from becoming an administrative override for the ranking network. The protocol can adapt operating parameters as conditions change while preserving the economic and security commitments on which participants rely.

12. SECURITY MODEL

Telegraph assumes that Miners, Evaluators, Validators, Consumers, evaluation-material contributors, and other participants may behave adversarially. A participant may control more than one role where the protocol permits it, and separate addresses are not treated as proof of economic independence.

The security model therefore does not depend on altruistic behavior. It combines economic admission costs, cryptographic commitments, authenticated provenance, deterministic execution, hidden evaluation, bounded contribution influence, independent verification, Byzantine fault-tolerant finalization, routing eligibility rules, and explicit accounting boundaries.

Telegraph produces verified rankings. The protocol verifies the execution, provenance, aggregation, ordering, and consensus processes that produce ranking state. It does not claim that every Miner response is objectively true, and authenticated provenance does not by itself establish semantic correctness.

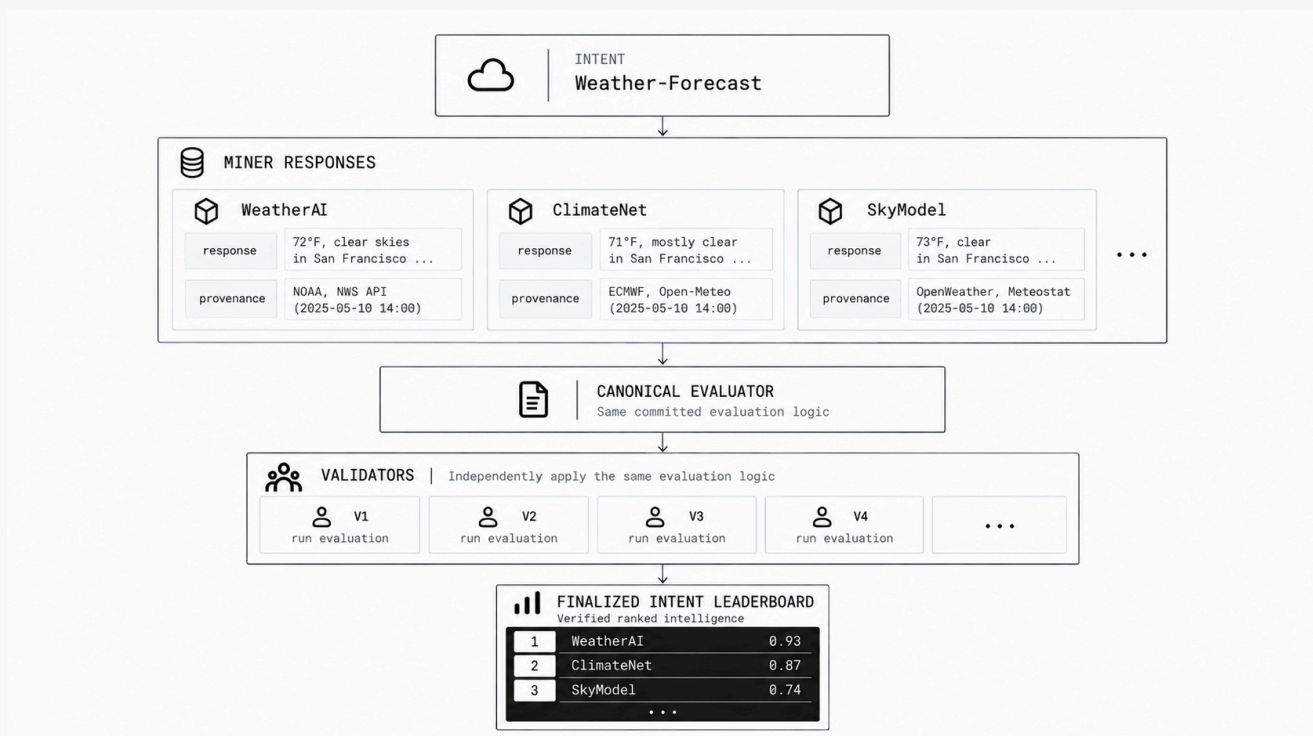


Figure 6. Within each Intent, Miner responses and provenance are measured by the Canonical Evaluator and independently reproduced by Validators to form a finalized leaderboard - verified ranked intelligence.

Attack surface	Primary control	Security outcome
Validator Byzantine behavior or failure	BFT quorum, deterministic verification, fixed finality rule	Unfinalized transitions cannot replace finalized state
Evaluator overfitting or false claims	Candidate competition plus hidden independent verification	Winning the public stage alone cannot make an Evaluator Canonical
Evaluation-material capture	Commitments, mechanical admission, bounded sampled influence	A single contributor cannot dominate mature hidden samples

Attack surface	Primary control	Security outcome
Ranking manipulation	Committed inputs, response provenance, independent execution, commit-reveal	Leader or individual Validator discretion cannot determine the ranking
Post-ranking degradation	Spot checks and Routing_Revocation	Degraded Miners can lose paid routing eligibility
Demand / reward manipulation	Qualified paid-demand accounting and fee-backed Evaluator cap	Self-funded demand cannot create unrestricted Evaluator emissions

12.1 Validator and Consensus Security

The Active Validator Set consists of the top N eligible Validator Profiles by staked MACHINA. At genesis, N = 64. Finality requires the Byzantine quorum derived from the active set.

$$\text{tau} = \text{floor}(2 \times |V| / 3) + 1$$

At the genesis Active Validator Set of 64, finality requires 43 distinct valid Validator participation. The quorum is not reduced during network partitions, outages, or periods of reduced availability. If the quorum is unavailable, new state does not finalize and the previous finalized state remains authoritative.

Validator penalties operate through unfinalized emissions, reward eligibility, active-set membership, status, and permanent profile revocation where objectively provable fraud is established. Validator principal is never burned, confiscated, transferred, or reduced as a protocol penalty.

The protocol distinguishes liveness failure, objectively provable cryptographic fraud, and incorrect deterministic execution so subjective disagreement about machine intelligence is not converted into a fraud judgment.

12.2 Evaluator Competition Security

Evaluator competition is deliberately split into a self-hosted candidate-ranking stage and an independent Canonical-verification stage. Candidate workloads may use standardized public, mock, historical, synthetic, or other non-sensitive material. Winning that stage only determines which candidate is verified first.

The presumptive winner must then be verified against its committed executable and hidden evaluation material by the assigned Validator subset. Hidden material is not sent to candidate-operated infrastructure. A failed candidate cannot become Canonical, verification advances to the next-ranked eligible candidate. If no challenger passes, the previous verified Canonical Evaluator remains active.

A material Evaluator change requires a new executable commitment and version. The implementation used during independent verification must match the registered commitment.

12.3 Shared Foundation Model Security

Where Intent-specific Evaluators depend on a shared Foundation Model, the shared backbone does not select or rank its own dependent Evaluators. Canonical selection remains external

to the Foundation Model and is determined through the protocol-defined competition and hidden-verification process.

A Foundation Model version change is security-relevant because the same Evaluator artifact may behave differently under a new backbone. A new `Foundation_Model_Version` therefore requires revalidation of affected Canonical Evaluators before that version becomes authoritative for those Intents.

Consensus state binds the Canonical Evaluator to the applicable Foundation Model, evaluation-pool, evaluation-engine, and other versioned execution inputs required to reproduce the evaluation context.

12.4 Evaluation-Material and Contribution Security

Evaluation material becomes eligible through prior commitment and deterministic admission rules. These rules may include schema validation, malformed-input rejection, size limits, duplicate detection, contribution-frequency limits, and versioning.

No single contributor may dominate the hidden material used for Canonical qualification. Let C_i denote the number of eligible independent contribution domains for Intent i . The maximum effective sampled contribution share is:

$$\text{MaxShare}_i = \max(0.02, 1 / C_i)$$

The rule supports small contributor sets during bootstrap and converges to a 2 percent maximum effective sampled share once at least 50 independent contributors participate. The cap applies to sampled influence, not merely to raw submitted row count.

Contribution controls may account for economically linked or otherwise attributable domains so simple address splitting does not automatically multiply influence. These controls reduce concentration and Sybil influence, they do not establish that all apparently distinct contributors are economically unrelated.

12.5 Hidden Sampling and Candidate Verification

Hidden Canonical-verification material is sampled only after the applicable candidate executable and evaluation state have been committed. The sample must be unpredictable before the protocol randomness is fixed, deterministic after selection, and reproducible by the assigned verification participants.

```
Hidden_Verification_Batch = Sample(Committed_Evaluation_State, Protocol_Randomness)
Verification_Seed = H(Protocol_Randomness || Intent_ID || Evaluator_Round_ID)
```

At genesis, the assigned verification subset contains $\text{ceil}(0.10 \times |V|)$ Validators, which equals seven Validators when $|V| = 64$. The subset retrieves the committed Evaluator executable, verifies its hash and version, derives the hidden sample, executes the artifact locally in the deterministic verification environment, and applies the Canonical qualification rules.

The candidate operator does not receive the hidden sample before or during independent verification. Evaluator registration principal is not slashed when qualification fails.

12.6 Miner Response and Ranking Integrity

Miner-ranking rounds commit the eligible Miner set, evaluation request or batch, collection window, valid Miner responses, provenance evidence, Canonical Evaluator commitment, and other deterministic execution inputs before the final ranking is accepted.

Where a Miner response requires transferable provenance, Telegraph uses zkTLS or another protocol-approved authenticated transcript mechanism. The evidence binds the authenticated Miner endpoint, Miner identity, Intent, ranking round, exact evaluation request, exact response, and protocol context. Invalid, substituted, or replayed evidence cannot produce a valid Local Score.

A timely valid Miner response required by the ranking round cannot be validly omitted from the committed response set. Validators then independently execute the current Canonical Evaluator against the same committed evaluation basis.

Local Scores use commit-reveal so Validators commit before seeing other Validators plaintext scores. Canonical Scores use the finalized stake snapshot and the protocol-defined stake-weighted median. Final Miner ordering is deterministic, including its Miner-ID tie-break.

12.7 Ranking-Round Leader Integrity

Ranking-round coordination follows a strict round-robin Leader schedule over the ordered Active Validator Set. The Leader role is operational: it coordinates the evaluation request, response collection, provenance generation, and Evaluation Package assembly. It does not choose the Canonical Evaluator or determine Miner scores.

$$\text{LeaderIndex} = \text{Round_ID} \bmod |V|$$

If the scheduled Leader fails within the applicable protocol window, the next Validator in the same round-robin order becomes eligible to continue the round. Leader replacement does not change the Canonical Evaluator, eligible Miner set, evaluation request, valid timely responses already received, or other committed ranking inputs.

Because the coordination order is predictable, ranking integrity does not rely on Leader unpredictability. It relies on committed inputs, completeness requirements, authenticated response evidence, independent Validator execution, deterministic aggregation, and BFT finalization.

12.8 Routing and Post-Ranking Security

Operational routing consumes finalized Miner Ranking state. Routing components cannot arbitrarily rewrite the ranking. Request-level failover may select another eligible Miner after an operational failure, but that failover does not modify ranking state.

Finalized rankings remain subject to protocol spot checks. At genesis, $k_{\text{spot}} = 10$ and a relative performance decline greater than 20 percent may produce a finalized Routing_Revocation. A revoked Miner is removed from active paid routing for the applicable period while its registration principal remains intact.

Persistent performance changes are therefore handled through ranking rounds, spot checks, and routing eligibility rather than through discretionary operational overrides.

12.9 Multi-Role Participation and Economic Manipulation

Telegraph assumes that one economic actor may participate as a Consumer, Miner, Evaluator, evaluation-material contributor, or Validator where the relevant role requirements permit it. The protocol does not infer economic independence from address separation.

Only finalized protocol-recognized paid Consumer activity qualifies for demand-dependent economic mechanisms. Evaluator competition, hidden verification, Miner-ranking requests, spot checks, bootstrap activity, Intent Bounties, Treasury-funded activity, grants, free traffic, and other internally generated protocol activity are excluded from Qualified Consumer Demand.

Evaluator rewards are additionally constrained by the irreversible protocol fee generated by the corresponding paid demand. At genesis, the protocol fee is 2 percent and $\beta = 0.80$, producing reward backing equal to 1.6 percent of qualified paid demand.

$$\text{RewardBackingUSDC}_i(d) = 0.80 \times 0.02 \times D_i(d) = 0.016 \times D_i(d)$$

The applicable Evaluator allocation cannot exceed the resulting fee-backed MACHINA cap. This prevents self-generated demand from creating unrestricted Evaluator emissions under the protocol accounting model. Splitting the same paid economic demand among wallets, Miners, or Intents does not increase aggregate raw allocation weight or fee-backed capacity except for deterministic rounding.

These accounting rules do not prove that a paid Consumer is independent of a Miner or Evaluator, and they are not a substitute for ranking, contribution, provenance, oracle, or consensus security. Multi-role participation remains part of the adversarial model.

12.10 Reference-Price and Settlement Security

The fee-backed Evaluator cap compares USDC-denominated backing with MACHINA-denominated emissions and therefore depends on a manipulation-resistant reference price. The protocol does not rely on a single current-block spot price.

The reference mechanism may use sufficiently long Ethereum TWAP observations, realized settlement execution data, minimum-liquidity requirements, multiple approved observations or venues, deviation limits, circuit breakers, and other protocol-defined oracle protections. An understated MACHINA reference price is the dangerous direction because it increases the token cap for a fixed amount of USDC backing.

If a valid reference price cannot be established under the applicable oracle rules, Evaluator emission distribution for the affected period does not proceed and the undistributed allocation is transferred to the Protocol Treasury.

Consumer payments are aggregated before MACHINA acquisition on Ethereum. Time-distributed settlement, execution limits, and deterministic entitlement accounting reduce predictable concentration and ensure Miner entitlements use the MACHINA actually acquired. Cumulative claims prevent repeated withdrawal of the same finalized entitlement.

12.11 Miner Credential Protection

Where a Miner service requires authenticated access, the Miner creates a dedicated Telegraph credential through its own provider and retains control over revocation and

rotation. Credentials should use the minimum permissions, rate limits, spend limits, expiration, and other restrictions supported by the underlying service.

The registered credential_hash binds the Miner registration to the encrypted credential package used by the protocol. Plaintext credentials are not stored on-chain.

For enclave-backed credential protection, Telegraph uses hardware-backed execution together with threshold cryptography, or an equivalent distributed confidential-compute mechanism. Distributed key material is maintained across authorized protected execution environments so no single Validator is intended to possess a persistent network-wide credential-decryption key.

Credential packages are encrypted before submission to the distributed Credential Store, and protocol state authenticates the active credential-management configuration and store commitment. During authorized use, decryption occurs inside the protected execution environment and reusable plaintext credential material is not exposed to the ordinary Validator host process.

Validator-set changes require the credential infrastructure to refresh, rekey, re-encrypt, or otherwise update distributed key material so removed Validators do not retain authority for future credential operations. These protections depend on the security of the selected threshold-cryptography, attestation, hardware, firmware, key-provisioning, and implementation stack.

12.12 Confidential Consumer Requests

Consumers may require stronger confidentiality than ordinary authenticated transport. For supported confidential requests, Telegraph derives an eligible Miner set from the requested security requirements and applies the current finalized Miner Ranking within that eligible set.

The Consumer encrypts the sensitive request for the approved attested execution environment. Decryption capability is provisioned only to that environment, the Miner executes there, and the result is encrypted for the authorized Consumer before leaving the protected context.

Ordinary Validator, routing, consensus, and settlement processes therefore need only the protocol metadata and commitments required for routing, accounting, verification, and settlement. They do not require the plaintext Consumer request, reusable Miner credential, or plaintext response.

If a confidential response must participate in protocol evaluation, the applicable evaluation may execute inside an authorized confidential environment using the Canonical Evaluator WASM, exposing only the evaluation output, commitments, and attestation evidence required for ranking verification.

The confidentiality guarantee remains conditional on the selected encryption, attestation, confidential-compute hardware, key-provisioning mechanism, and implementation. Telegraph does not infer legal or regulatory compliance from self-declared Miner metadata.

12.13 Operational, Collector, and Replay Security

Collector inputs are treated as untrusted external data. Collector content cannot directly select a Miner, alter a Miner Ranking, select a Canonical Evaluator, modify an Intent

registration, or change protocol economic state. Internal orchestration-to-Engine requests require cryptographic authentication or an equivalent authenticated mechanism.

Protocol commitments bind the Intent, round or job identifier, protocol version, state roots, identities, and other execution context required by the applicable mechanism. Payment authorizations, job resolutions, score commitments, and settlement claims use replay-protection rules so evidence valid in one context cannot be reused to create another valid protocol event.

Optional Transparency Attachments may contain reasoning traces, citations, references, confidence information, or provenance metadata and may be cryptographically bound to a Signal. Their presence does not prove that the attached reasoning or claims are correct.

12.14 Failure Recovery

Telegraph preserves previously finalized state when a new ranking, settlement, governance, or other consensus-relevant transition cannot obtain the required quorum. The protocol does not weaken consensus thresholds to maintain liveness.

Once sufficient Validator participation returns, normal verification and finalization resume from the last finalized state. Failed replacement ranking rounds therefore do not create competing finalized rankings.

Operational delivery may use request-level failover where defined, but operational failover does not substitute for protocol finality or rewrite previously finalized state.

12.15 Security Boundary

Telegraph verifies the process that produces ranking state. It authenticates and commits the relevant execution inputs, verifies the current Evaluator execution, aggregates Validator results deterministically, and requires Byzantine quorum before the resulting state becomes authoritative.

Ranking quality still depends on the participating Miners, the current Canonical Evaluator, the quality and diversity of evaluation material, the qualification rules, and the security assumptions of external cryptographic, oracle, transport, and confidential-compute systems used by the implementation.

Continuous Miner competition and continuous Evaluator competition allow those inputs to improve over time. The protocol therefore provides verified, reproducible ranking state under its stated assumptions, it does not claim universal truth or eliminate every external or economic attack surface.

PART II - REFERENCE

Normative mechanics, state, parameters, and procedures

Part I explains the Telegraph at the mechanism level. Part II defines the corresponding protocol rules, state, commitments, calculations, and procedures in normative detail. Where additional precision is required, the definitions in Part II govern.

The protocol reference preserves the logical separation between evaluation, ranking, consensus, routing, accounting, and settlement. A mechanism may reference finalized state produced by another mechanism, but that reference does not transfer authority between protocol roles.

A. PROTOCOL DEFINITIONS AND STATE

This section defines the common notation, canonical settlement domain, state commitments, sequencing rules, and cumulative-entitlement model used by the remaining protocol-reference sections.

A.1 Normative Scope

Protocol rules are evaluated against the finalized protocol version and state applicable to the event being processed. An implementation may choose internal data structures or execution strategies that differ from this document only where those choices do not change the externally observable protocol result, required commitment, eligibility rule, economic result, or finalization condition.

Consensus-critical arithmetic uses deterministic integer units. Floating-point arithmetic is not relied upon for consensus, entitlement calculation, or any state transition whose result must be reproduced by independent Validators.

A protocol transition that does not obtain the required finalization authorization does not replace the previously finalized state.

A.2 Notation

Symbol	Definition
$\mathbb{N}$	set of natural numbers
$[0,1]$	closed real interval used by Local Scores
$H(x)$	protocol cryptographic hash of x , component-specific hash functions are defined where interoperability requires them
$ $	concatenation
I	set of active Intents
P_i	set of registered Miners / Intelligence Providers for Intent i
M_i	eligible Miner set used by the ranking or routing mechanism for Intent i
$E_{reg,i}$	registered Evaluator set for Intent i
E_i	eligible Evaluator set used by the applicable Evaluator competition for Intent i
E_i^*	current finalized Canonical Evaluator for Intent i
R_i	current finalized Miner Ranking for Intent i
V	Active Validator Set
$ V $	number of active Validators
τ	Byzantine finalization threshold $\text{floor}(2 \times V / 3) + 1$
epoch / period	bounded protocol interval used by the applicable ranking, accounting, reward, settlement, or governance mechanism

Symbol	Definition
Signal	machine-consumable intelligence result delivered through Telegraph for a resolved Intent
USDC	Consumer payment and accounting asset
MACHINA	protocol-native economic unit of intelligence; native asset for settlement, rewards, and security

A.3 Core Protocol Objects

An Intent identifies a versioned machine-intelligence task and the compatibility boundary under which Miners may register, Evaluators may compete, ranking state may be produced, and Consumer requests may be routed.

A Miner registration binds a protocol identity and settlement identity to the Intents, endpoint configuration, pricing state, credential commitment where applicable, and configuration commitment required to call and interpret the service.

An Evaluator registration binds an Intent-specific Evaluator identity to the executable artifact, artifact hash, version, runtime metadata, evaluation-material commitments, economic recipient, bond, and eligibility state required by the Evaluator protocol.

For each active Intent, the protocol maintains the state required to determine the eligible Miner set, eligible Evaluator set, one finalized Canonical Evaluator, one finalized Miner Ranking, routing eligibility, and the evaluation and ranking commitments required to reproduce the applicable protocol computation.

A.4 Canonical Settlement Domain

Ethereum is the canonical economic settlement domain of Telegraph. MACHINA issuance and settlement, canonical entitlement commitments, economic state sequencing, and reward claims reside on Ethereum.

Consumer demand may originate from multiple supported networks or off-chain payment interfaces. USDC recognized by those interfaces is consolidated into the canonical settlement domain under the applicable payment and settlement rules.

Telegraph maintains a single canonical economic settlement domain so MACHINA market settlement, cumulative entitlement state, and participant claims remain unified across supported sources of demand.

Operational intelligence delivery does not wait for Ethereum settlement. Requests, Signals, usage accounting, and high-frequency sessions may complete before the corresponding economic state is committed on Ethereum.

A.5 Payment and Settlement Units

Consumer-denominated protocol activity is accounted for in USDC. Each successfully recognized paid delivery records the gross USDC value and the Miner attribution required by the economic protocol.

USDC allocated to Miner settlement is converted through the canonical Ethereum settlement mechanism. Final Miner MACHINA entitlement for a settlement period is based on the MACHINA actually acquired for that period.

Economic conversion, cumulative entitlement finalization, and participant claims may occur at different cadences from intelligence delivery.

A.6 Protocol State Root

Finalized consensus state is authenticated by a top-level commitment, `Protocol_State_Root`, associated with the applicable protocol state sequence. The top-level root may authenticate logically separate subroots including:

Authenticated subroot	Committed state
Ranking_State_Root	finalized Miner Rankings and associated ranking state for active Intents
Canonical_Evaluator_Root	current Canonical Evaluator identity, executable hash, version, and consensus-relevant configuration for each active Intent
Routing_State_Root	routing eligibility, finalized Routing_Revocation transitions, and other consensus-relevant routing state
Evaluation_State_Root	ranking-round commitments required for deterministic audit, including evaluation-package roots, eligible-Miner roots, Miner-response roots, score commitments and reveals, stake-snapshot identifiers, Canonical Scores, and round metadata
Miner_Entitlement_Root	cumulative lifetime Miner entitlements
Validator_Entitlement_Root	cumulative lifetime Validator emission entitlements
Evaluator_Entitlement_Root	cumulative lifetime Evaluator emission entitlements
Penalty_Status_Root	consensus-relevant participant penalty and eligibility state
Active_Validator_Set_Root	authenticated Active Validator Set state

The implementation may authenticate additional subroots required by later protocol versions, provided those roots are bound to the finalized protocol version and do not alter the meaning of existing finalized state.

A.7 State Submission and Sequencing

A canonical Ethereum state submission contains or cryptographically commits the `Chain_ID`, protocol version, `State_Sequence`, `Settlement_Period_ID` where applicable, `Validator_Set_ID` or `Active_Validator_Set_Root`, signer set or signer bitmap, `Protocol_State_Root`, applicable entitlement roots, aggregate Validator authorization, and any additional commitments required by the settlement contracts.

Canonical settlement contracts verify consensus authorization and enforce on-chain successor-state, settlement, and claim rules. They do not independently execute the complete Evaluator competition, Miner evaluation, ranking, routing, or reward-calculation logic.

The settlement contracts do not need to iterate over every Miner, Validator, Evaluator, Intent, or individual reward recipient. Economic participants claim independently using Merkle proofs against the applicable finalized cumulative entitlement roots.

A newly accepted state must satisfy the protocol successor rule and cannot replay a previously accepted economic period.

$$\text{State_Sequence_new} = \text{State_Sequence_previous} + 1$$

Settlement_Period_ID follows its applicable successor rule and cannot already have been finalized. Duplicate or previously accepted submissions cannot create duplicate entitlement increments, duplicate rewards, or duplicate settlement transitions.

A.8 Finalization Authorization

A consensus-relevant transition becomes authoritative only after the required Validator quorum authorizes the exact state commitment under the applicable chain, protocol version, Validator-set, state-sequence, and transition context.

The Byzantine finalization threshold is derived from the Active Validator Set and is never replaced by an arbitrary weaker quorum.

$$\tau = \text{floor}(2 \times |V| / 3) + 1$$

At an Active Validator Set of 64, $\tau = 43$. If fewer than τ valid authorizations are obtained, the proposed transition does not finalize and the previously finalized state remains canonical.

Aggregate signature or equivalent consensus proof establishes that the required Validator quorum authorized the committed protocol state. It does not prove that an underlying machine-generated response is objectively true.

A.9 Data Availability and Reproducibility

Consensus-relevant data required to reproduce a finalized transition must satisfy the applicable protocol data-availability rules before that transition can finalize.

For ranking state, the retained or retrievable audit package includes the Intent and round identifiers, eligible Miner-set commitment, exact evaluation request or batch, Miner identities, exact responses used for scoring, response commitments, authenticated provenance evidence, Evaluation_Package_Root, Miner_Response_Root, Canonical Evaluator identity and committed executable, applicable evaluation-material commitments and authorized data, Validator score commitments, revealed Local Scores, stake-snapshot state, Canonical Score aggregation, Miner-ranking computation, and finalized ranking.

Where evaluation material or Miner responses are confidential, replay access may be limited to authorized verification or attested execution environments while the cryptographic commitments required for protocol verification remain available.

Replay establishes whether a finalized transition was computed consistently with the committed inputs and protocol rules. Authenticated provenance establishes the integrity of the retrieval used by the protocol, it does not establish the objective correctness of the returned intelligence.

A.10 Cumulative Entitlement State

Miner, Validator, and Evaluator economic rewards use cumulative lifetime gross-entitlement accounting. A newly finalized entitlement root adds newly earned value to the previously finalized cumulative entitlement and does not subtract prior withdrawals from the committed lifetime total.

For participant p , the claimable amount is the proven finalized lifetime entitlement minus the amount already withdrawn.

$$\text{Claimable}_p = \text{ProvenLifetimeEntitlement}_p - \text{WithdrawnAmount}_p$$

A participant supplies a Merkle proof against the applicable finalized entitlement root. The settlement contract verifies the participant identity, proof, cumulative lifetime entitlement, finalized root, and previously withdrawn amount. Only a positive entitlement delta may be transferred.

After payment, WithdrawnAmount_p increases by the amount transferred. Withdrawal does not reduce the cumulative lifetime entitlement committed by the off-chain protocol ledger.

This cumulative model allows rewards to accumulate across multiple settlement or emission periods without requiring a participant to claim during each period.

Once finalized, cumulative entitlement is monotonic and is not retroactively reduced by later penalties, loss of Validator or Canonical status, replacement by another participant, or a later accounting period.

A.11 Separation of Operational, Consensus, and Economic Finality

Telegraph intentionally permits several cadences to coexist. A Signal may be delivered before a new ranking is finalized. Ranking state may be finalized before the related USDC-to-MACHINA settlement is executed. Economic entitlement may become claimable after the underlying Consumer activity has already completed.

The protocol connects these stages through authenticated request and response records, finalized ranking and routing state, usage accounting, settlement-period identifiers, entitlement roots, $\text{Protocol_State_Root}$ commitments, and canonical Ethereum successor-state rules.

The separation permits low-latency intelligence delivery while retaining deterministic ranking, aggregated economic settlement, and independently auditable cumulative claims.

A.12 Version and Context Binding

Consensus-critical commitments bind the identifiers and versioned context required by the mechanism that produced them. Depending on the mechanism, this includes Intent_ID , Round_ID or Job_ID , protocol version, Canonical Evaluator hash and version, Foundation Model version where applicable, evaluation-pool and engine versions, Validator-set state, settlement period, state sequence, and chain domain.

Evidence, authorizations, score commitments, job resolutions, and settlement submissions valid in one protocol context cannot be reused as authorization for an unrelated context merely because an underlying payload or participant identity is the same.

Material changes to an Intent interface, Evaluator artifact, Foundation Model version, or other compatibility-sensitive versioned state follow the explicit versioning and revalidation rules defined in the corresponding protocol-reference section.

B. INTENT AND MINER REGISTRATION PROTOCOL

This section defines the normative identity, versioning, registration, configuration, pricing commitment, credential binding, output mapping, update, and exit rules for Intents and Miners. Evaluator registration is specified separately in Part II C.

B.1 Intent Identity

An Intent is a uniquely identified, permissionlessly registered category of machine-intelligence task. The protocol derives the Intent identifier from the registrant, committed schema, and registration block context:

```
Intent_ID = H(Registrant_Address || Schema_Hash || Block_Number)
```

The Intent identity is used as the protocol namespace for Miner registration, Evaluator competition, ranking, routing, accounting, and settlement associated with that task. An Intent registration or activation bond may be required where enabled, the final genesis value, if required, is recorded in Part III.

B.2 Intent Versioning and Compatibility

Intents use semantic versioning to preserve compatibility between registered integrations. A material change to an Intent interface or execution requirement creates a new Intent version under the protocol versioning rules. A change from `weather_check@v1.0` to `weather_check@v1.1` is an example of such version progression.

Compatibility-sensitive Miner configuration is evaluated against the applicable Intent version. A change that alters a committed on-chain output slot layout requires an updated Miner registration or an Intent version change according to the applicable compatibility rules. Previously finalized state remains bound to the version under which it was produced.

B.3 Miner Registration

To serve an active Intent, a Miner submits an on-chain registration transaction and the required registration bond. Registration is permissionless: any address satisfying the applicable registration requirements and supplying a valid unified YAML configuration may register.

The registration binds the Miner operational service configuration to its on-chain identity and settlement state. The registration fields are:

Registration field	Normative meaning
Supported Intents	Array of Intent_ID values the Miner claims to serve.
Miner Settlement Address	EVM address to which the Miner MACHINA settlement entitlement is assigned.
Minimum Signal Price (min_price_usdc)	Miner-declared minimum USDC price for each supported Intent. The genesis protocol floor is 0.01 USDC, a Miner may declare a higher floor.
Credential Hash (credential_hash)	Content-addressed hash of the encrypted Miner API credential ciphertext where authenticated access is

Registration field	Normative meaning
	required. It binds the encrypted credential package to the Miner registration.
YAML Schema Hash and URI	Cryptographic hash and off-chain reference to the Miner unified declarative configuration.
Registration Bond	Anti-spam / participation bond required by the registration rules. The final genesis Miner bond value is to be specified before deployment.

Registration principal is protected by the protocol no-principal-slashing invariant. Protocol penalties may affect routing, eligibility, status, or unfinalized rewards where applicable, but the Miner registration principal is not burned, confiscated, transferred, or reduced as a protocol penalty.

B.4 Unified YAML Configuration

The unified YAML configuration defines the operational information required for Telegraph to call the Miner and process the resulting response. The on-chain registration commits the configuration through its schema hash and URI, the committed configuration therefore forms part of the registration version used by routing and execution.

The YAML also defines deterministic transformation of supported response fields when an on-chain consumer requires bounded structured output. Operational implementations may retrieve the referenced configuration off-chain, but they must verify it against the committed hash before using it as authoritative registration configuration.

B.5 Canonical OnChainData Mapping

Where a Miner response exposes fields for on-chain consumption, the YAML onchain_output block maps selected response fields into the canonical EVM structure:

```
struct OnChainData {
    address[] addresses,
    uint256[] integers,
    string[] strings,
    bool[] bools,
}
```

Each mapping declares the destination array type, slot index, field name, and source path. The registration mechanism validates the declared mapping against the canonical OnChainData structure. Downstream contracts may use the same committed schema to decode Telegraph outputs deterministically.

Slot indexes committed by a registration remain stable for that registration version. A modification that changes the slot layout is compatibility-sensitive and therefore requires an updated registration or Intent version under Section B.2.

B.6 Pricing Commitment

For every supported Intent, `min_price_usdc` is the minimum price the Miner is willing to accept. At genesis, the protocol-wide minimum Signal price is 0.01 USDC. A Miner may commit a higher minimum price.

A change to a committed Miner price requires an updated registration under the applicable update rules. The amount ultimately paid for an eligible request is determined by the protocol pricing mechanism, including the request-volume Demand Multiplier defined in the economic protocol. The registration price is therefore a Miner floor, not a complete replacement for protocol pricing.

B.7 Credential Commitment and Rotation

A Miner whose service requires authenticated access creates a dedicated Telegraph credential through its underlying service provider and commits the encrypted credential package by `credential_hash`. Plaintext credentials are not stored on-chain.

The credential package is content-addressed. Credential onboarding or rotation is authorized by the address controlling the Miner registration. A rotation creates a new encrypted package and a new `credential_hash`, after the updated registration state becomes active, new requests use the new package.

The Miner retains the ability to revoke or replace its provider credential independently. The full distributed Credential Store, threshold key-management, enclave, attestation, and confidential-execution procedures are specified in Part II K.

B.8 Registration Updates and Eligibility

Any change to committed Miner state that affects price, `credential_hash`, YAML schema commitment, supported Intent compatibility, or another versioned registration commitment must be authorized by the registration controller and enter protocol state through the applicable update mechanism.

Registration establishes the Miner as a participant for its declared Intents, it does not by itself guarantee paid routing. Routing uses the current finalized ranking and the Miner eligibility state applicable to the request. A Miner may lose routing opportunity through ranking performance, availability failure, `Routing_Revocation`, deregistration, or another protocol-defined disqualifying condition without loss of registration principal.

B.9 Miner Deregistration and Unbonding

A Miner may submit a deregistration transaction under the protocol exit rules. After deregistration, the Miner registration bond enters a seven-day unbonding period, subject to unresolved protocol obligations.

During the unbonding period the Miner receives no new routed Consumer demand. The Miner remains subject to evaluation, accounting, and other protocol obligations arising from its previously active registration. Settlement obligations created by requests fulfilled before deregistration continue through the normal settlement process.

At completion of the applicable unbonding conditions, the Miner principal is returned in full. The protocol does not impose principal slashing on Miner registration bonds.

B.10 Flow and Workflow Boundary

A Flow is one protocol-level intelligence operation. It resolves a request to an Intent, selects an eligible Miner according to finalized ranking and routing state, obtains the Miner response, records the usage required for economic accounting, and delivers the resulting Signal to the Consumer. Payment authorization may use x402 or another supported payment or session mechanism. Real-time delivery does not wait for final economic settlement.

A Workflow is application-level orchestration composed from one or more Flows and may contain state, memory, conditional execution, or multiple intelligence operations. Applications, Agents, and the Autonomous Engine may compose Flows into Workflows. Client or application logic determines how delivered Signals are combined, stored, interpreted, or used in later operations.

The protocol boundary remains Intent resolution, ranking-based routing, intelligence delivery, verification, accounting, and settlement. Higher-level orchestration does not alter the registration, ranking, or settlement rules defined by the protocol.

B.11 Registration-State Validation

An implementation treats a Miner registration as valid only when the on-chain registration is authorized, the referenced Intent version is active, the required bond condition is satisfied, the YAML commitment resolves to configuration matching its committed hash, declared onchain_output mappings satisfy the canonical OnChainData structure, and the registration is not in a deregistered or otherwise ineligible state for the operation being attempted.

Every request, ranking event, direct job, accounting record, and settlement attribution that references a Miner is bound to the applicable finalized registration and Intent context. Later registration updates do not retroactively alter the configuration or economic state under which an earlier finalized event was produced.

C. EVALUATOR PROTOCOL

This section defines the normative registration, execution, evaluation-material, competition, hidden-verification, Canonical-selection, versioning, and lifecycle rules for Evaluators. Evaluator participation is permissionless and Intent-specific. For each active Intent i , registered Evaluators form E_i and the protocol maintains one current Canonical Evaluator E_i^* .

C.1 Evaluator Registration and Eligibility

An Evaluator registers for a specific Intent through protocol state. Registration commits the information required for candidate competition, independent Validator verification, economic attribution, and eligibility.

Registration field	Normative meaning
Intent_ID	The active Intent for which the Evaluator participates.
Evaluator identity	The address controlling the Evaluator registration and associated protocol rights.
Economic recipient	The address to which any eligible Evaluator reward entitlement is attributed.
Externally callable endpoint	The self-hosted endpoint used during candidate-ranking competition.
Executable artifact reference	The content-addressed or protocol-supported reference from which the committed executable can be retrieved.
Executable hash	The cryptographic hash of the committed Evaluator implementation.
Evaluator version	The version identifier for the registered implementation.
Deterministic runtime metadata	The execution metadata required to reproduce the committed implementation deterministically.
Standardized input/output interface	The protocol interface through which candidate workloads and standardized results are exchanged.
Evaluation-material commitments	Commitments to applicable Intent-specific evaluation material supplied through the Evaluator ecosystem.
Registration bond and eligibility state	The anti-spam/Sybil-resistance bond and the current state determining whether the Evaluator may participate.

WASM is the committed executable format used by the current deterministic Validator-verification architecture. Candidate Evaluators operate their own competition infrastructure during the self-hosted stage. A material change to Evaluator behavior requires a new executable commitment and Evaluator version.

The registration bond provides participation friction. The principal remains owned by the registrant and is never reduced, burned, confiscated, or transferred as a protocol penalty.

C.2 Deterministic WASM Verification Environment

The candidate-ranking stage is self-hosted, while Canonical verification executes the candidate's committed WASM artifact inside the Validator verification environment. The artifact used during verification must match the executable hash committed by the registration.

The deterministic verification sandbox enforces the following execution constraints:

- no unrestricted filesystem access,
- no access to Validator private keys or the Validator host environment,
- network access only to protocol-authorized external resources declared or committed for the Evaluator,
- isolation from the Validator node process, and
- deterministic limits for execution time, memory, output size, and other protocol-defined runtime resources.

Where a mechanism permits a bounded numeric execution tolerance, the applicable tolerance is defined by that mechanism and is applied deterministically. It is not a general subjective scoring band.

C.3 Evaluator Logic and Evaluation Material

An Evaluator has two protocol-relevant components. Evaluator logic defines how Miner performance is measured and how the standardized protocol-facing result is produced. Evaluation material supplies the Intent-specific information required by that logic.

Evaluation material may contain task examples, reference information, expected or known outputs, ground truth, labels, rubrics, test cases, adversarial cases, structured evaluation inputs, and other Intent-specific measurement material.

The Evaluator developer is responsible for constructing the evaluation logic and obtaining or producing the material required by that logic. Dataset discovery, collection or scraping, cleaning, labeling, synthetic generation, ground-truth construction, rubric development, fine-tuning, and equivalent domain-specific work occur outside Telegraph.

Telegraph provides common machinery for registration, commitment, mechanical admission, contribution bounding, sampling, competition, verification, and finalization. It does not operate a separate corpus-construction or ground-truth pipeline for each Intent.

C.4 Evaluation Contributions and Commitments

Evaluation material intended to participate in Canonical Evaluator selection must be committed before use. An eligible contribution is bound to its Intent, material, metadata, and version:

```
Evaluation_Commitment = H(Intent_ID || Material_Hash || Metadata_Hash || Version)
```

Changing committed material creates a new version and commitment. Eligible committed contributions form the evaluation-material state for the Intent.

Admission is mechanical. The applicable rules include schema validation, malformed-input rejection, contribution-size limits, duplicate detection, contribution-frequency limits, contribution versioning, and minimum contribution requirements where defined by the protocol.

The protocol does not select a contributor dataset through subjective preference. Qualification emerges from the committed material state, bounded contributor influence, unpredictable sampling, candidate competition, deterministic qualification rules, independent verification, and continued competition.

C.5 Bounded Contributor Influence

No individual evaluation contributor may dominate the hidden material used for Canonical qualification. Let C_i be the number of eligible independent evaluation contributors for Intent i . For community-created Intents, the maximum effective sampled share attributable to one contributor domain is:

$$\text{MaxShare}_i = \max(0.02, 1 / C_i)$$

The adaptive rule permits bootstrap operation with a small contributor set and converges to a 2 percent mature-pool cap when at least 50 eligible independent contribution domains participate.

Eligible contributors	Maximum effective sampled share
4	approximately 25% per contributor domain
10	approximately 10% per contributor domain
25	approximately 4% per contributor domain
50 or more	2% per contributor domain

The cap applies to effective sampled influence, not raw submitted material volume. Contribution controls may account for economically linked or otherwise attributable contribution domains where practical so that trivial address splitting does not automatically multiply influence.

Genesis or protocol-bootstrap Intents may begin with predefined evaluation material before sufficient permissionless contributions exist. Bootstrap material receives no permanent preference after ordinary contribution participation becomes available.

C.6 Candidate-Ranking Stage

Candidate Evaluators remain independently operated and self-hosted during the initial competition stage. The protocol supplies a standardized workload using public, mock, historical, synthetic, or otherwise non-sensitive material appropriate to the Intent. Each candidate processes the workload through its registered endpoint and returns the standardized result required by the Intent evaluation specification.

For candidate e , the candidate-ranking result is derived from the standardized outputs according to the Intent-specific aggregation rule:

$$\text{Candidate_Score}_e = \text{Aggregate}(\text{Result}_e, 1, \dots, \text{Result}_e, N)$$

Candidate execution remains outside Validator infrastructure during this stage. Validators therefore do not execute the full $N_m \times N_e$ candidate matrix. The resulting ordering determines the sequence in which eligible candidates enter hidden Canonical verification. Candidate-ranking success alone does not grant Canonical status.

C.7 Hidden Qualification

The highest-ranked eligible candidate becomes the presumptive Canonical Evaluator. Promotion requires independent performance on committed evaluation material that was unavailable to candidate-operated infrastructure during the self-hosted competition stage. Hidden verification material is sampled only after the applicable candidate executable and committed evaluation state have been fixed. Conceptually:

```
Hidden_Verification_Batch = Sample(Committed_Evaluation_State, Protocol_Randomness)
```

The hidden sample must be unpredictable before the applicable randomness is fixed, deterministic after selection, reproducible by the assigned verification participants, and subject to the contribution-share limits in C.5.

Qualification may require minimum hidden evaluation volume, a minimum performance threshold, minimum statistical confidence, sufficient cross-batch stability, a minimum Canonical replacement margin, or another deterministic Intent-specific qualification rule. Multiple independently sampled hidden batches may be used where required. The applicable sample, confidence, margin, stability, cadence, and qualification thresholds are protocol parameters.

Hidden evaluation material is not distributed to the self-hosted candidate population before or during Canonical verification.

C.8 Validator Verification Subset

A bounded rotating subset of the Active Validator Set independently verifies the presumptive winner. At genesis, the subset size is 10 percent of the Active Validator Set, rounded upward:

```
VerificationSubsetSize = ceil(0.10 × |V|)
```

With 64 active Validators, VerificationSubsetSize = 7. The subset is selected deterministically from finalized protocol randomness and the applicable Evaluator competition context:

When no candidate requires Canonical verification for an Intent during the applicable period, no Evaluator-verification assignment is created for that Intent.

```
Verification_Seed = H(Protocol_Randomness || Intent_ID || Evaluator_Round_ID)
```

Each assigned Validator independently performs the required verification steps:

- retrieve the presumptive winner's committed executable,
- verify the artifact hash, Evaluator version, and registered execution metadata,
- derive the hidden verification sample from the committed evaluation state and finalized protocol randomness,
- execute the candidate locally inside the deterministic verification environment, and
- evaluate the resulting performance under the applicable hidden qualification rules.

The resulting subset selection and verification inputs must be independently reproducible by protocol participants from finalized state and the applicable commitments.

C.9 Canonical Promotion, Sequential Verification, and Mismatch Handling

If the presumptive candidate satisfies the applicable hidden qualification requirements, it becomes or remains the Canonical Evaluator. If it fails, verification advances to the

next-ranked eligible candidate and repeats under the applicable protocol-defined hidden-sampling procedure. If every challenger fails, the previously verified Canonical Evaluator remains active.

Candidate ranking and Candidate verification are separate stages. A candidate may self-report standardized results during competition. Before Canonical promotion, Validators execute the committed WASM artifact independently. If that artifact fails to reproduce the behavior claimed during competition within the applicable deterministic verification tolerance, the candidate fails verification and is removed from the current promotion path. Failed verification may affect unfinalized rewards, status, or eligibility according to the applicable protocol rules. Evaluator registration principal is never slashed.

C.10 Canonical Evaluator State

Each active Intent maintains one current Canonical Evaluator. Canonical state must bind enough versioned context to reproduce the applicable Canonical decision and subsequent Miner evaluation. The committed state includes, as applicable:

- Intent_ID,
- Canonical Evaluator identity,
- executable hash and Evaluator version,
- Foundation_Model_Version where the Evaluator depends on the shared Foundation Model,
- Evaluation_Pool_Version,
- Evaluation_Engine_Version,
- the applicable evaluation specification,
- activation round or activation state, and
- qualification metadata required to reproduce the Canonical decision.

A material change to the Evaluator artifact, Foundation Model version, evaluation-pool version, or evaluation-engine version creates a new execution context and therefore follows the applicable validation or revalidation process.

Canonical status creates no permanent incumbency advantage. A new Evaluator may register, enter candidate competition, become the presumptive winner, pass independent hidden verification, and replace the incumbent. The current Canonical Evaluator defines the protocol-facing measurement method used for Miner ranking for that Intent.

C.11 Shared Foundation Model and Revalidation

Telegraph may use a shared Foundation Model as a common computational backbone for Intent-specific Evaluators. Where an Evaluator depends on the shared backbone, its behavior is determined by the active Foundation Model version, the Evaluator artifact, and the applicable evaluation input:

```
Evaluator_Output = f(Foundation_Model_Version, Evaluator_Artifact, Evaluation_Input)
```

Evaluator competition and Foundation Model training are separate protocol lifecycles. A new Evaluator may compete for Canonical status without changing or retraining the Foundation Model, and a qualified replacement may become Canonical while the shared backbone remains unchanged.

Foundation Model retraining may occur separately using accumulated eligible training material and the applicable training pipeline. A successful retraining event creates a new `Foundation_Model_Version`.

Because a new backbone version may change the behavior of an unchanged Evaluator, activation of a new `Foundation_Model_Version` triggers revalidation of affected Canonical Evaluators before that backbone becomes authoritative for those Intents. Revalidation uses the applicable evaluation and hidden-verification infrastructure. An Evaluator that continues to satisfy the Canonical qualification requirements may remain Canonical, an Evaluator that no longer qualifies is replaced through the ordinary competition and Canonical-selection process.

The Foundation Model does not determine which Evaluator becomes Canonical. Canonical selection remains external to the backbone and follows the protocol-defined Evaluator competition and verification process.

C.12 Deactivation, Exit, and Principal Protection

A candidate Evaluator may be deactivated by the address controlling its registration under the applicable exit rules. Bond withdrawal remains subject to the applicable Evaluator unbonding requirements, unresolved verification events, and any status or reward consequences that must be resolved before exit.

Evaluator unbonding duration, registration-bond requirements, deterministic execution resource limits, competition cadence, hidden-verification parameters, contribution limits, contributor-diversity requirements, and bounded execution tolerances are governable only within the invariant boundaries defined in Part I Section 11 and Part II J. Governance cannot directly appoint a Canonical Evaluator or bypass the competition and verification process.

Evaluator principal remains protected throughout registration, competition, verification, deactivation, and exit. Protocol penalties operate through unfinalized rewards, status, eligibility, or participation rights, they do not reduce the underlying registration principal.

D. RANKING PROTOCOL

This section defines the normative process by which Telegraph evaluates eligible Miners for an Intent, derives Canonical Scores, produces a deterministic Miner Ranking, and activates that ranking through protocol finalization. It uses the current Canonical Evaluator defined in Part II C. Ranking production, routing, delivery, and economic settlement remain distinct protocol processes connected through finalized state and authenticated accounting.

D.1 Miner-Ranking Round

For each active Intent, Miner ranking proceeds through discrete ranking rounds. Before Miner results are known, the protocol fixes the execution context for the round. The fixed context includes:

- Intent_ID,
- Round_ID,
- the eligible Miner Set,
- the evaluation request or evaluation-request batch,
- the response-collection window,
- the current Canonical Evaluator,
- the applicable evaluation configuration, and
- the deterministic execution context required to reproduce the evaluation.

The eligible Miner Set is committed before ranking results are known:

```
Eligible_Miner_Set_Root = MerkleRoot(Eligible_Miners)
```

Eligible Miners are evaluated under equivalent conditions. Implementations may use batching, bounded concurrency, or distributed scheduling, provided the applicable evaluation inputs, collection window, and eligibility rules remain equivalent for the round.

A Miner required to participate in protocol evaluation must return its response within the applicable collection window. Non-response receives the protocol-defined failure treatment for the round and may reduce ranking position or routing eligibility. Non-response by itself does not reduce the Miner registration principal.

D.2 Ranking-Round Leader Rotation and Failover

One active Validator coordinates each Miner-ranking round. Leader selection follows a strict round-robin schedule over the ordered Active Validator Set applicable to the round. Each successive ranking round advances the Leader turn to the next active Validator in that order.

```
LeaderIndex = Round_ID mod |V|
```

The Leader coordinates the protocol-defined evaluation request, response collection, required provenance evidence, and construction of the committed Evaluation Package. The Leader does not select the Canonical Evaluator, choose Miner scores, alter the scoring method, or exercise discretion over the final ranking order.

If the scheduled Leader fails to complete its assignment within the applicable protocol window, the next Validator in the same round-robin order becomes eligible to continue the

round. Leader replacement does not change the Canonical Evaluator, evaluation request, eligible Miner Set, valid Miner responses already received within the applicable collection window, or other committed ranking inputs. A failed Leader assignment is handled under the Validator liveness rules in Part II E.

D.3 Miner Response Provenance

A Miner response used for ranking must be attributable to the registered Miner interaction on which the ranking computation relies. Where the Miner endpoint does not independently provide transferable authenticated evidence, the Leader produces a zTLS proof or another protocol-approved authenticated-transcript proof for the ranking interaction.

The provenance evidence binds, as applicable:

- the authenticated Miner endpoint,
- Miner_ID,
- Intent_ID,
- Round_ID,
- the exact evaluation request,
- the exact Miner response,
- the cryptographic commitment to the request and response where used, and
- the protocol-domain context required to prevent reuse in another ranking event.

The provenance mechanism establishes the integrity and origin of the request-response interaction presented for ranking. It does not establish that the semantic content of the Miner response is correct. Miner quality is determined by execution of the current Canonical Evaluator.

A ranking package containing invalid or substituted Leader-to-Miner transcript data does not finalize. Where the invalid package results from objectively provable fabrication, substitution, or alteration by the Leader, the violation is handled under the applicable Category B rules in Part II E.

D.4 Response Completeness

The ranking process commits both the eligible Miner Set and the valid Miner responses received within the collection window. A timely valid response from an eligible Miner cannot be selectively omitted from the Evaluation Package.

```
Miner_Response_Root = MerkleRoot(Valid_Miner_Responses)
```

A ranking package that violates the applicable response-completeness conditions is invalid. Completeness is therefore a property of the committed ranking basis, not a discretionary choice of the Leader.

D.5 Ranking Evaluation Package

All Validators evaluating a Miner-ranking round operate from the same committed evaluation basis. The round derives an Evaluation_Package_Root that binds the consensus-relevant inputs required to reproduce and audit the ranking computation.

```
Evaluation_Package_Root = H(Committed_Evaluation_Package)
```

The Evaluation Package binds, as applicable:

- Intent_ID,
- Round_ID,
- Eligible_Miner_Set_Root,
- the evaluation-request or request-batch commitment,
- the response-collection deadline or applicable collection window,
- participating Miner identities,
- Miner response commitments and Miner_Response_Root,
- provenance commitments and associated authenticated-transcript evidence,
- the current Canonical Evaluator identity,
- the Canonical Evaluator executable hash and version,
- the applicable evaluation-material or evaluation-configuration commitment,
- the protocol version, and
- other deterministic execution inputs required by the Canonical Evaluator or ranking mechanism.

Full evaluation data may remain off-chain where appropriate. Consensus-relevant commitments become part of finalized protocol state, while the corresponding data required for deterministic replay and audit must remain available under the protocol data-availability rules.

D.6 Independent Validator Evaluation

Validators assigned to the ranking round independently execute the current Canonical Evaluator against the committed Miner evaluation inputs. For Validator v , Miner m , and Intent i :

$$\text{Local_Score}_{(v,m)} = E_i(\text{Evaluation_Input}_m)$$

A Local_Score is a protocol execution result. Validators use the same Canonical Evaluator and the same committed Evaluation Package. Before accepting the score, the Validator verifies the applicable Intent, Evaluation Package commitment, Miner identity, Miner response commitment, provenance evidence, Canonical Evaluator identity, executable hash and version, deterministic execution environment, and resulting Local_Score.

Validators do not substitute alternative evaluation logic or modify the authenticated request or Miner response before execution. Where the applicable Evaluator permits a bounded numeric execution tolerance, that tolerance is defined by the relevant protocol mechanism and applied deterministically.

D.7 Local-Score Commit-Reveal

Validators commit to independently produced Local Scores before observing plaintext scores submitted by other Validators. For Validator v and Miner m :

$$\text{Score_Commitment}_{(v,m)} = H(\text{Validator_ID} || \text{Intent_ID} || \text{Round_ID} || \text{Miner_ID} || \text{Evaluation_Package_Root} || \text{Canonical_Evaluator_WASM_Hash} || \text{Local_Score}_{(v,m)} || \text{nonce}_{(v,m)})$$

During the commit phase, Validator v broadcasts the signed $\text{Score_Commitment}_{(v,m)}$. After the commit phase closes, the Validator reveals $\text{Local_Score}_{(v,m)}$ and $\text{nonce}_{(v,m)}$. The reveal is valid only if it reconstructs the earlier commitment. Missing or invalid participation is handled under the applicable Validator liveness and incorrect-execution rules.

The commitment does not reveal the underlying Local_Score. A reveal that does not reconstruct the prior commitment is invalid.

D.8 Canonical Score Aggregation

The protocol-facing Miner score is normalized to the closed interval $[0,1]$. An Evaluator may internally use any deterministic combination of models, datasets, algorithms, rubrics, or task-specific measurements, but its protocol-facing ranking result must resolve to the standardized score domain.

$$\text{Local_Score} \in [0,1]$$

A Miner requires at least the Byzantine quorum of distinct valid Validator score reveals before its Canonical Score may finalize:

$$\tau = \text{floor}(2 \times |V| / 3) + 1$$

At the genesis Active Validator Set of 64, $\tau = 43$. Aggregation uses the finalized Validator stake snapshot associated with the ranking round. Let S_{total} be the stake represented by Validators with valid revealed scores for the Miner. Scores are ordered numerically with their corresponding stake weights. The Canonical Score is the first score for which cumulative represented stake exceeds $S_{\text{total}} / 2$.

For each valid revealer v , $w_v = \text{EffectiveStake}_v$ at Stake_Snapshot. For the valid revealers contributing to Miner m , $S_{\text{total}} = \sum w_v$.

Stake, delegation, or Validator-set changes occurring after the finalized snapshot do not alter the weight applied to the current ranking round.

$$\text{Canonical_Score}_m = \text{StakeWeightedMedian}(\text{Local_Scores}_m)$$

D.9 Deterministic Miner Ranking

Canonical Scores produce the Miner Ranking for the Intent. Miners are ordered first by descending Canonical Score. Equal Canonical Scores are resolved by ascending Miner_ID.

$$R_i = \text{Sort}(M_i, \text{Canonical_Score DESC}, \text{Miner_ID ASC})$$

The ordering contains no discretionary Leader decision. The same valid finalized inputs therefore produce the same ranking order for every conforming implementation.

D.10 Ranking Finalization and Continuity

A newly computed Miner Ranking becomes active only after Byzantine fault-tolerant finalization under Part II E. Finalized ranking state commits the Miner Ranking together with the applicable Canonical Evaluator state, evaluation state, routing state, protocol sequence, and associated state roots.

If a replacement ranking round fails to finalize, the preceding finalized ranking remains authoritative, subject to the ordinary routing-eligibility rules. The protocol does not weaken its finalization threshold to force a new ranking through during reduced Validator availability.

Miner Ranking represents current measured performance. Each completed ranking round can produce new ordering. Historical rank provides no scoring advantage in a later round:

improving Miners may rise, degrading Miners may fall, and newly registered eligible Miners compete under the same applicable evaluation rules.

D.11 Ranking-Based Routing Boundary

Consumer routing consumes the current finalized Miner Ranking, routing does not create or modify ranking state. Before the first finalized ranking for an Intent, routing is uniform among eligible registered Miners. After ranking exists, routing is probabilistic and ranking-based, with higher-ranked eligible Miners receiving greater routing opportunity under the current routing policy.

Routing weights are protocol parameters and do not alter the underlying ranking. Operational delivery failure may cause request-level failover to another eligible Miner without changing ranking state. The complete routing protocol is defined in Part II F.

D.12 Spot Checks and Routing Revocation

Finalized ranking state remains subject to protocol spot checks so material Miner degradation can affect routing eligibility between ordinary ranking rounds. Let k_{spot} denote the spot-check frequency parameter. At genesis:

$$k_{\text{spot}} = 10$$

Spot-check selection derives deterministically from finalized protocol randomness. The selected Miner and the Validator responsible for initiating or verifying the check cannot choose the trigger after observing the selection. A spot-check event for Intent i may be derived conceptually as:

$$H(\text{Protocol_Randomness} \parallel \text{Intent_ID_i}) \bmod k_{\text{spot}} = 0$$

The exact source of Protocol_Randomness is defined by the consensus architecture and must derive from finalized state that cannot be selected unilaterally by the Miner or responsible Validator. During a spot check, the selected Miner receives an evaluation request compatible with the Intent and is measured using the current Canonical Evaluator. Spot-check activity is protocol evaluation traffic and is not classified as paid Consumer demand. If the selected Miner fails to respond within the applicable window, the failure is handled under the routing-eligibility rules.

If the measured performance declines beyond the applicable routing-revocation threshold, the protocol may produce a Routing_Revocation state transition. At genesis, the threshold is a 20 percent relative performance decline. The Routing_Revocation contains the consensus-relevant evidence required to verify the state transition. After Validator verification and finalization, the affected Miner is removed from active routing for the applicable period and its routing opportunity is redistributed among the remaining eligible Miners. Miner registration principal remains intact.

D.13 Ranking Lifecycle

Ranking is continuously refreshed and remains connected to the broader competitive system without merging those processes. Evaluator competition may replace the Canonical Evaluator under Part II C, subsequent ranking rounds then use the newly finalized Canonical Evaluator. Routing consumes the resulting finalized Miner Ranking, while payment and economic settlement are handled independently under Parts II G and H.

The resulting protocol sequence is: evaluation request → authenticated Miner responses → Canonical Evaluator execution → Validator Local Scores → commit-reveal → Canonical Scores → deterministic Miner Ranking → BFT finalization → routing.

E. VALIDATOR AND CONSENSUS PROTOCOL

This section defines Validator membership, key authority, protocol assignments, Byzantine finalization, state-submission duties, exit and unbonding, and the penalty rules that govern Validator participation. Validators verify protocol execution and finalize state, they do not independently determine the semantic truth of Miner intelligence.

E.1 Validator Profiles and Active Validator Set

Validator participation is represented by a Validator Profile registered on the canonical protocol contracts. The Active Validator Set V is determined by stake competition. At each epoch boundary, the top N eligible Validator Profiles ranked by protocol-recognized staked MACHINA become the Active Validator Set for the following epoch. Delegated MACHINA contributes to the Validator stake weight according to the delegation rules defined in Part II H.

Autonomous agents and other software operators may also act as Validators by registering a Validator Profile, staking MACHINA, and running the Validator software under the same participation, verification, consensus, and penalty rules as other Validator operators.

V_e = top N eligible Validator Profiles by staked MACHINA at epoch e

N is the Validator Cap. At genesis, $N = 64$. The Validator Cap is governable subject to the governance rules in Part II J. Stake determines competition for Active Validator Set membership, while Validator principal remains protected by the protocol no-principal-slashing invariant.

A participant may operate a Validator through ordinary software or autonomous software, provided the Validator Profile, stake, keys, protocol implementation, and eligibility requirements are satisfied.

E.2 Manager Key and Operator Key

Each Validator Profile separates control of economic principal from online protocol operation through two authorized keys.

- **Manager Key:** the high-security EVM address controlling the Validator Profile and its staked MACHINA. The Manager Key stakes, initiates unbonding, claims rewards, and authorizes or rotates the Operator Key.
- **Operator Key:** the online key used by Validator software for protocol operations, including signing verification results, consensus messages, state commitments, and other authorized protocol messages.

The Operator Key has no authority to withdraw or transfer the Validator's staked MACHINA. If an Operator Key is compromised, the Manager Key may authorize a replacement under the protocol key-rotation mechanism. Key rotation does not alter previously finalized protocol state.

E.3 Validator Assignments and Execution Duties

An active Validator must complete the protocol assignments allocated to it. Assignments depend on the protocol operation and may include:

- producing or verifying Local Scores and ranking computations,
- participating in Local-Score commit-reveal,
- verifying a candidate Evaluator during Canonical qualification,
- verifying a Routing_Revocation transition,
- signing consensus and protocol-state commitments,
- performing canonical state-submission duties when selected as Block Builder, and
- producing another verification result or protocol message required for finalization.

Validators must use the finalized protocol state, committed execution inputs, and protocol-defined deterministic computation applicable to the assignment. A required assignment not completed within its protocol window is handled under Category A. An objectively incorrect deterministic result is handled under Category C unless the evidence satisfies the higher Category B standard for cryptographically provable fraud.

Ranking-round Leader coordination is defined in Part II D. That role rotates strictly round-robin across the ordered Active Validator Set. Other randomness-dependent Validator assignments remain governed by their respective protocol rules.

E.4 Byzantine Finalization and Consensus Signing Domain

A ranking or other protocol-state transition becomes finalized only after the required quorum of the applicable Active Validator Set authorizes the corresponding state commitment. The Byzantine quorum is:

$$\tau = \text{floor}(2 \times |V| / 3) + 1$$

At the genesis Validator Cap of 64, $\tau = 43$. Each Validator signs only after completing the verification required for the applicable state transition.

Validator signatures are bound to the exact chain, protocol version, Validator set, state sequence, and protocol-state commitment:

```
Consensus_Message = H(Chain_ID || Protocol_Version || Validator_Set_ID || State_Sequence ||
                      Protocol_State_Root)
```

The consensus proof includes or commits to:

- Validator_Set_ID or Active_Validator_Set_Root,
- the applicable signer set or signer bitmap,
- State_Sequence,
- Protocol_State_Root, and
- the aggregate Validator signature.

A verifier of the finalized state confirms that the represented signers belong to the applicable Active Validator Set, that at least τ distinct active Validators signed, that the aggregate signature is valid for the committed Consensus_Message, and that the State_Sequence is the expected successor state. Eligible signatures may be aggregated using BLS or another protocol-supported aggregate-signature mechanism.

The aggregate authorization proves that the required Validator quorum authorized the committed protocol state. It does not prove that the semantic content of a machine-generated response is objectively true.

A deployment operating below the participation required for Byzantine quorum is bootstrap operation. It does not receive the same decentralization or Byzantine-finality guarantees

as normal quorum-backed operation. The launch and bootstrap transition rules are defined in Part II I.

E.5 Network Partitions and Failed Quorum

Telegraph does not reduce its Byzantine threshold to preserve liveness during a network partition, Validator outage, or communication failure. If fewer than τ valid signatures can be obtained, the proposed transition remains unfinalized and the previously finalized state remains canonical.

Consensus resumes from the last finalized state once the required quorum can again communicate and verify a valid successor transition. The protocol does not finalize competing successor states under automatically weakened quorum requirements.

E.6 Protocol State, Data Availability, and Deterministic Audit

Finalized consensus state is authenticated by `Protocol_State_Root` and the domain-specific authenticated subroots defined in Part II A. A Validator must not authorize a transition whose consensus-relevant data fails the applicable data-availability requirement.

For a finalized Miner-ranking transition, the historical data required to reproduce the computation must remain retained or retrievable under the protocol data-availability rules. The replay package includes the committed ranking context, exact evaluation request or batch, Miner identities and responses, response commitments, authenticated transcript evidence, `Evaluation_Package_Root`, `Miner_Response_Root`, Canonical Evaluator artifact and hash, applicable evaluation material or authorized evaluation data, Validator score commitments and reveals, stake-snapshot state, Canonical Score aggregation, ranking computation, and finalized Miner Ranking.

An auditor can verify the authenticated Miner interaction, reproduce the Canonical Evaluator execution, verify Local-Score commit-reveal, reproduce Canonical Score aggregation, reproduce the deterministic Miner Ranking, and compare the result with the finalized `Ranking_State_Root`. Where the relevant evaluation material or Miner response is confidential, replay access may be restricted to an authorized verification or attested execution environment while the corresponding commitments remain available for protocol verification.

Replay establishes consistency with committed protocol inputs and protocol rules. Authenticated provenance establishes integrity of the recorded interaction, neither property alone establishes the objective truth of the Miner response.

E.7 Block Builder and Canonical Ethereum State Submission

One active Validator is selected as Block Builder for each canonical state-submission period. Block Builder selection remains randomness-derived:

$$\text{BlockBuilderIndex} = H(\text{Protocol_Randomness} \parallel \text{"block"} \parallel \text{Period_ID}) \bmod |V|$$

The Block Builder does not determine or modify finalized state. Its responsibility is to package and submit the already-finalized consensus commitments to the canonical settlement contracts on Ethereum, including as applicable:

- `Protocol_State_Root`,
- `Miner_Entitlement_Root`, `Validator_Entitlement_Root`, and `Evaluator_Entitlement_Root`,

- `Validator_Set_ID` or `Active_Validator_Set_Root`,
- signer set or signer bitmap,
- aggregate Validator signature,
- `State_Sequence`,
- `Settlement_Period_ID`, and
- other settlement metadata required by the canonical contracts.

If the selected Block Builder fails to submit a valid finalized state within the applicable submission window, the next Validator in the deterministic failover sequence becomes eligible to submit it. Failure to complete the assigned submission duty is a Category A event.

The Ethereum settlement contracts verify consensus authorization and successor-state sequencing. A submission must reference the expected successor `State_Sequence` and applicable `Settlement_Period_ID`. The same finalized economic period cannot be applied twice, and a duplicate or previously accepted submission cannot create duplicate rewards, entitlement increments, or settlement transitions.

The settlement contracts do not independently execute the complete Miner evaluation, ranking, Evaluator competition, routing, or reward-calculation logic. They verify the consensus authorization and enforce the canonical on-chain state-transition, sequencing, settlement, and claim rules. A valid successor state is recorded only after those checks succeed.

E.8 Validator Exit and Unbonding

A Validator Profile enters unbonding when it voluntarily exits, is displaced from the Active Validator Set by stake competition, or is removed under an applicable protocol-ejection rule. At genesis, the Validator unbonding period is 21 days.

During unbonding, the stake remains locked and does not contribute to Active Validator Set competition. The unbonding interval provides deterministic active-set transitions, prevents rapid stake movement between Validator Profiles, and allows outstanding reward and protocol-accounting state associated with the prior active period to finalize.

Validator principal is returned in full after the applicable unbonding process. Protocol penalties may affect unfinalized emissions, reward eligibility, Validator status, or participation rights, they do not reduce the Validator's staked MACHINA.

E.9 Penalty Invariant and Evidence Classification

Validator penalties enforce liveness, correct protocol execution, and cryptographic integrity under one invariant: Validator principal is never slashed, burned, transferred, or confiscated as a protocol penalty. Penalties operate through emissions, reward eligibility, Validator status, Active Validator Set membership, and permanent participation rights where objectively provable fraud is established.

The protocol distinguishes three penalty categories:

- Category A - failure to perform required protocol work,
- Category B - objectively provable cryptographic fraud, and
- Category C - incorrect or persistently inconsistent deterministic protocol execution that does not meet the Category B evidentiary standard.

Subjective model disagreement, evaluation-quality disagreement, statistical uncertainty, or behavior inside an explicitly permitted execution tolerance is not Category B fraud. A stale-state, configuration, software, synchronization, or other non-fraudulent deterministic mismatch is handled under Category C where applicable.

E.10 Category A - Liveness Failure

A Validator that fails to complete a required assignment does not earn the Validator emission associated with that assignment. The forfeited emission is redistributed among eligible Validators that successfully completed the applicable protocol work according to the reward-distribution rules.

A Validator is Liveness-Ejected if either condition is satisfied:

1. required protocol assignments are missed continuously for an unbroken period of 7 days, or
2. more than 50 percent of required protocol assignments are missed within any rolling 7-day window.

A successful required assignment resets the continuous-failure counter. Upon Liveness-Ejection, the Validator is removed from V and enters the standard 21-day unbonding process. Liveness failure does not reduce the Validator's staked MACHINA. After satisfying the applicable registration and eligibility requirements, the operator may later compete for Active Validator Set membership again.

E.11 Category B - Cryptographically Provable Fraud

Category B applies when a Validator submits or signs protocol data that can be objectively demonstrated through cryptographic evidence to be fraudulent, fabricated, or inconsistent with the committed protocol inputs. Examples include:

- submitting fabricated or cryptographically invalid source data,
- submitting a protocol payload whose committed data does not match the data actually received or executed,
- falsely attesting that an Evaluator artifact, hash, execution result, or verification result was valid when the protocol can cryptographically demonstrate otherwise,
- signing a state transition containing data known by the Validator to violate the applicable cryptographic commitments,
- forging, altering, or substituting protocol evidence, or
- another objectively provable form of protocol-level cryptographic fraud defined by the protocol.

When Category B fraud is finalized:

1. the Validator forfeits all Validator emissions attributable to the affected epoch that have not already been finalized for withdrawal,
2. the Validator immediately becomes ineligible for additional Validator emissions,
3. the Validator Profile is removed from the Active Validator Set,
4. the Validator Profile is permanently revoked,
5. the Manager Key and Operator Key associated with the fraudulent Validator Profile become permanently ineligible to register or operate another Validator Profile, and

6. the Validator's staked MACHINA enters the standard unbonding process and is returned in full after the applicable unbonding period.

Forfeited emissions are redistributed according to the protocol reward-distribution rules. No Validator stake is burned, transferred to the Protocol Treasury, or otherwise reduced. Category B therefore separates ownership of MACHINA principal from the right to participate as a Validator.

E.12 Category C - Incorrect Protocol Execution

Category C applies when a Validator produces protocol outputs inconsistent with the deterministic computation, committed state, or verification result required for its assignment, without sufficient evidence to classify the behavior as Category B fraud. Examples include:

- submitting an incorrect deterministic Miner-ranking result,
- signing an incorrect state commitment,
- submitting a candidate-Evaluator verification result inconsistent with the required deterministic execution,
- using obsolete protocol state or an obsolete Canonical Evaluator commitment,
- repeatedly submitting malformed or invalid consensus messages, or
- producing another objectively incorrect protocol result below the Category B fraud standard.

A single qualifying incorrect assignment causes forfeiture of the Validator emission for that assignment. The forfeited amount is redistributed among eligible Validators that successfully completed the corresponding protocol work. Each qualifying Category C event also produces one strike.

At genesis, Category C ejection occurs after 5 strikes within any rolling 30-day window. The Validator is then removed from V and enters the standard 21-day unbonding process. Category C ejection is not permanent: after correcting the software, configuration, synchronization, or execution problem, the operator may later compete for Active Validator Set membership again. Validator principal remains intact.

Where a protocol computation permits an explicit numerical verification tolerance, a result inside that mechanism-specific tolerance is valid. The tolerance is not a general subjective scoring band.

E.13 Validator Evidence and Evaluator Verification Mismatch

Penalty classification follows the evidence available for the affected protocol event. Category B requires objective cryptographic evidence of dishonest protocol behavior. Category C covers objectively incorrect execution where that higher evidentiary standard is not met. Category A covers non-performance.

Candidate Evaluator ranking and Candidate Evaluator verification remain separate stages. A candidate may self-report results during competition, but Canonical promotion depends on Validator execution of the committed executable against the applicable hidden verification inputs. If the committed executable fails to reproduce the required behavior within the applicable verification tolerance, the candidate fails verification and the promotion path advances according to Part II C. A Validator that knowingly falsifies the corresponding

cryptographic verification evidence is subject to Category B, an ordinary incorrect deterministic verification result is handled under Category C.

No penalty category reduces Validator principal, and previously finalized cumulative economic entitlements are not retroactively clawed back as a substitute for slashing.

F. ROUTING PROTOCOL

This section defines the normative rules by which finalized Miner Ranking state is converted into request-level Miner selection and Signal delivery. Routing consumes ranking and eligibility state, it does not create ranking state, select a Canonical Evaluator, or determine Miner quality. Payment authorization and economic settlement are specified in Part II G.

F.1 Routing Inputs and Eligible Miner Set

For an ordinary routed request associated with Intent i , the routing layer reads the current finalized Miner Ranking R_i together with the current Miner eligibility state. Only Miners that are eligible for the request may receive routed demand.

Eligibility may require, as applicable:

- an active Miner registration for the resolved Intent,
- compatibility with the applicable Intent version and registered integration requirements,
- satisfaction of the applicable pricing requirements,
- absence of an active Routing_Revocation or other disqualifying protocol status,
- availability under the operational routing policy, and
- satisfaction of any supported Consumer routing restrictions defined in F.10.

Where the number of registered Miners for an Intent exceeds the capacity supported by the applicable evaluation or routing mechanisms, the protocol may enforce a maximum active Miner count according to governable protocol parameters. Registration alone does not guarantee active routing eligibility.

F.2 Pre-Ranking and Ranking-Based Routing

Before the first finalized Miner Ranking exists for an Intent, ordinary routing is distributed uniformly among eligible registered Miners for that Intent. Once a finalized ranking exists, routing transitions to the ranking-based policy.

After ranking is available, routing is probabilistic across the qualifying portion of the eligible Miner Ranking. Higher-ranked eligible Miners receive greater routing opportunity. The exact routing distribution, qualifying range, and routing weights are protocol parameters and may be modified through the applicable governance process.

The routing policy does not require a universal $P(m) = \text{Score}_m / \sum \text{Score}_j$ formula. Different Canonical Evaluators may operate over different score scales and distributions, so routing probability is derived from finalized ranking position and governable routing-policy parameters rather than assuming raw scores are universally comparable.

Miner participation is economically bounded by registration requirements, operating cost, measured ranking performance, and Consumer demand. A Miner that consistently performs poorly may receive little or no routed paid demand.

Routing weights affect demand distribution only. They do not modify Canonical Scores, alter the deterministic Miner Ranking, or create a new ranking state.

F.3 Request-Level Operational Failover

A Miner selected under the routing policy may fail to complete an individual request because of unavailability, quota exhaustion, transport failure, or another supported operational failure. The Engine may then attempt another eligible Miner according to the routing-failover policy.

Operational failover is request-local. It does not change the finalized Miner Ranking, Canonical Score, or routing weights. Persistent performance or availability failures are handled through subsequent ranking rounds, spot checks, eligibility rules, and Routing_Revocation as defined in Part II D.

F.4 Intent Resolution

Intent resolution occurs after a request reaches the Engine. The resolver maps the request to one or more compatible registered Intents. Miner selection occurs only after the applicable Intent and eligibility context have been established.

Free-form request text, Collector output, or question-generation output does not carry authoritative Miner-selection instructions. Ordinary Miner selection is derived from finalized ranking and eligibility state.

F.5 Autonomous Engine Architecture

The Autonomous Engine is the execution and routing layer that converts direct Consumer requests or externally generated events into Miner requests and Signals. It contains three logical layers:

1. Collectors,
2. autonomous orchestration and question generation, and
3. the Engine and Miner router.

These logical layers may execute in one node binary or as separate services. The Autonomous Engine consumes finalized protocol state. It does not determine Miner Rankings, select Canonical Evaluators, grade Miner responses, or perform consensus.

F.6 Collectors

Collectors connect Telegraph to external information and trigger sources. Sources may include APIs, market feeds, event streams, blockchain state, databases, scheduled sources, or other machine-readable systems.

A Collector is a self-contained registered integration that defines how its source is accessed, triggered, and normalized. Collector output is input data. A Collector does not determine Intent_ID, select a Miner, alter Miner Ranking, assign routing weight, select a Canonical Evaluator, or determine an evaluation outcome.

F.7 Autonomous Orchestration and Question Generation

The orchestration layer executes Collectors according to their schedules or trigger conditions and converts relevant trigger data into one or more machine questions or tasks:

External Source -> Collector -> Trigger Data -> Question / Task Generation -> Engine

Question generation may use an LLM or another compatible generation mechanism. No specific commercial model provider is a protocol requirement. Generated questions are treated as requests to be resolved by the Engine and do not carry authoritative Miner or Intent routing instructions.

F.8 Engine Request Processing

For each direct or autonomous request, the Engine performs the applicable processing sequence:

- receives the request or generated question,
- resolves one or more compatible Intents,
- reads the current finalized Miner Ranking and Miner eligibility state,
- selects an eligible Miner under the routing policy,
- constructs the Miner-specific request from the registered integration metadata,
- dispatches the request,
- performs operational failover where necessary,
- receives the Miner response, and
- produces the resulting Signal and delivery metadata.

The request-driven path is:

```
Consumer Request -> Engine -> Intent Resolution -> Miner Ranking -> Eligible Miner -> Miner
Response -> Signal -> Consumer
```

The autonomous path is:

```
External Source -> Collector -> Trigger Data -> Question Generation -> Engine -> Intent
Resolution -> Miner Ranking -> Eligible Miner -> Miner Response -> Signal -> Subscriber
```

Signals may be distributed through API, WebSocket, Intelligence Terminal, subscriptions, or other supported delivery interfaces.

F.9 Collector and Internal-Request Security

Collector inputs are untrusted external data. Collector content cannot directly select a Miner, alter Miner Ranking, select an Evaluator, modify an Intent registration, or change protocol economic state. Intent resolution and routing occur independently after Collector output has become a request.

Internal orchestration-to-Engine requests must be authenticated. A plaintext internal header is not sufficient authority for bypassing payment or access controls.

Daemon-to-Engine requests use cryptographic authentication or another protocol-supported authenticated internal-request mechanism.

F.10 Restricted and Policy-Constrained Routing

A Consumer may apply supported eligibility restrictions to the Miner set used for a request. Such restrictions may include:

- explicit Miner allowlists,
- required execution-environment attestations,
- required security capabilities,

- geographic or jurisdictional metadata where the protocol provides an applicable verification mechanism,
- organizational or counterparty policies, and
- other Intent-compatible routing constraints.

The current Miner Ranking is applied within the resulting eligible set. Restricting eligibility does not rewrite the underlying global Miner Ranking. Telegraph does not infer legal or regulatory compliance solely from Miner self-declared metadata, applications remain responsible for applying policies appropriate to their use case.

F.11 Confidential Routing Boundary

Where a Consumer requests confidential execution, the routing layer derives the eligible Miner set from the applicable confidentiality and policy requirements and then applies the current Miner Ranking within that set. A selected Miner must support the required confidential-compute capability and applicable execution attestation.

Confidential execution does not require ordinary routing, Validator, consensus, or settlement processes to receive plaintext Consumer payloads. Detailed encryption, attestation, key-provisioning, confidential evaluation, credential protection, and compromise-containment rules are specified in Part II K.

F.12 Signal Identity and Auditability

A Signal contains or is associated with sufficient metadata to reconstruct its execution path. Applicable metadata may include:

- Signal_ID,
- signal_hash,
- Intent_ID,
- Miner_ID,
- request commitment,
- response commitment,
- Collector identity where applicable,
- trigger commitment where applicable,
- routing metadata, and
- payment or protocol-generation classification.

An autonomous Signal is traceable through:

```
Collector -> Trigger -> Question -> Intent -> Miner -> Response -> Signal
```

A request-driven Signal is traceable through:

```
Consumer Request -> Intent -> Miner -> Response -> Signal
```

F.13 Direct-Job Selection Boundary

The direct on-chain job rail defined in Part II G permits a funded job to specify a Miner_ID explicitly. A valid explicit direct-job selection is not overridden by normal probabilistic routing. The selected Miner must remain actively registered, compatible with the requested Intent, available, within the applicable pricing requirements, and free of an active Routing_Revocation or other disqualifying status.

This direct-job exception changes request-level Miner selection for that job only. It does not modify the finalized Miner Ranking or the ordinary routing policy used for requests without explicit Miner selection.

F.14 Routing, Delivery, and Settlement Separation

Ranking production, operational routing, intelligence delivery, and economic settlement are separate protocol processes connected through finalized state and authenticated accounting. Real-time Signal delivery does not wait for final economic settlement.

Paid request-response, streaming, session, subscription, and direct-job activity enters the payment and settlement mechanisms defined in Part II G. Routing determines which eligible Miner receives the request, settlement determines how the resulting paid activity is accounted for and economically finalized.

G. PAYMENT AND SETTLEMENT PROTOCOL

This section defines the normative payment, delivery-accounting, direct-job, callback, and settlement rules used by Telegraph. Consumer economic activity is denominated in USDC. Paid delivery may occur synchronously, through pre-funded high-frequency sessions, or through direct on-chain jobs. These delivery rails converge into the same protocol accounting and canonical Ethereum settlement architecture.

For every successfully paid intelligence delivery, the gross paid amount is divided according to the applicable protocol-fee rule. At genesis, 2 percent is allocated to the Protocol Treasury and 98 percent is attributed to Miner settlement. Real-time intelligence delivery does not wait for USDC-to-MACHINA conversion, cumulative entitlement finalization, or participant withdrawal.

G.1 Canonical Payment and Settlement Architecture

Ethereum is the canonical economic settlement domain. Canonical settlement contracts maintain or authenticate the protocol commitments required for settlement, sequencing, and participant claims, including MACHINA settlement state, Protocol Treasury state, finalized protocol-state commitments, Active Validator Set state, cumulative entitlement roots, settlement sequencing, and reward claims.

Telegraph separates four stages of a paid intelligence operation:

1. Consumer payment authorization,
2. intelligence delivery,
3. USDC accounting and Miner attribution, and
4. canonical Ethereum economic settlement.

Consumer payment authorization may originate through supported payment networks, managed integrations, or pre-funded escrow. Telegraph does not require every request, response, WebSocket delivery, or acknowledgement to create an Ethereum transaction. High-frequency activity may be authorized and accounted for off-chain, subject to the protocol exposure limits defined below, and settled in aggregate.

All payment records entering protocol accounting must bind sufficient protocol context to prevent replay, cross-session reuse, cross-request reuse, and duplicate economic attribution. Finalized accounting for one paid delivery must not be counted more than once.

G.2 x402 Synchronous Request Flow

Telegraph supports x402-compatible synchronous payment flows. A standard paid request proceeds as follows:

1. A Consumer submits a request associated with an Intent.
2. Telegraph determines the applicable SignalPrice under the economic rules in Part II H.
3. If valid payment authorization is absent, Telegraph returns HTTP 402 Payment Required together with the applicable USDC payment requirements.
4. The Consumer supplies a valid payment authorization.

5. Telegraph verifies the authorization, including the required amount, asset, destination, expiry, and replay-protection context.
 6. The request is routed to an eligible Miner according to the routing rules in Part II F.
 7. The Miner returns the intelligence result.
 8. Telegraph records the paid delivery and exact Miner attribution.
 9. The Consumer receives the result and the applicable protocol receipt.
- At the genesis fee split, the accounting for a successful paid request is:

```
ProtocolFee = 0.02 × SignalPrice
MinerSettlementUSDC = 0.98 × SignalPrice
```

The Miner does not wait for an immediate MACHINA conversion. Miner-attributable USDC enters the aggregated settlement ledger and is processed through the canonical Ethereum settlement mechanism. Each authorization and paid-delivery record must be uniquely bound to its applicable payment and protocol context so that the same authorization or delivery cannot create duplicate accounting.

G.3 Pre-Funded Escrow

Consumers requiring repeated or high-frequency access may pre-fund a protocol-supported USDC escrow. The escrow permits paid intelligence delivery without an on-chain transaction for every individual delivery.

For Consumer a, spendable capacity is defined by:

```
EffectiveBalance_a = EscrowBalance_a - AccountedSpend_a - ReservedInFlightExposure_a
```

Additional paid delivery is permitted only while `EffectiveBalance_a` is sufficient for the requested operation and the applicable in-flight exposure rules remain satisfied. Accounted spend, reserved exposure, and subsequently acknowledged spend must be tracked so the same escrow balance cannot be concurrently committed beyond its funded capacity.

G.4 WebSocket Cumulative Acknowledgements

High-frequency WebSocket delivery uses authenticated cumulative acknowledgements. During a session, the Consumer periodically signs the latest accepted delivery sequence together with the cumulative USDC tab for that session.

Conceptually:

```
ACK_k = Sign(Session_ID || Sequence_k || CumulativeTab_k || Context)
```

A valid later acknowledgement supersedes earlier acknowledgements for the same session. Sequence and cumulative-spend state are monotonic: a later valid acknowledgement cannot reduce previously acknowledged spend or reuse an earlier delivery sequence as new spend.

The acknowledgement signature must bind the session and protocol context required to prevent reuse in another session, account, or accounting domain. Authenticated cumulative acknowledgements enter the same USDC accounting system as synchronous payment receipts.

G.5 Sliding In-Flight Exposure

Telegraph limits delivery that has been transmitted but not yet covered by the latest cumulative acknowledgement through a bounded sliding in-flight window.

Additional paid data may be transmitted only while:

$$\text{AcknowledgedSpend} + \text{MaximumInFlightExposure} \leq \text{AuthorizedSessionLimit}$$

When the permitted window is exhausted, further paid delivery pauses until additional valid acknowledgements or funding become available. If acknowledgements cease or the session becomes invalid, further paid delivery is suspended.

Protocol nodes replicate or otherwise coordinate the session-accounting state required to prevent multiple concurrent sessions from collectively exceeding the Consumer's funded capacity. The aggregate security requirement is:

$$\text{AggregateInFlightExposure} \leq \text{AvailableEscrow}$$

Normal WebSocket delivery does not rely on unsecured post-delivery debt.

G.6 Unified Paid-Delivery Accounting

Authenticated HTTP payment receipts, cumulative WebSocket acknowledgements, pre-funded session accounting, and other protocol-recognized paid-delivery records converge into the same USDC accounting ledger. For each fulfilled paid delivery, the protocol records sufficient information to attribute economic activity to the correct Consumer, Intent, Miner, price, protocol fee, and Miner-settlement amount.

The accounting state is finalized through the applicable consensus and settlement process. Real-time delivery may precede that final economic settlement. The exact Miner MACHINA entitlement is calculated only after the corresponding aggregated Ethereum settlement operation has completed and the actual acquired MACHINA amount is known, as defined in Part II H.

G.7 Managed and Web2 Access

Telegraph may expose managed-wallet, fiat-onboarding, or comparable integrations so developers can use the protocol without manually constructing blockchain transactions. A managed integration may:

- accept a supported funding source,
- acquire or deposit USDC,
- maintain a Consumer payment wallet or account,
- process x402 payment authorization automatically,
- maintain pre-funded Telegraph escrow,
- operate WebSocket sessions, and
- create direct on-chain jobs.

No specific commercial wallet, fiat processor, custody provider, or managed-access vendor is a consensus-critical Telegraph dependency. Every supported integration must ultimately produce USDC-denominated authorization and accounting state compatible with the protocol settlement rules.

G.8 On-Chain Callbacks

A smart contract may request an on-chain callback when a direct Telegraph job or another explicitly callback-enabled operation reaches its required completion condition. Callback execution cost is separate from the intelligence price, and the requesting application must provide sufficient callback-gas funding under the applicable callback mechanism.

A callback may receive compact result data including:

- Job_ID,
- Intent_ID,
- Miner_ID,
- outputHash, and
- bounded OnChainData.

Large Miner responses remain available through the authenticated off-chain result-availability mechanism. Failure of an optional callback does not invalidate an otherwise successfully finalized job unless callback success was explicitly defined as a job condition.

Callback execution must be bounded against excessive gas consumption, reentrancy, and repeated execution. If callback gas funding is insufficient at execution time, the callback remains pending or follows the applicable expiry rule. Validators are not required to finance third-party callback execution from their own capital.

G.9 Direct On-Chain Job Creation and Selection

Telegraph provides a direct on-chain job rail for autonomous agents and smart contracts requiring a one-shot, escrow-backed request to a specific Miner. This rail is distinct from ordinary HTTP routing and WebSocket settlement and may reach job-specific finality independently of the next ordinary epoch or cumulative settlement submission.

A funded direct job specifies, as applicable:

- Intent_ID,
- Miner_ID,
- the request or a request commitment,
- YAML-compatible input parameters,
- USDC budget,
- expiry,
- optional callback and callback-gas funding.

The required USDC is escrowed before execution. The selected Miner_ID must correspond to a Miner that remains actively registered, compatible with the Intent, available, within the applicable pricing requirements, and free of an active Routing_Revocation or other disqualifying status.

A valid explicit Miner selection is not overridden by ordinary probabilistic routing. This is the direct-job selection boundary defined in Part II F and does not alter the finalized Miner Ranking.

G.10 Direct Job Execution and Result Commitment

The direct-job execution pipeline is:

```
Funded Job -> Detect -> Selected Miner -> Miner Response -> Response Authentication -> outputHash
-> Validator Authorization -> On-Chain Completion
```

The Telegraph execution service detects the funded job and sends the committed request to the selected Miner. After the Miner response is obtained, Telegraph computes:

```
outputHash = H(Miner_Response)
```

The raw response remains available through the authenticated result-availability layer. Where the job uses the Leader-mediated authenticated HTTPS path, zkTLS or another protocol-approved authenticated-transcript mechanism binds the exact request and Miner response used for the job.

G.11 Job-Specific Validator Authorization and Finality

Validators verify the job-resolution package before authorizing completion. The package includes or commits to, as applicable:

- Job_ID,
- funded state,
- Intent_ID,
- selected Miner_ID,
- request commitment,
- Miner-response provenance,
- outputHash,
- payment state,
- protocol-fee calculation,
- Miner attribution, and
- callback commitment.

The job-resolution commitment conceptually binds:

```
JobResolutionRoot = H(Job_ID || Intent_ID || Miner_ID || Request_Commitment || outputHash ||
Payment_State || Protocol_Context)
```

A successful direct job requires the same Byzantine quorum rule used by normal protocol finalization:

$$\tau = \text{floor}(2 \times |V| / 3) + 1$$

At the genesis Active Validator Set of 64, $\tau = 43$. Once the required Validator authorization is available, an authorized participant submits the resolution to the job contract. The contract verifies funded job state, quorum authorization, result commitment, and replay protection before completion.

The same Job_ID cannot be completed twice, and an authorization produced for one job cannot be reused for another. Job-specific Validator authorization establishes that the Telegraph job was executed and recorded according to protocol rules. It does not establish that the semantic content of the Miner response is objectively true.

A direct job becomes final immediately after successful job-specific on-chain resolution. Internal job-resolution infrastructure may reuse Validator or scheduling infrastructure also used by ordinary protocol processing, but job completion does not depend on the next ordinary epoch boundary. Resulting accounting may subsequently be incorporated into the normal global protocol-state commitment.

G.12 ERC-8183 Agentic Commerce Mapping

Telegraph exposes the direct-job rail through an ERC-8183-compatible Agentic Commerce interface. The job lifecycle maps conceptually to:

```
Open -> Funded -> Submitted -> Completed
```

with the applicable rejection or expiry paths. The Agent creating and funding the job acts as the Client. The selected Telegraph Miner provides the requested intelligence through the Telegraph Provider integration. The Miner response is committed through outputHash as the deliverable commitment.

The protocol-controlled ERC-8183 Evaluator accepts completion only after the required Telegraph Validator quorum authorizes the corresponding JobResolutionRoot. This ERC-8183 Evaluator verifies Telegraph completion evidence, it does not independently judge the semantic correctness of the returned intelligence.

G.13 Direct-Job Economics and Aggregated Settlement

For a successful direct job, the genesis economic split is:

```
JobProtocolFee = 0.02 × JobPriceUSDC
JobMinerSettlementUSDC = 0.98 × JobPriceUSDC
```

The protocol fee is allocated to the Protocol Treasury, and the Miner-settlement amount is attributed to the selected Miner_ID. Direct-job completion does not trigger a separate per-job MACHINA purchase. Miner-attributable USDC enters the same aggregated Miner settlement ledger used by the other paid delivery rails.

The resulting USDC is later processed through the canonical Ethereum USDC-to-MACHINA settlement mechanism. The Miner's resulting cumulative entitlement is authenticated through Miner_Entitlement_Root and claimed through the cumulative entitlement model defined in Part II A and Part II H. Immediate job finality and later economic settlement therefore remain separate protocol events.

G.14 Unified Payment and Settlement Flow

Telegraph supports three primary paid Consumer rails:

```
Synchronous Request: Consumer Request -> x402 Authorization -> Miner -> Response -> USDC
Accounting
Streaming: Pre-Funded Escrow -> WebSocket Delivery -> Cumulative ACK -> Sliding In-Flight Window
-> USDC Accounting
```

Direct On-Chain Job: Funded Job -> Explicit Miner -> Response -> outputHash -> Validator Quorum -> Immediate On-Chain Completion -> Optional Callback -> USDC Accounting

All paid rails converge into the same economic split and canonical settlement process:

```
Gross Paid USDC -> 2% Protocol Treasury + 98% Miner Settlement -> Aggregated Ethereum
Settlement -> Actual MACHINA Acquired -> Cumulative Miner Entitlement -> Pull Claim
```

This architecture permits synchronous APIs, high-frequency machine streams, and autonomous smart-contract jobs to use different execution and finality cadences while preserving one canonical accounting and economic settlement system.

H. ECONOMIC PROTOCOL

This section defines Telegraph's normative economic accounting, emissions, demand-funded Miner compensation, Evaluator allocation, pricing, and cumulative entitlement rules. Consumer-denominated activity is accounted for in USDC. MACHINA is the protocol-native economic unit of intelligence and the native asset used for settlement, rewards, and security. Canonical economic settlement occurs on Ethereum. Miners are compensated from paid Consumer demand, Validators and Evaluators are compensated from the fixed MACHINA emission schedule. The Protocol Treasury receives its fixed emission allocation, protocol fees, and allocations explicitly redirected to it by protocol rules.

H.1 Economic Architecture

Telegraph separates demand-funded intelligence supply from emission-funded protocol infrastructure. The economic architecture has four primary flows:

- **Miner compensation:** paid Consumer USDC is attributed to the serving Miner, split between the Protocol Treasury and Miner settlement, aggregated on Ethereum, converted into MACHINA through the protocol settlement process, and converted into cumulative Miner entitlement. This flow translates fulfilled intelligence consumption into open-market demand for MACHINA.
- **Validator compensation:** Validators receive the fixed Validator share of MACHINA emissions subject to completed protocol work, Internal Audit Fee accounting, and finalized Validator penalties.
- **Evaluator compensation:** the Evaluator emission pool is allocated across Intents using Qualified Consumer Demand and is constrained by the fee-backed economic cap before payment to the eligible Canonical Evaluator.
- **Protocol Treasury:** the Treasury receives its fixed emission share, protocol fees, undistributed Evaluator emissions, and other allocations explicitly redirected to it under protocol rules.

Miner emissions are zero. Paid Consumer demand does not alter the fixed emission schedule. A period with no Consumer demand therefore does not eliminate Validator emissions, while Miner compensation requires paid Consumer activity.

H.2 MACHINA Supply and Emission Schedule

MACHINA is the native token of Telegraph and the protocol-native economic unit of intelligence. In this specification, that is an economic relationship rather than a fixed physical conversion: one MACHINA does not equal one request, Signal, inference, or unit of computation. Fulfilled intelligence is priced and accounted for in USDC, while Miner-attributable settlement value is used to acquire MACHINA through the canonical settlement mechanism. Its supply schedule is fixed at genesis and immutable.

- Maximum supply $S_{\max}$: 21,000,000 MACHINA.
- Genesis daily emission E_0 : 7,200 MACHINA per 24-hour emission period.
- Halving interval H_{halve} : four years, approximately 1,461 emission days.
- Pre-mine allocation: 0.
- Team / VC allocation: 0.

For emission day d , where $d = 0$ is the genesis emission day:

$$E(d) = E_0 \times 2^{(-\text{floor}(d / 1461))}$$

Emission ceases when cumulative minted supply reaches S_{max} . If the remaining mintable supply is less than $E(d)$, the protocol mints only the remaining amount for that emission period and then permanently halts emissions.

Each $E(d)$ is divided according to the immutable genesis emission split:

- Validators: 60 percent.
- Evaluators: 20 percent.
- Protocol Treasury: 20 percent.

Miners receive no MACHINA from protocol emissions. MACHINA used for Miner compensation is acquired from the open market using USDC generated by paid Consumer activity.

H.3 Protocol Treasury

The Protocol Treasury receives three primary protocol-native inflows:

1. 20 percent of daily MACHINA emissions,
2. the protocol fee charged on paid Consumer activity, and
3. protocol emissions or economic allocations explicitly redirected to the Treasury under the rules of the protocol.

At genesis, the protocol fee is:

$$f_{\text{protocol}} = 0.02$$

For GrossPaidUSDC recognized as finalized paid Consumer activity:

$$\text{ProtocolFee} = 0.02 \times \text{GrossPaidUSDC}$$

$$\text{MinerSettlementUSDC} = 0.98 \times \text{GrossPaidUSDC}$$

The same fee split applies across supported paid Consumer interfaces, including x402 requests, pre-funded escrow, WebSocket accounting, session-based payment, subscription-based payment, and other supported paid Consumer interfaces. A payment recognized as finalized protocol activity is economically final under the applicable settlement mechanism and is not subsequently removed from the accounting period through a refund mechanism.

Treasury expenditure may include protocol-authorized Ethereum transaction costs, Validator reimbursements, Autonomous Engine bootstrap activity, Miner evaluation-query reimbursement, protocol development, security audits, infrastructure, incident response, Miner and ecosystem grants, new-Intent bootstrap incentives, and other expenditures authorized under governance. Treasury-funded activity is excluded from Qualified Consumer Demand and does not modify the immutable 60/20/20 emission split.

H.4 Miner Revenue Accounting

For each fulfilled paid Consumer delivery, Telegraph records the finalized gross USDC value attributable to the Miner that served the request. For Miner m during settlement period N :

$$\text{GrossUSDC}_{(m,N)} = \text{finalized Consumer-paid USDC attributable to Miner } m$$

$$\text{ProtocolFee}_{(m,N)} = 0.02 \times \text{GrossUSDC}_{(m,N)}$$

$$\text{MinerUSDC}_{(m,N)} = 0.98 \times \text{GrossUSDC}_{(m,N)}$$

$$U_N = \sum \text{MinerUSDC}_{(m,N)}$$

Miner revenue accounting may be derived from settled x402 payments, cumulative WebSocket acknowledgements, pre-funded session accounting, authenticated HTTP receipts, direct-job accounting, and other protocol-recognized paid-delivery records. The same paid activity MUST NOT be counted more than once.

Protocol-generated ranking traffic, Miner evaluation traffic, Evaluator competition traffic, spot checks, grants, Treasury-funded traffic, bootstrap traffic, free requests, and other non-Consumer protocol activity do not contribute to $\text{GrossUSDC}_{(m,N)}$. Real-time Signal delivery does not wait for final economic settlement.

H.5 Aggregated Ethereum Miner Settlement

USDC allocated to Miner compensation is consolidated into the canonical Ethereum settlement domain. Telegraph uses time-distributed execution to acquire MACHINA using aggregated Miner-settlement USDC. Settlement execution may be divided across multiple transactions and times while preserving deterministic attribution of the resulting MACHINA.

Settlement execution parameters may include:

- settlement-period duration,
- execution cadence,
- minimum settlement size,
- maximum execution size,
- price-impact limits,
- liquidity requirements,
- scheduling variation,
- failed-execution handling, and
- rollover conditions.

For settlement period N , U_N is the finalized USDC allocated to Miner settlement. After the corresponding Ethereum execution completes, M_N is the exact amount of MACHINA acquired and allocated to that period. Final Miner entitlement is calculated from M_N , the protocol does not use an estimated future exchange rate to determine final entitlement.

Settlement is pipelined across accounting periods. During settlement period N , Consumer activity and Miner attribution for period N may continue to be recorded while the completed economic conversion associated with the preceding settlement period $N-1$ is processed. The entitlement for a completed period is finalized only after both its finalized Miner-settlement USDC and the exact MACHINA acquired for that period are known. If $U_{(N-1)} = 0$, no Miner entitlement allocation is calculated for that period and the division is not performed.

H.6 Miner Settlement Allocation and Cumulative Entitlement

Where $U_N > 0$, each Miner receives a proportional MACHINA entitlement based on its finalized Miner-settlement USDC attribution:

$$\text{PeriodMinerEntitlement}_{(m,N)} = \text{floor}(\text{MinerUSDC}_{(m,N)} \times M_N / U_N)$$

All settlement calculations use deterministic integer arithmetic in the applicable smallest token units. The aggregate allocation MUST satisfy:

$$\sum \text{PeriodMinerEntitlement}_m(m, N) \leq M_N$$

Any deterministic remainder created by integer division follows the protocol settlement remainder rule. If $U_N = 0$, no Miner settlement allocation is calculated for that period.

The resulting period entitlement is added to cumulative lifetime gross entitlement:

$$\text{LifetimeMinerEntitlement}_m(\text{new}) = \text{LifetimeMinerEntitlement}_m(\text{old}) + \text{PeriodMinerEntitlement}_m(m, N)$$

The complete cumulative Miner state is authenticated by `Miner_Entitlement_Root`. Claims use:

$$\text{Claimable}_m = \text{LifetimeMinerEntitlement}_m - \text{WithdrawnAmount}_m$$

Withdrawal changes `WithdrawnAmount_m`, not the cumulative lifetime gross entitlement recorded by the protocol. Finalized cumulative entitlement is monotonic and is not retroactively reduced.

H.7 Validator Emission Distribution

Validators receive 60 percent of daily MACHINA emissions. For emission day d :

$$\text{ValidatorEmissionPool}(d) = 0.60 \times E(d)$$

Validator emissions do not depend on Consumer demand. At genesis, 2 percent of the applicable Validator emission pool is allocated to the Internal Audit Fee for Validators that successfully perform assigned Candidate Evaluator verification.

The remaining Validator emission pool is allocated according to successfully completed protocol work. Reward-bearing assignments may include Miner-ranking verification, `Local_Score` computation, commit-reveal participation, BFT consensus participation, routing-state verification, protocol-state verification, canonical submission duties, and other protocol-defined Validator assignments.

Validators assigned equivalent work receive equivalent base reward treatment under the applicable reward schedule. A Validator that fails a required assignment does not earn the emission associated with that assignment. Category A, B, and C penalties are applied to unfinalized Validator emissions before cumulative entitlement is updated, forfeited emissions may be redistributed among eligible Validators according to finalized reward rules.

Validator reward accounting may also include delegated-stake accounting and Validator commission. Where MACHINA is delegated to an active Validator, the protocol tracks the proportional share of Validator rewards attributable to delegated stake, the Validator retains the configured commission and the remaining delegated reward becomes claimable by the Delegator through pull-based claiming.

Validator principal is never slashed, burned, confiscated, transferred, or reduced as a protocol penalty.

$$\begin{aligned} \text{LifetimeValidatorEntitlement}_v(\text{new}) &= \text{LifetimeValidatorEntitlement}_v(\text{old}) + \\ &\text{EarnedValidatorEmission}_v \end{aligned}$$

The cumulative Validator state is authenticated by `Validator_Entitlement_Root` and claimed through the cumulative entitlement mechanism defined in Part II A.

H.8 Evaluator Emission Pool and Qualified Consumer Demand

Evaluators receive 20 percent of daily MACHINA emissions:

$$\text{EvaluatorEmissionPool}(d) = 0.20 \times E(d)$$

Evaluator emission is allocated across active Intents using Qualified Consumer Demand. The allocation uses paid economic value, not request count or a subjective difficulty coefficient. Differences in service complexity, compute cost, scarcity, and Consumer value are expressed through Miner pricing and actual Consumer expenditure.

For Intent i during emission day d :

$$D_i(d) = \text{finalized gross USDC paid by Consumers for successfully fulfilled intelligence associated with Intent } i$$

Qualified Consumer Demand includes only protocol-recognized paid Consumer activity. It excludes:

- Miner-ranking evaluation traffic,
- Evaluator competition traffic,
- spot checks,
- protocol testing,
- Treasury-funded requests,
- Autonomous Engine bootstrap traffic funded by the protocol,
- Intent Bounties and grants,
- free requests, and
- other internally generated protocol activity.

Because the protocol fee is 2 percent at genesis:

$$F_i(d) = 0.02 \times D_i(d)$$

Demand and fee accounting use finalized protocol records and MUST NOT count the same economic activity more than once. Splitting the same total paid demand across multiple wallets, Miners, or Intents does not increase aggregate fee-backed reward capacity because the allocation basis is linear in finalized paid USDC.

H.9 Demand-Weighted Evaluator Allocation

$$D_{\text{total}}(d) = \sum D_i(d)$$

where the sum spans active Intents with Qualified Consumer Demand during emission day d .

Where $D_{\text{total}}(d) > 0$:

$$\text{RawEvaluatorAllocation}_i(d) = \text{floor}(\text{EvaluatorEmissionPool}(d) \times D_i(d) / D_{\text{total}}(d))$$

The allocation is linear in paid USDC. Request count is not part of the Evaluator allocation formula, and the Consumer-pricing Demand Multiplier is not applied again to Evaluator emission allocation. Subject only to deterministic integer rounding, dividing equivalent total paid demand among several Intents does not create additional aggregate raw emission weight.

H.10 Fee-Backed Evaluator Reward Cap

Demand weighting does not alone determine the final Evaluator reward. Each Intent allocation is constrained by the irreversible protocol fees generated by that Intent during the same allocation period.

Let β be the Evaluator reward-backing safety factor. At genesis:

```
beta = 0.80
RewardBackingUSDC_i(d) = beta × F_i(d)
RewardBackingUSDC_i(d) = 0.016 × D_i(d)
```

The USDC-equivalent value of the Evaluator reward for an Intent MUST NOT exceed its reward backing:

```
USDCValue(EvaluatorReward_i(d)) ≤ RewardBackingUSDC_i(d)
```

At genesis, the Evaluator reward value is therefore bounded to 1.6 percent of $D_i(d)$, while the corresponding paid Consumer activity has generated a 2 percent protocol fee. Reward backing applies only to the corresponding emission period. Unused reward backing does not accumulate, transfer between Intents, or support a later-period reward.

H.11 MACHINA Reference Price for Evaluator Caps

The fee-backed cap compares a USDC-denominated backing amount with a MACHINA-denominated emission. Telegraph therefore uses a manipulation-resistant reference price $P_{\text{ref}}(d)$, expressed as USDC per MACHINA.

```
FeeBackedCap_i(d) = floor(RewardBackingUSDC_i(d) / P_ref(d))
FinalEvaluatorAllocation_i(d) = min(RawEvaluatorAllocation_i(d), FeeBackedCap_i(d))
```

P_{ref} MUST NOT rely on a single current-block spot price and must be resistant to short-duration manipulation. The reference construction may incorporate sufficiently long Ethereum TWAP observations, realized protocol settlement execution data, minimum-liquidity requirements, multiple approved observations or venues where applicable, deviation limits, circuit breakers, and other protocol-defined oracle protections.

For the Evaluator cap, an understated MACHINA reference price is the dangerous direction because it permits a larger token allocation for the same USDC backing. The reference-price construction therefore applies conservative rules intended to prevent temporary downward manipulation from increasing Evaluator entitlement. If a valid reference price cannot be established under the applicable oracle rules, Evaluator emission distribution for the affected period does not proceed and the corresponding undistributed emission is transferred to the Protocol Treasury.

H.12 Canonical Evaluator Reward Recipient

For each active Intent i , $\text{FinalEvaluatorAllocation}_i(d)$ is assigned to the economic recipient registered for the finalized Canonical Evaluator E_i^* , provided the Evaluator is reward-eligible at the applicable emission snapshot.

Reward eligibility requires that the Evaluator is validly registered, remains the finalized Canonical Evaluator for the Intent, has passed the applicable protocol verification, has not been revoked, and satisfies the applicable availability and eligibility rules.

Candidate Evaluators receive no Canonical Evaluator emission merely for participating in competition.

If a new Evaluator becomes the finalized Canonical Evaluator, allocations finalized after the Canonical transition accrue to the new Canonical Evaluator. Evaluator registration principal is not slashed, burned, confiscated, or reduced as a protocol penalty. Penalties may affect unfinalized emission, Canonical status, eligibility, registration status, or future participation.

H.13 Missing Canonical Evaluator and Residual Emissions

If an Intent has a positive `FinalEvaluatorAllocation_i(d)` but no eligible Canonical Evaluator at the applicable emission snapshot, that allocation is transferred to the Protocol Treasury. It is not redistributed among other Evaluators or Intents.

Any difference between `RawEvaluatorAllocation_i(d)` and `FinalEvaluatorAllocation_i(d)` caused by the fee-backed cap is transferred to the Protocol Treasury. Deterministic rounding remainders in the Evaluator emission pool are also transferred to the Treasury. If $D_{total}(d) = 0$, the entire Evaluator emission pool for that emission day is transferred to the Treasury. Unused Evaluator emission is not carried forward as future reward capacity.

H.14 Cumulative Evaluator Entitlement

For Evaluator x , the protocol sums all finalized Intent allocations earned by that Evaluator during emission day d as `EarnedEvaluatorEmission_x(d)`. Cumulative lifetime entitlement is updated as:

$$\text{LifetimeEvaluatorEntitlement}_x(\text{new}) = \text{LifetimeEvaluatorEntitlement}_x(\text{old}) + \text{EarnedEvaluatorEmission}_x(d)$$

The complete cumulative Evaluator state is authenticated by `Evaluator_Entitlement_Root`. Claims use:

$$\text{Claimable}_x = \text{LifetimeEvaluatorEntitlement}_x - \text{WithdrawnAmount}_x$$

Previously finalized cumulative entitlement is not reduced by a later penalty, loss of Canonical status, or Canonical replacement. Any applicable penalty must affect entitlement before the corresponding cumulative root is finalized.

H.15 Signal Pricing and Demand Multiplier

Each Miner registers a minimum USDC price for each supported Intent. For Miner m and Intent i , `min_price_usdc(m,i)` is the registered minimum price. At genesis, the protocol-wide minimum permitted price floor is 0.01 USDC. A Miner may register a higher minimum price.

The paid request price is:

$$\text{SignalPrice}_i(m,i) = \text{min_price_usdc}_i(m,i) \times \text{DemandMultiplier}_i$$

The Demand Multiplier is derived from authenticated paid protocol activity for the applicable Intent over the protocol pricing window. At genesis:

Paid request volume in pricing window	Demand Multiplier
0-999	1.0x

Paid request volume in pricing window	Demand Multiplier
1,000-9,999	1.5x
10,000-99,999	2.5x
100,000-999,999	5.0x
1,000,000+	10.0x

Only protocol-recognized paid Consumer requests count toward the Demand Multiplier. Protocol evaluation traffic, free traffic, Treasury-funded requests, ranking activity, Intent Bounties, bootstrap activity, and other non-Consumer protocol traffic do not contribute to the request count. Pricing uses finalized or authenticated protocol accounting state from the applicable preceding pricing window so protocol participants derive the same pricing input.

A Miner minimum price may be changed only through an authorized registration update. The protocol minimum price floor and Demand Multiplier schedule are governable economic parameters. The Demand Multiplier affects the Consumer price and is not separately applied to the Evaluator emission-allocation formula.

H.16 Protocol Fee Application

For every paid Consumer transaction at the genesis fee rate:

```
ProtocolFee = 0.02 × SignalPrice
MinerSettlementUSDC = 0.98 × SignalPrice
```

When multiple deliveries are aggregated into a session, WebSocket tab, subscription settlement, or another supported payment mechanism:

```
ProtocolFee = 0.02 × GrossFinalizedConsumerUSDC
MinerSettlementUSDC = 0.98 × GrossFinalizedConsumerUSDC
```

The protocol fee is directed to the Protocol Treasury. The Miner-settlement portion enters the aggregated Ethereum settlement mechanism defined in H.5-H.6. The same economic split also applies to successful direct on-chain jobs as defined in Part II G.

H.17 Economic Security Invariants

The economic protocol preserves the following invariants:

- **Miner Demand Invariant:** Miners receive no protocol emissions. Miner MACHINA entitlement originates from open-market MACHINA acquired using USDC generated by paid Consumer activity.
- **Validator Emission Invariant:** Validators receive the fixed 60 percent protocol emission allocation subject to finalized participation and penalty accounting. Validator reward penalties affect emissions and eligibility, not Validator principal.
- **Evaluator Emission Invariant:** under the genesis reward-backing parameter, the maximum USDC-equivalent Evaluator reward attributable to Consumer demand for an Intent is bounded by $0.80 \times \text{ProtocolFee}_i$.

- Linear Demand Invariant: equivalent total paid demand has equivalent aggregate raw Evaluator emission weight regardless of how that demand is divided among Consumer wallets, Miners, or Intents, subject only to deterministic integer rounding.
- Single-Period Backing Invariant: fee backing generated during one allocation period may support Evaluator reward only for that period and is not reused later.
- No Subjective Difficulty Invariant: Telegraph assigns no economic difficulty multiplier to an Intent, complexity and economic value are expressed through market pricing and paid Consumer expenditure.
- Treasury Residual Invariant: Evaluator emission that cannot be validly distributed is transferred to the Protocol Treasury and is not redistributed in a way that increases another Intent's reward entitlement.
- Cumulative Entitlement Invariant: once finalized, cumulative Miner, Validator, and Evaluator gross entitlement is monotonic and is not retroactively reduced.

H.18 Economic Flow

Paid Consumer intelligence follows:

Consumer USDC -> finalized protocol accounting -> 2% Protocol Treasury fee -> 98% Miner-settlement USDC -> Ethereum settlement -> time-distributed MACHINA acquisition -> actual acquired MACHINA known -> Miner period allocation -> cumulative Miner_Entitlement_Root -> Miner pull claim

Validator compensation follows:

Protocol emission -> 60% Validator pool -> participation / Internal Audit Fee / penalty accounting -> cumulative Validator_Entitlement_Root -> Validator pull claim

Evaluator compensation follows:

Protocol emission -> 20% Evaluator pool -> Qualified Consumer Demand share -> fee-backed economic cap -> eligible Canonical Evaluator -> cumulative Evaluator_Entitlement_Root -> Evaluator pull claim

The Protocol Treasury receives:

20% fixed Treasury emission + protocol fees + undistributed Evaluator emissions + other explicitly redirected protocol allocations

These flows preserve the separation between demand-funded intelligence supply, emission-funded protocol infrastructure, and protocol-controlled Treasury resources. Ranking production, intelligence delivery, and economic settlement remain distinct processes connected through finalized protocol state and authenticated accounting. The Miner flow is the direct economic link between intelligence consumption and MACHINA: greater finalized Miner-attributable paid intelligence value produces greater USDC settlement value available for time-distributed open-market MACHINA acquisition, subject to the settlement rules and market-execution constraints above.

H.19 Entitlement Transition Verification

A new cumulative entitlement root may replace the previous finalized root only when the transition is consistent with the applicable economic inputs, prior cumulative state, protocol sequence, and deterministic arithmetic rules.

For Miner entitlements, Validators verify the previous cumulative entitlement state, finalized USDC attribution for the completed settlement period, the exact MACHINA acquired for that period, reconciliation of Miner allocations to the actual MACHINA allocated to Miner settlement, deterministic integer precision and remainder handling, the equality of each new cumulative entitlement to its prior cumulative entitlement plus the newly earned period amount, and the requirement that no finalized Miner lifetime entitlement decreases.

For Validator emissions, Validators verify that the applicable emission pool matches the protocol emission schedule, completed and missed assignments are accounted for, applicable emission forfeitures and redistribution are correctly applied, and each cumulative Validator entitlement transition is monotonic and arithmetically correct.

For Evaluator emissions, Validators verify that the Evaluator emission pool matches the protocol emission schedule, Canonical Evaluator and reward-state inputs are correct, Treasury redirections and finalized forfeitures are correctly applied, and each cumulative Evaluator entitlement transition is monotonic and arithmetically correct.

The resulting Miner_Entitlement_Root, Validator_Entitlement_Root, and Evaluator_Entitlement_Root become authenticated components of the finalized Protocol_State_Root.

I. AUTONOMOUS ENGINE AND BOOTSTRAP PROTOCOL

This section defines the normative bootstrap state, launch classifications, Foundation participants, Intent Bounties, public bootstrap visibility, and the transition from genesis operation to open permissionless operation. The Autonomous Engine execution and routing mechanics are defined in Part II F, this section specifies how that machinery may be initialized and operated during bootstrap without creating permanent protocol privilege.

I.1 Bootstrap Model

Telegraph MAY launch with the minimum infrastructure required to operate complete end-to-end protocol flows before broad external participation exists. Bootstrap infrastructure MAY include Intents, Evaluators, Miners, Collectors, Validators, Autonomous Engine nodes, bootstrap applications, and public interfaces.

The Telegraph Operating Company MAY operate bootstrap infrastructure, including Foundation Miners, Collectors, Autonomous Engine nodes, Validators, and bootstrap applications. Operation by the Company does not create a permanent protocol role, ranking right, routing share, Canonical Evaluator entitlement, or governance privilege.

Bootstrap or Treasury-funded activity is protocol-funded activity. It MUST remain separately classified from independent paid Consumer activity wherever demand-dependent protocol mechanisms require Qualified Consumer Demand or equivalent paid-demand accounting.

A one-time bootstrap contribution creates no MACHINA allocation, repayment right, ranking privilege, routing privilege, Canonical Evaluator privilege, or governance privilege unless such value is separately earned through the ordinary protocol mechanisms.

I.2 Genesis Intent Library and Initialization

At genesis, ordinary MACHINA-denominated registration requirements may be temporarily impossible because circulating MACHINA is initially unavailable. Telegraph MAY therefore encode a limited Genesis Intent Library and the bootstrap participants required to make those Intents operational directly into genesis state.

Genesis state MAY include, as applicable:

- Intent configuration and versioned interface state,
- initial Canonical Evaluators and their committed artifacts,
- committed evaluation material,
- Foundation Miners,
- Collectors and Autonomous Engine bootstrap configuration,
- bootstrap Validators and Validator-set state,
- the initial `Foundation_Model_Version` where applicable,
- initial training material required by the bootstrap intelligence architecture,
- initial Evaluator artifacts, including Evaluator-generation or training artifacts where applicable,
- evaluation and training configuration required to initialize the bootstrap Intent, and
- other protocol state strictly required for genesis operation.

This exemption applies only to genesis initialization. Subsequent registrations, replacements, updates, and newly created Intents MUST follow the normal permissionless protocol rules defined by the applicable protocol sections.

I.3 Foundation Miners

Foundation Miners are bootstrap Miners used to make one or more Genesis Intents operational before sufficient external supply exists. Foundation Miners MUST be publicly identifiable as bootstrap participants.

Before the first finalized Miner Ranking exists for an Intent, Foundation Miners participate under the same pre-ranking routing rule that applies to all eligible registered Miners for that Intent. After ranking state exists, Foundation Miners MUST compete under the same ranking, routing, pricing, eligibility, spot-check, and Routing_Revocation rules as equivalent external Miners.

Foundation status creates no permanent routing preference, reserved traffic share, protected ranking position, or Company-controlled allocation. A higher-performing external Miner may gain greater routing opportunity through the ordinary Miner-ranking process without Company approval.

I.4 Bootstrap Evaluators, Evaluation State, and Foundation Model

Genesis initialization MAY include the evaluation state required to make a bootstrap Intent measurable before permissionless Evaluator participation has developed. This state MAY include committed evaluation material, initial Evaluator artifacts, Evaluator-generation or training artifacts, evaluation configuration, and the initial Foundation Model version and training material where the applicable Evaluator architecture uses the shared backbone.

Bootstrap Evaluator and training artifacts initialize protocol operation only. They create no permanent control over an Intent. Subsequent Evaluator registration, evaluation contributions, candidate competition, hidden verification, Canonical replacement, Foundation Model retraining, Foundation Model version changes, and Canonical revalidation MUST follow Parts II C and the applicable consensus rules.

A qualified external Evaluator may replace a bootstrap Evaluator through the ordinary competition and hidden-verification process. A Foundation Model update creates a new `Foundation_Model_Version` and triggers the revalidation requirements defined in Part II C before that new backbone becomes authoritative for affected Intents.

I.5 Bootstrap Validators and Finality

Telegraph MAY begin operation with a Validator population below the participation required for normal Byzantine fault-tolerant finality. Such operation is explicitly bootstrap operation and does not receive the same decentralization or Byzantine-finality guarantees as normal quorum-backed protocol operation.

Bootstrap operation MUST NOT redefine or weaken the protocol BFT threshold. Once sufficient eligible Validators are active, normal finality uses the immutable quorum rule:

$$\tau = \text{floor}(2 \times |V| / 3) + 1$$

At $|V| = 64$, $\tau = 43$. Transitions requiring normal BFT finality that do not obtain the required quorum remain unfinalized. The transition from bootstrap operation to normal

quorum-backed finality occurs by satisfying the ordinary Validator participation and protocol-state requirements, not by lowering the threshold.

I.6 Bootstrap Collectors, Autonomous Engine, and Public Interfaces

Bootstrap Collectors, Autonomous Engine nodes, and applications MAY be operated to demonstrate request-driven and continuous intelligence flows before external operators exist. Their authority remains bounded by Part II F.

Collectors remain input integrations and MUST NOT select Miners, alter Miner Rankings, select Canonical Evaluators, modify Intent registration state, or change protocol economic state. The Autonomous Engine consumes finalized ranking and eligibility state and MUST NOT create Miner Rankings, select Canonical Evaluators, or perform consensus.

External Collectors, applications, Consumers, and Autonomous Engine operators MAY enter without a protocol migration. Bootstrap operation therefore initializes availability of the system, it does not establish an exclusive application or routing layer.

I.7 Protocol-Funded Activity Classification

Protocol-funded activity MUST be distinguishable from paid Consumer activity in protocol accounting and public demand reporting. This includes, as applicable, bootstrap activity, Treasury-funded requests, Miner-ranking evaluation traffic, Evaluator competition traffic, spot checks, protocol testing, Intent Bounties, grants, free traffic, and other internally generated protocol activity.

Such activity does not count as Qualified Consumer Demand, does not receive demand-weighted Evaluator emission treatment, and does not contribute to the Consumer-pricing Demand Multiplier where the pricing rules exclude non-Consumer protocol traffic. Where protocol-funded Miner calls create reimbursable execution costs, those payments are classified as protocol evaluation or bootstrap expenditure, not Consumer demand.

I.8 Intent Bounties

A newly created or low-activity Intent may lack sufficient evaluation activity to establish or refresh useful Miner Ranking state. Telegraph MAY support USDC-denominated Intent Bounties or another explicitly funded bootstrap mechanism for this purpose.

The conceptual bounty flow is:

**Intent Bounty -> Protocol Evaluation -> Miner Responses -> Canonical Evaluator -> Validator
Verification -> Miner Ranking**

Intent Bounties fund protocol evaluation activity. They are not Consumer demand and MUST NOT increase:

- Demand Multiplier request volume,
- Qualified Consumer Demand,
- Evaluator demand-weighted emissions, or
- organic-demand statistics or equivalent public paid-demand metrics.

Where bounty-funded Miner calls create reimbursable execution costs, those amounts are protocol evaluation expenditure and remain economically distinct from paid Consumer usage.

I.9 Signal Dashboard and Bootstrap Visibility

Telegraph MAY expose a Signal Dashboard or equivalent public interfaces showing the state and activity required to understand active Intents, supply, ranking, routing, and demand. The public interface MAY expose:

- current Miner Ranking and leading Miner,
- active Miner count,
- current Canonical Evaluator,
- request volume and paid Consumer volume,
- settled USDC volume,
- routing status,
- Foundation Miner status,
- historical ranking information, and
- Miner revenue metrics.

Public reporting MUST preserve the protocol classification boundary between paid Consumer activity and protocol-generated activity. Paid Consumer demand is displayed separately from protocol evaluation traffic, bootstrap traffic, Treasury-funded activity, Intent Bounties, and free traffic where those metrics are exposed.

I.10 Automatic Transition to Open Operation

Telegraph does not require a centralized manual cutover from bootstrap operation to open operation. Bootstrap participants operate under the same ranking, routing, payment, verification, eligibility, and replacement rules that apply to equivalent external participants once the relevant normal protocol mechanism is active.

The transition occurs through ordinary competitive state changes:

- external Miners can displace Foundation Miners in routing opportunity by outperforming them in Miner Rankings,
- external Evaluators can replace bootstrap Evaluators by winning candidate competition and passing hidden verification,
- external Collectors, applications, and Consumers can use the same public protocol interfaces,
- paid Consumer activity becomes the demand measured by the economic system while bootstrap and Treasury-funded activity remains separately classified, and
- normal quorum-backed Validator finality applies once the required Validator participation exists.

No permanent Company routing share, Foundation Miner allocation, Agent Zero reduction schedule, bootstrap Evaluator incumbency right, or manual network cutover is required. Bootstrap state initializes the network, open protocol competition determines subsequent state.

I.11 Relationship to the Autonomous Engine and Ranking Protocol

The Autonomous Engine and bootstrap protocol do not create an alternative ranking path. Ranking production continues to follow the Evaluator, Miner-ranking, Validator, and BFT mechanisms defined in Parts II C-E. Operational delivery continues to follow Part II F, and paid activity settles through Parts II G-H.

Conceptually, the bootstrap-to-open system preserves the same protocol chain:

Evaluation -> Finalized Miner Ranking -> Ranked Routing -> Signal Delivery -> Authenticated Accounting -> Economic Settlement

Bootstrap participants may occupy initial positions in this chain only where genesis initialization requires them. Once ordinary permissionless participation is available, the protocol mechanisms themselves determine replacement, ranking, routing opportunity, economic entitlement, and finality.

J. GOVERNANCE PROTOCOL

This section defines the normative authority, activation threshold, governable parameter classes, timelocks, proposal lifecycle, emergency veto, immutable invariants, and execution boundaries of Telegraph governance. Governance may modify bounded protocol parameters only within the immutable rules defined by the protocol.

J.1 Governance Authority and Activation

Governance is exercised by the Active Validator Set. Each active Validator has one governance vote, independent of stake weight. Delegated or self-staked MACHINA affects competition for Active Validator Set membership, but it does not create additional governance votes for an already active Validator.

On-chain governance becomes active only when the Active Validator Set contains at least 43 Validators. Before that activation threshold is reached, governable parameters remain fixed at their genesis values.

After activation, a proposal passes only when it receives affirmative votes from at least the protocol governance threshold:

$$\text{Governance_Threshold} = \text{floor}(2 \times |V| / 3) + 1$$

At the genesis Validator Cap of 64, Governance_Threshold = 43. If the Validator Cap or applicable Active Validator Set size changes, the governance threshold is recomputed from $|V|$. A fixed 43-vote rule does not persist independently of the active-set size.

J.2 Governable Parameter Classes

Governance may modify only parameters explicitly designated as governable. Changes apply prospectively after the applicable timelock and do not modify finalized historical ranking, accounting, settlement, consensus, or cumulative entitlement state.

J.2.1 Validator and Consensus Parameters

- Validator Cap N ,
- Validator assignment and liveness thresholds,
- Category A and Category C penalty thresholds,
- Validator unbonding period,
- protocol epoch or accounting-period duration where applicable,
- commit and reveal phase durations,
- Evaluator-verification subset size, and
- Internal Audit Fee percentage.

The Byzantine finalization rule is not replaceable by an arbitrary governance quorum. Finalization continues to require:

$$\tau = \text{floor}(2 \times |V| / 3) + 1$$

J.2.2 Registration and Participation Parameters

- Miner registration-bond requirement,
- Evaluator registration-bond requirement,
- Intent registration or activation bond where applicable,

- Miner deregistration unbonding period,
- Evaluator unbonding period,
- maximum active Miner count per Intent where capacity limits are enabled, and
- deterministic execution resource limits.

Registration principal remains protected by the no-principal-slashing invariant. A governed change to a bond or unbonding parameter does not authorize confiscation, burning, transfer, or reduction of existing principal as a protocol penalty.

J.2.3 Evaluator and Ranking Parameters

- minimum Evaluator competition sample volume,
- statistical confidence threshold,
- minimum Canonical replacement margin,
- cross-batch stability requirements,
- number or size of hidden evaluation batches,
- evaluation-material admission limits,
- mature evaluation-contribution influence cap,
- bootstrap contribution-share parameters,
- minimum contributor-diversity requirements,
- evaluation execution tolerance where bounded numeric variation is permitted,
- Evaluator competition cadence, and
- Miner-ranking cadence.

Changing these parameters does not permit governance to directly select a Canonical Evaluator or alter a finalized Miner Ranking. Canonical status and Miner Ranking continue to result from the protocol-defined competition, verification, aggregation, and finalization mechanisms.

J.2.4 Routing Parameters

- ranking-based routing distribution weights,
- qualifying routing range or equivalent routing-policy parameters,
- spot-check frequency k_{spot} ,
- routing-revocation degradation threshold,
- routing-recovery parameters, and
- operational failover parameters where defined by protocol state.

Governance cannot assign routing preference to a specific Miner outside the protocol-defined ranking and eligibility rules.

J.2.5 Economic Parameters

- protocol fee percentage,
- protocol minimum Signal price floor,
- Demand Multiplier thresholds and multipliers,
- Evaluator fee-backing coefficient β ,
- applicable settlement cadence parameters, and
- oracle or reference-price parameters used by mechanisms such as P_{ref} , subject to the protocol security requirements.

Economic parameter changes are prospective. They do not rewrite finalized economic periods or reduce previously finalized cumulative entitlements.

J.3 Genesis Governance Timelocks

Governance changes become executable only after the parameter-specific timelock applicable to the passed proposal. At genesis, the timelocks are:

Parameter class	Genesis timelock
Validator Cap	30 days
Validator unbonding	30 days
Miner / Evaluator unbonding	30 days
Ranking or Evaluator competition cadence	30 days
Evaluator statistical qualification parameters	14 days
Evaluation contribution and sampling parameters	14 days
Penalty and liveness thresholds	14 days
Registration bonds	14 days
Protocol fee	14 days
Minimum Signal price	14 days
Demand Multiplier schedule	14 days
Routing distribution	14 days
Spot-check parameters	14 days
Routing-revocation threshold	14 days
Internal Audit Fee	14 days
Other ordinary operational parameters	14 days

A proposal affecting multiple parameters uses the longest timelock applicable to any affected parameter. The timelock begins only after the proposal has passed the voting stage.

J.4 Proposal Lifecycle

Governance follows four ordered stages: submission, discussion, voting, and execution.

J.4.1 Submission

Any active Validator may submit a governance proposal. The proposal must target parameters explicitly designated as governable and must include the proposer identity, target parameter or parameters, current value, proposed value, activation conditions, applicable timelock, and a rationale commitment or content-addressed reference.

J.4.2 Discussion

Each submitted proposal enters a 7-day public discussion period. Governance votes are not counted during this period.

J.4.3 Voting

After discussion, a 7-day voting period begins. Only Validators belonging to the Active Validator Set applicable to the proposal may vote. Each active Validator has one vote. Passage requires at least `Governance_Threshold` affirmative votes. A proposal that fails to reach the threshold expires without execution.

J.4.4 Execution

A successful proposal enters its applicable timelock. After the timelock expires, the approved change becomes executable and takes effect at the next protocol boundary applicable to that parameter. A governance action cannot be applied retroactively to finalized protocol state.

J.5 Emergency Veto

A pending governance proposal may be cancelled before execution through an emergency supermajority veto. The threshold is:

$$\text{Emergency_Veto_Threshold} = \text{ceil}(5 \times |V| / 6)$$

At an Active Validator Set of 64, `Emergency_Veto_Threshold` = 54. A successful veto cancels the pending proposal. The same materially equivalent proposal cannot be resubmitted for 90 days.

Emergency veto authority only prevents a pending governance change from becoming active. It does not authorize direct modification of protocol state and does not bypass the normal governance proposal process.

J.6 Immutable Protocol Invariants

The following protocol properties are outside governance authority:

- maximum MACHINA supply of 21,000,000,
- the genesis emission schedule and halving rule,
- the 60% Validator / 20% Evaluator / 20% Protocol Treasury emission allocation,
- zero pre-mine,
- zero team or VC allocation,
- zero Miner protocol emissions,
- the rule that Validator, Miner, and Evaluator principal is never slashed, burned, confiscated, or transferred as a protocol penalty,
- the rule that finalized cumulative economic entitlements are not retroactively clawed back,
- the Byzantine finalization threshold rule $\tau = \text{floor}(2 \times |V| / 3) + 1$,
- the requirement that Canonical Evaluator status results from protocol-defined competition and verification,
- the requirement that Miner Rankings result from protocol-defined evaluation, aggregation, and finalization,
- the prohibition on governance directly appointing a Miner, Evaluator, or Validator to a competitive protocol position, and
- the prohibition on governance retroactively rewriting finalized ranking, accounting, settlement, or consensus state.

A governance proposal that attempts to violate an immutable invariant is invalid and cannot enter executable state.

J.7 Governance Boundary

Governance controls bounded parameters and prospective configuration. It does not substitute for protocol competition, deterministic execution, ranking, routing eligibility, or BFT finalization. Governance cannot directly select a Canonical Evaluator, change a finalized Miner Ranking, grant a specific Miner routing preference outside the ranking and eligibility rules, appoint a participant to a competitive protocol position, or weaken the immutable Byzantine quorum formula.

Where a governed parameter affects a future ranking round, Evaluator competition, routing period, settlement period, or other versioned operation, the new value applies only after the proposal has passed, its timelock has expired, and the applicable activation boundary has been reached. Previously finalized state remains canonical and auditable under the parameter values that governed it at the time of finalization.

K. SECURITY AND FAILURE HANDLING

This section defines the normative security assumptions, integrity controls, confidentiality mechanisms, replay protections, and failure behavior that apply across Telegraph. It consolidates the security consequences of the registration, evaluation, ranking, consensus, routing, payment, and economic protocols defined in Parts II A-J. Telegraph assumes that Miners, Evaluators, Validators, Consumers, Collectors, and evaluation-material contributors may behave adversarially or may fail. Security therefore depends on cryptographic commitments, authenticated provenance, deterministic execution, bounded influence, independent verification, Byzantine quorum, eligibility rules, and economic accounting boundaries rather than on trusted participant identity.

K.1 Validator and Consensus Security

The Active Validator Set is the top N eligible Validator Profiles by staked MACHINA under Part II E. At genesis, N = 64. A protocol transition that requires Byzantine finality MUST obtain at least:

$$\tau = \text{floor}(2 \times |V| / 3) + 1$$

At the genesis Active Validator Set of 64, $\tau = 43$. The quorum MUST NOT be reduced during a network partition, Validator outage, or period of reduced availability. If the required quorum is unavailable, the proposed transition remains unfinalized and the previous finalized state remains authoritative. Normal finalization resumes from the last finalized state when sufficient Validator participation returns.

A quorum authorization proves that the required active Validators authorized the exact committed protocol state and context. It does not prove that an underlying machine-generated response is objectively true. Validators verify protocol execution, commitments, provenance, deterministic calculations, and state transitions, semantic Miner quality remains defined by the current Canonical Evaluator for the applicable Intent.

Validator principal is never slashed, burned, confiscated, transferred, or reduced as a protocol penalty. Category A, B, and C consequences operate through unfinalized emissions, reward eligibility, Validator status, Active Validator Set membership, and permanent revocation where objectively provable Category B fraud is established. The exact penalty classifications and thresholds are defined in Part II E.

Telegraph distinguishes objective protocol fraud from ordinary model error, evaluation disagreement, statistical uncertainty, and deterministic execution error. Subjective disagreement is not sufficient evidence for Category B fraud. A stale-state, configuration, synchronization, or other non-fraudulent deterministic mismatch is handled under the applicable incorrect-execution rules.

K.2 Evaluator Competition and Foundation Model Security

Evaluator security depends on a two-stage competition process. Candidate Evaluators are self-hosted during candidate ranking and receive standardized public, mock, historical, synthetic, or otherwise non-sensitive workloads. Candidate-ranking results determine verification order only, they do not grant Canonical status.

The highest-ranked eligible candidate becomes the presumptive Canonical Evaluator and enters hidden verification. The protocol retrieves the committed executable, verifies its artifact hash, version, execution metadata, and applicable state bindings, and causes the assigned Validator verification subset to execute it independently against hidden material. A candidate that fails the applicable qualification requirements cannot become Canonical, verification advances to the next-ranked eligible candidate. If no challenger passes, the previously verified Canonical Evaluator remains active.

Where an Evaluator depends on the shared Foundation Model, the Foundation Model does not select the Evaluator that becomes Canonical. Canonical selection remains external to the backbone and is determined by Evaluator competition and protocol verification. This prevents the common computational backbone from also becoming the authority that selects among Evaluators that depend on it.

A Foundation Model version change is security-relevant because it may change the behavior of an otherwise unchanged Evaluator. Activation of a new `Foundation_Model_Version` therefore triggers revalidation of affected Canonical Evaluators before the new backbone becomes authoritative. Canonical state binds the Foundation Model version, Evaluator version, `Evaluation_Pool_Version`, `Evaluation_Engine_Version`, evaluation specification, and other execution context required to reproduce the applicable Canonical decision.

K.3 Evaluation Material and Bounded Contribution Influence

Evaluator developers and other eligible participants are responsible for producing or sourcing the task-specific material used by their evaluation methods. Telegraph does not centrally construct a separate ground-truth or corpus pipeline for each Intent. Eligible evaluation material **MUST** be committed before it can participate in hidden Canonical verification. A contribution commitment may be represented as:

$$\text{Evaluation_Commitment} = H(\text{Intent_ID} \parallel \text{Material_Hash} \parallel \text{Metadata_Hash} \parallel \text{Version})$$

The protocol applies deterministic admission controls including schema validation, malformed-input rejection, size limits, duplicate detection, contribution-frequency limits, contribution versioning, and applicable minimum contribution requirements.

For community-created Intents, no individual contributor may dominate the effective hidden sample. Let C_i be the number of eligible independent evaluation contributors for Intent i . The maximum effective sampled contribution share is:

$$\text{MaxShare}_i = \max(0.02, 1 / C_i)$$

The rule permits bootstrap operation with a small contributor set and converges to a 2 percent mature-pool cap once at least 50 eligible independent contribution domains participate. The cap applies to effective sampled influence, not raw submitted row count. Contribution controls may account for economically linked or otherwise attributable domains so that simple address splitting does not automatically multiply influence. Bootstrap evaluation material creates no permanent preference once ordinary contribution participation is available.

K.4 Hidden Sampling and Candidate Verification

Hidden Canonical-verification material is selected only after the applicable candidate executable and evaluation-material state have been committed. Conceptually:

```
Hidden_Verification_Batch = Sample(Committed_Evaluation_State, Protocol_Randomness)
```

The sample **MUST** be unpredictable before the applicable finalized randomness is fixed, deterministic after selection, reproducible by the assigned verification participants, and subject to the contribution-share limits defined in Part II C.

The Validator verification subset is selected deterministically from the finalized protocol context:

```
Verification_Seed = H(Protocol_Randomness || Intent_ID || Evaluator_Round_ID)
```

At genesis:

```
VerificationSubsetSize = ceil(0.10 × |V|) = 7 when |V| = 64
```

Assigned Validators retrieve the committed executable, verify its hash and version, derive the hidden sample, execute the candidate in the deterministic verification environment, and apply the committed Canonical qualification rules. Hidden material **MUST NOT** be sent to candidate-operated infrastructure before or during local verification.

A candidate self-report that cannot be reproduced from the committed executable and applicable inputs fails verification. Failure removes that candidate from the current promotion path and advances verification to the next eligible candidate. Evaluator registration principal remains protected by the no-principal-slashing invariant.

K.5 Miner Response Provenance and Ranking Integrity

Ranking integrity requires the eligible Miner set, response-collection window, evaluation request or batch, valid Miner responses, provenance evidence, Canonical Evaluator state, and deterministic execution context to be committed before final ranking state is accepted.

Where a Miner response does not independently provide transferable authenticated evidence, the ranking infrastructure uses zkTLS or another protocol-approved authenticated transcript mechanism. The evidence binds, as applicable:

- the authenticated Miner endpoint,
- Miner_ID,
- Intent_ID,
- Round_ID,
- the exact evaluation request,
- the exact Miner response, and
- the applicable protocol-domain context.

Authenticated provenance proves properties of the request-response interaction used by the protocol. It does not establish that the Miner response is objectively correct. Invalid, substituted, context-mismatched, or replayed evidence cannot produce a valid Local_Score.

A timely valid response required by the ranking round cannot be validly omitted from the committed response set. Validators independently execute the current Canonical Evaluator against the same committed evaluation basis. Local Scores use commit-reveal before

plaintext results are visible to other Validators, and Canonical Scores use the finalized ranking-round stake snapshot and the protocol-defined stake-weighted median. Final Miner ordering is deterministic, including the ascending Miner_ID tie-break for equal Canonical Scores.

K.6 Ranking-Round Leader Integrity

Ranking-round coordination follows strict round-robin rotation over the ordered Active Validator Set:

$$\text{LeaderIndex} = \text{Round_ID} \bmod |V|$$

The scheduled Leader coordinates the protocol-defined evaluation request, response collection, response-provenance evidence, and construction of the committed evaluation package. The Leader does not select the Canonical Evaluator, choose Miner scores, or modify deterministic ranking output.

If the scheduled Leader fails within the applicable protocol window, the next Validator in the same round-robin order becomes eligible to continue the round. Leader replacement does not change the Canonical Evaluator, eligible Miner set, evaluation request, valid timely responses already received, or other committed ranking inputs. A failed Leader assignment is handled under the Validator liveness rules.

Because ranking-round Leader order is predictable, ranking integrity does not depend on Leader unpredictability. It depends on committed inputs, response completeness, authenticated provenance, independent Validator execution, commit-reveal, deterministic aggregation, and BFT finalization. Other mechanisms that use finalized protocol randomness, including hidden sampling, Validator verification-subset selection, spot checks, and Block Builder selection, remain unchanged.

K.7 Routing and Post-Ranking Security

Operational routing MUST consume finalized Miner Ranking and routing-eligibility state. Routing components, Collectors, the Autonomous Engine, and application integrations cannot arbitrarily rewrite a finalized ranking. Request-level operational failover may choose another eligible Miner after a supported delivery failure, but that failover does not modify ranking state.

Finalized ranking state remains subject to protocol spot checks. At genesis, $k_{\text{spot}} = 10$ and a relative performance decline greater than 20 percent may produce a Routing_Revocation. A finalized Routing_Revocation removes the affected Miner from active routing for the applicable period and redistributes routing opportunity according to the current routing policy. Miner registration principal remains intact.

Where a Consumer applies supported confidentiality, security, jurisdictional, organizational, allowlist, or other Intent-compatible constraints, the eligible Miner set is restricted first and the finalized ranking is applied within that eligible set. Telegraph does not infer legal or regulatory compliance solely from self-declared Miner metadata, applications remain responsible for applying the policies appropriate to their use case.

K.8 Miner Credential Protection and Credential Store

The credential and confidential-execution security model separates three classes of sensitive information: Miner access credentials, Consumer request and response payloads, and protocol metadata required for routing, accounting, ranking, and consensus. Telegraph minimizes plaintext exposure for the first two classes while retaining the protocol metadata required for operation.

A Miner service that requires authentication should use a dedicated Telegraph credential issued through the Miner's own service provider. The credential should use the minimum permissions, spend limits, rate limits, expiration, and other restrictions supported by that provider. The Miner retains provider-level control and may revoke or rotate the credential independently. The on-chain Miner registration binds the encrypted credential package through `credential_hash`.

Where enclave-backed credential protection is active, Telegraph uses hardware-backed execution together with threshold cryptography, or an equivalent distributed confidential-compute mechanism. Distributed key material is maintained across authorized protected execution environments. No single Validator is intended to possess a persistent network-wide credential-decryption key. Public encryption material and commitments identifying the active credential-management configuration are published through protocol state, while private key shares remain associated with authorized protected execution environments.

Threshold operations require the protocol-defined authorization threshold before a Miner credential can be used. The security objective is that no single ordinary Validator process holds persistent network-wide credential-decryption capability.

Hardware attestation is used before sensitive key material is provisioned to an execution environment. Attestation establishes evidence about the reported software and hardware measurements, it does not eliminate all hardware, firmware, implementation, or side-channel risk.

Encrypted Miner credentials are stored in a distributed Credential Store. Each package is content-addressed and bound to the corresponding Miner registration through `credential_hash`. The protocol maintains an authenticated commitment to Credential Store state, such as a Merkle root. Public protocol state stores the commitments and identifiers needed for verification, plaintext Miner credentials are not stored on-chain.

K.9 Credential Onboarding, Use, Rotation, and Compromise Containment

A Miner using authenticated access follows the credential lifecycle below:

1. create a dedicated Telegraph credential through the Miner service provider,
2. apply the minimum permissions, rate limits, spend or economic limits, and expiration appropriate to the integration,
3. obtain the current Telegraph credential-encryption configuration,
4. encrypt the credential before submission to the Credential Store,
5. upload the encrypted package through an authenticated submission path, and
6. commit the resulting `credential_hash` through a transaction authorized by the address controlling the Miner registration.

The submission path MUST verify that the updater is authorized to modify the corresponding Miner registration. Credential rotation creates a new encrypted package and a new credential_hash. New requests use the updated credential package only after the new registration state becomes active.

For an authorized credential use, the protected execution environment verifies the applicable protocol authorization, retrieves the package matching the active credential_hash, completes the required threshold operation, establishes the authenticated connection to the Miner, supplies the credential from within the protected context, and clears temporary plaintext credential material when the operation completes. The surrounding ordinary Validator host process MUST NOT receive the reusable plaintext credential when enclave-backed protection is active.

Changes to the authorized Validator or credential-management set require refresh, rekeying, re-encryption, or another supported distributed key-management procedure so removed Validators do not receive key material required for future credential operations. The complete private decryption key need not be reconstructed in an ordinary Validator process. A Validator joining an enclave-backed credential role must complete the required attestation before applicable key material is provisioned.

Enclave provisioning binds the Validator identity, approved execution-environment measurement, and distributed key share. Applicable key material is provisioned to the protected execution environment and is not intentionally exposed through the ordinary Validator application interface.

Connections to Miners use authenticated transport such as TLS. The protected execution environment verifies the authenticated Miner endpoint before transmitting a credential or sensitive request data. If a Miner credential is suspected to be compromised, the Miner may revoke it through the underlying provider, issue a replacement, and update its Telegraph registration. Compromise of one Miner credential does not grant authority over another Miner's credentials. Misuse of credential-management infrastructure by a Validator is handled under the applicable protocol-violation rules without reducing Validator principal.

K.10 Confidential Consumer Requests

For supported requests requiring stronger confidentiality than ordinary TLS routing provides, Telegraph uses an attested confidential-execution path when the selected eligible Miner supports the required confidential-compute capability. The Consumer specifies the applicable confidentiality and routing requirements, Telegraph derives the eligible Miner set, and the finalized Miner Ranking is applied within that set.

A confidential request follows this security sequence:

Consumer -> confidentiality and routing requirements -> eligible Miner set -> finalized ranking -> Miner execution attestation -> encrypted request -> authorized confidential execution -> encrypted result -> authorized Consumer

Decryption capability is provisioned directly to the attested execution environment through the supported secure key-provisioning mechanism. Ordinary Validator, routing, consensus, and settlement processes do not require the plaintext request key. Plaintext Consumer inputs and plaintext Miner outputs remain inside the authorized confidential execution environment and with the authorized Consumer as required by the request, intermediary

Telegraph components may process ciphertext and the metadata required for routing, accounting, and settlement.

Where a confidential Miner response must participate in protocol evaluation, the applicable evaluation may execute inside an authorized confidential execution environment using the Canonical Evaluator WASM. The protocol may expose the resulting evaluation output, commitments, and attestation evidence needed for ranking verification without exposing the underlying plaintext response to ordinary Validator processes. The encrypted response or committed payload is cryptographically bound so alteration is detectable.

Live paid Consumer traffic is not the Candidate Evaluator competition test set. Miner ranking may use separate protocol evaluation inputs and committed evaluation material, allowing confidential production traffic to remain confidential while ranking continues independently.

The confidentiality properties of this path depend on the selected encryption, attestation, confidential-compute hardware, key-provisioning mechanism, and implementation. Telegraph does not treat hardware attestation as eliminating all implementation or hardware risk.

K.11 Multi-Role Participation and Economic Security

Telegraph assumes that one economic actor may participate in multiple permitted roles, including Consumer, Miner, Evaluator, evaluation-material contributor, or Validator. Address separation is not treated as proof of economic independence. The security model therefore relies on protocol accounting, competition, verification, and finalization rules rather than an assumption that participant identities are economically unrelated.

Only finalized protocol-recognized paid Consumer activity qualifies for demand-dependent economic mechanisms. Evaluator competition, hidden verification, Miner-ranking requests, spot checks, protocol testing, bootstrap activity, Intent Bounties, Treasury-funded activity, grants, free traffic, and other internally generated protocol activity are excluded from Qualified Consumer Demand.

Evaluator rewards are additionally bounded by irreversible protocol fee generation. At genesis:

$$F_i(d) = 0.02 \times D_i(d)$$

$$\text{RewardBackingUSDC}_i(d) = 0.80 \times F_i(d) = 0.016 \times D_i(d)$$

The applicable Evaluator MACHINA allocation cannot exceed the corresponding fee-backed cap derived from P_{ref} . Equivalent total paid demand has equivalent aggregate raw allocation weight regardless of how it is divided among Consumer wallets, Miners, or Intents, subject only to deterministic rounding. These accounting rules are economic constraints, they do not replace Evaluator competition, provenance, bounded contribution influence, ranking verification, or BFT consensus.

K.12 Reference Price, Settlement, and Replay Security

The fee-backed Evaluator cap compares USDC-denominated backing with MACHINA-denominated emissions and therefore depends on a manipulation-resistant reference price P_{ref} . P_{ref} MUST NOT rely on a single current-block spot price. The reference mechanism may use sufficiently long Ethereum TWAP observations, realized protocol settlement execution data,

minimum-liquidity requirements, multiple approved observations or venues where applicable, deviation limits, circuit breakers, and other protocol-defined oracle protections.

For the Evaluator cap, an understated MACHINA reference price is the dangerous direction because it permits a larger token allocation for a fixed amount of USDC backing. If a valid `P_ref` cannot be established under the applicable oracle rules, Evaluator emission distribution for the affected period does not proceed and the corresponding undistributed allocation is transferred to the Protocol Treasury.

Consumer payments are aggregated before MACHINA acquisition on Ethereum. Settlement may be time-distributed under the execution controls defined in Part II H. Miner entitlement is based on the MACHINA actually acquired, not on an estimated future exchange rate.

Cumulative entitlement accounting and `WithdrawnAmount` state prevent repeated withdrawal of the same finalized entitlement, and previously finalized cumulative entitlements are not retroactively reduced.

Protocol evidence and authorizations bind the identifiers and versioned context required by the mechanism that produced them, including the relevant Intent, round or job, protocol version, state roots, Validator-set context, settlement period, and chain domain. A proof, score commitment, job authorization, or state submission valid in one context MUST NOT be reusable as authorization for an unrelated protocol event.

Replay and deterministic audit establish whether a finalized transition was computed consistently with the committed protocol inputs and protocol rules. Authenticated transcripts establish provenance and integrity of the retrieval used by the protocol. Neither replay nor transcript evidence establishes that the semantic content of a Miner response is objectively true.

K.13 Collector, Internal-Request, and Signal Security

Collector inputs are untrusted external data. Collector content cannot directly select a Miner, alter Miner Ranking, select a Canonical Evaluator, modify an Intent registration, or change protocol economic state. After Collector output becomes a request, Intent resolution and routing occur under the same finalized protocol state used by direct Consumer requests.

Internal orchestration-to-Engine and daemon-to-Engine requests MUST use cryptographic authentication or an equivalent authenticated internal-request mechanism. A plaintext internal header alone is not sufficient authority to bypass payment or access controls.

Signals retain the metadata and commitments required to reconstruct the applicable execution path. Depending on the path, this may include `Signal_ID`, `signal_hash`, `Intent_ID`, `Miner_ID`, request commitment, response commitment, Collector identity, trigger commitment, routing metadata, and payment or protocol-generation classification.

A Miner may attach optional supporting material to a Signal as a Transparency Attachment. Such an attachment may contain reasoning traces, citations, source references, confidence information, provenance metadata, or other Miner-supplied explanatory material. When present, it is cryptographically bound to the associated Signal through the applicable commitment or receipt mechanism. Telegraph does not treat a Transparency Attachment, its reasoning, citations, claims, or conclusions as proof of correctness. Consumers and downstream applications may evaluate that material according to their own requirements.

K.14 Failure Recovery

Telegraph preserves the last finalized state when a new ranking, settlement, governance-dependent, or other protocol transition cannot obtain its required authorization. The protocol does not weaken consensus requirements to preserve liveness. Once sufficient Validator participation returns, normal finalization resumes from the last valid finalized state.

If a replacement Miner-ranking round fails, the preceding finalized Miner Ranking remains active subject to normal routing eligibility. If an Evaluator challenger fails hidden verification, sequential verification advances to the next eligible candidate and the previously verified Canonical Evaluator remains active when no challenger qualifies. If a selected ranking-round Leader or Block Builder fails its assignment, deterministic failover advances to the next eligible Validator under the applicable schedule without changing already committed protocol inputs.

Operational delivery failures may trigger request-level Miner failover without rewriting ranking state. Persistent Miner degradation is handled by subsequent ranking, spot checks, and Routing_Revocation. Failed economic execution follows the settlement rollover or failed-execution rules defined by the applicable settlement parameters, unfinalized economic transitions do not overwrite previously finalized entitlement state.

K.15 Security Boundary

Telegraph produces verified rankings. The protocol verifies and finalizes the committed execution, provenance, evaluation, aggregation, ordering, routing eligibility, accounting, and consensus processes that produce and use ranking state. It does not claim to prove that every machine-generated answer, model output, source assertion, Transparency Attachment, or external dataset is objectively true.

Ranking quality depends on the participating Miners, the current Canonical Evaluator, the quality and diversity of committed evaluation material, the Shared Foundation Model where applicable, and the current qualification parameters. Continuous Miner and Evaluator competition allow the system to replace weaker supply and evaluation mechanisms as stronger participants enter. Protocol security therefore establishes reproducible state transitions and bounded authority around that competitive process, it does not replace domain-specific judgment with a universal truth oracle.

PART III - GENESIS, PARAMETERS & APPENDICES

Initial configuration and consolidated protocol reference

Part III consolidates Telegraph's genesis configuration and reference registries. It does not create a second set of protocol mechanics. Parts I and II define the architecture and normative procedures, this Part records the initial values, formulas, roots, commitments, and version bindings used to initialize and audit those mechanisms.

Governable values in this Part are genesis values and may change only through the governance process defined in Part II J. Immutable values and properties remain outside governance. Where this specification does not state a numeric genesis value, this Part preserves that value as deployment-finalized rather than introducing an unstated constant.

A. GENESIS PARAMETER REGISTRY

The following registry consolidates the initial Telegraph configuration and the protocol-wide constants defined elsewhere in this specification.

A.1 MACHINA

- Maximum supply $S_{\max}$: 21,000,000 MACHINA.
- Genesis daily emission E_0 : 7,200 MACHINA per 24-hour emission period.
- Halving interval: approximately 1,461 emission days, or four years.
- Validator emission allocation: 60 percent.
- Evaluator emission allocation: 20 percent.
- Protocol Treasury emission allocation: 20 percent.
- Miner protocol-emission allocation: 0 percent.
- Pre-mine: 0.
- Team / VC allocation: 0.

$$E(d) = E_0 \times 2^{(-\text{floor}(d / 1461))}$$

Emission ceases when cumulative minted supply reaches $S_{\max}$. If the remaining mintable supply is smaller than the scheduled emission for a period, only the remainder is minted and protocol emission then permanently halts.

A.2 Validators and Consensus

- Genesis Validator Cap N : 64.
- BFT threshold: $\tau = \text{floor}(2 \times |V| / 3) + 1$, at $|V| = 64$, $\tau = 43$.
- Validator unbonding period: 21 days.
- Continuous liveness-ejection threshold: 7 consecutive days without completing a required assignment.
- Rolling liveness-ejection threshold: more than 50 percent of required assignments missed within any rolling 7-day window.
- Category C ejection threshold: 5 strikes within any rolling 30-day window.
- Evaluator verification subset: $\text{ceil}(0.10 \times |V|)$, at 64 Validators, 7 Validators.
- Internal Audit Fee: 2 percent of the applicable Validator emission pool.
- No reduced BFT threshold exists.
- Validator principal is never slashed, burned, confiscated, transferred, or reduced as a protocol penalty.

A.3 Miner Ranking and Routing

- Protocol-facing Local_Score domain: $[0,1]$.
 - Canonical Score aggregation: stake-weighted median using the finalized ranking-round stake snapshot.
 - Minimum valid score reveals required for a Canonical Score: τ .
 - Equal Canonical Scores are ordered by ascending Miner_ID.
 - Before the first finalized Miner Ranking for an Intent, routing is uniform among eligible registered Miners.
 - After ranking exists, routing is probabilistic and ranking-based.
 - Genesis spot-check parameter k_{spot} : 10.
-

- Genesis Routing Revocation threshold: more than 20 percent relative performance decline.
- Ranking-round Leader coordination uses strict round-robin rotation over the ordered Active Validator Set as defined in Part II D. Other randomness-dependent mechanisms retain finalized protocol randomness.

A.4 Evaluator Selection

- Candidate Evaluators are self-hosted during the initial competition stage.
- Candidate-ranking workloads use standardized public, mock, historical, synthetic, or otherwise non-sensitive material.
- Candidate ranking determines the order in which Evaluators enter Canonical verification, candidate-ranking victory alone does not grant Canonical status.
- The presumptive winner's committed executable is retrieved and independently verified against hidden evaluation material.
- If a candidate fails hidden verification, verification advances to the next-ranked eligible candidate.
- If no challenger passes, the previously verified Canonical Evaluator remains active.

A.5 Hidden Evaluation

- Hidden evaluation material consists of eligible committed evaluation contributions for the Intent.
- Mature maximum effective contributor share: 2 percent.
- Bootstrap contribution rule: $\text{MaxShare}_i = \max(0.02, 1 / C_i)$.
- Hidden verification batches are sampled from committed evaluation state using finalized protocol randomness.
- Hidden verification material is not sent to candidate-operated endpoints.
- Canonical promotion requires successful hidden verification under the applicable deterministic qualification parameters.

A.6 Registration

- Miner deregistration unbonding period: 7 days.
- Validator unbonding period: 21 days.
- Miner registration bond: final genesis value to be specified before deployment.
- Evaluator registration bond: final genesis value to be specified before deployment.
- Intent registration or activation bond: final genesis value to be specified before deployment if required.
- Registration principal is never slashed.

This specification does not state a separate numeric genesis Evaluator-unbonding period. Governance may modify the Evaluator-unbonding parameter once governance is active, subject to the applicable timelock, this registry does not introduce an unstated genesis value.

A.7 Consumer Economics

- Protocol fee: 2 percent of gross paid Consumer USDC.
- Miner settlement allocation: 98 percent of gross paid Consumer USDC.
- Protocol minimum Signal price: 0.01 USDC.
- Evaluator fee-backing coefficient beta: 0.80.

- Only protocol-recognized paid Consumer requests count toward the Demand Multiplier pricing metric, non-Consumer protocol traffic is excluded.

Qualified paid request volume	Demand Multiplier
0-999	1.0×
1,000-9,999	1.5×
10,000-99,999	2.5×
100,000-999,999	5.0×
1,000,000+	10.0×

$$\text{SignalPrice}_{(m,i)} = \text{min_price_usdc}_{(m,i)} \times \text{DemandMultiplier}_i$$

The Demand Multiplier affects Consumer pricing and is not separately applied to Evaluator emission allocation.

A.8 Evaluator Economics

$$\text{EvaluatorEmissionPool}(d) = 0.20 \times E(d)$$

$$D_{\text{total}}(d) = \sum D_i(d)$$

$$\text{RawEvaluatorAllocation}_i(d) = \text{floor}(\text{EvaluatorEmissionPool}(d) \times D_i(d) / D_{\text{total}}(d))$$

$$F_i(d) = 0.02 \times D_i(d)$$

$$\text{RewardBackingUSDC}_i(d) = \text{beta} \times F_i(d)$$

At genesis $\text{beta} = 0.80$, therefore $\text{RewardBackingUSDC}_i(d) = 0.016 \times D_i(d)$.

$$\text{FeeBackedCap}_i(d) = \text{floor}(\text{RewardBackingUSDC}_i(d) / P_{\text{ref}}(d))$$

$$\text{FinalEvaluatorAllocation}_i(d) = \min(\text{RawEvaluatorAllocation}_i(d), \text{FeeBackedCap}_i(d))$$

Only the eligible verified Canonical Evaluator receives the applicable Intent allocation. Undistributed Evaluator emission is transferred to the Protocol Treasury and is not carried forward as future reward capacity.

A.9 Settlement

- Canonical economic settlement domain: Ethereum.
- Consumer accounting asset: USDC.
- Miner settlement asset: MACHINA acquired from the open market using Miner-settlement USDC.
- Real-time intelligence delivery does not wait for Ethereum settlement.
- Economic settlement is aggregated.
- Miner, Validator, and Evaluator rewards use cumulative lifetime entitlement accounting where applicable.

A.10 Governance

- Governance activation threshold: 43 active Validators.
- Governance voting threshold: $\text{floor}(2 \times |V| / 3) + 1$, at 64 Validators, 43 affirmative votes.

- Discussion period: 7 days.
- Voting period: 7 days.
- Emergency veto threshold: $\text{ceil}(5 \times |V| / 6)$, at 64 Validators, 54.
- Emergency-veto cooldown for a materially equivalent proposal: 90 days.
- Each active Validator has one governance vote independent of stake weight.

Parameter class	Genesis timelock
Validator Cap	30 days
Validator unbonding	30 days
Miner / Evaluator unbonding	30 days
Ranking or Evaluator competition cadence	30 days
Evaluator statistical qualification parameters	14 days
Evaluation contribution and sampling parameters	14 days
Penalty and liveness thresholds	14 days
Registration bonds	14 days
Protocol fee	14 days
Minimum Signal price	14 days
Demand Multiplier schedule	14 days
Routing distribution	14 days
Spot-check parameters	14 days
Routing-revocation threshold	14 days
Internal Audit Fee	14 days
Other ordinary operational parameters	14 days

A proposal affecting multiple parameters uses the longest applicable timelock. Timelocks begin only after the proposal has passed.

A.11 Intent Identity

```
Intent_ID = H(Registrant_Address || Schema_Hash || Block_Number)
```

A material Intent interface change creates a new version according to the protocol versioning rules. Previously finalized state remains bound to the version under which it was produced.

A.12 Bootstrap

- Genesis may include initial Intents, Canonical Evaluators, evaluation material, Foundation Miners, Collectors, Validators, and other infrastructure required to initialize protocol operation.

- Where applicable, genesis may include the initial Foundation Model version, initial training material, initial Evaluator artifacts, Evaluator-generation or training artifacts, and evaluation/training configuration required by bootstrap Intents.
- Bootstrap participants receive no permanent ranking, routing, Evaluator, or governance privilege.
- Bootstrap and Treasury-funded activity is excluded from Qualified Consumer Demand and demand-weighted Evaluator reward calculations.
- Foundation Miners receive no permanent routing preference and compete under the normal routing rules once ranking state exists.
- Bootstrap operation below normal BFT participation does not weaken the protocol quorum rule.

A.13 Immutable Protocol Properties

- Maximum MACHINA supply, emission schedule and halving rule, and the 60% Validator / 20% Evaluator / 20% Treasury emission split are immutable.
- Pre-mine = 0, team / VC allocation = 0, Miner protocol emissions = 0.
- Validator, Miner, and Evaluator principal is never slashed, burned, confiscated, or transferred as a protocol penalty.
- Finalized cumulative economic entitlements cannot be retroactively reduced.
- The BFT threshold rule $\tau = \text{floor}(2 \times |V| / 3) + 1$ is immutable.
- Canonical Evaluator status must result from protocol-defined competition and hidden verification.
- Miner Rankings must result from protocol-defined evaluation, aggregation, verification, and finalization.
- Governance cannot directly appoint a competitive protocol winner or retroactively alter finalized ranking, consensus, accounting, or settlement state.

A.14 Foundation Model and Evaluation Versioning

- Shared Foundation Model: enabled where required by the applicable Evaluator architecture.
- Foundation Model training lifecycle: separate from ordinary Evaluator competition.
- A successful Foundation Model update creates a new `Foundation_Model_Version`.
- A Foundation Model version change triggers revalidation of affected Canonical Evaluators before the new backbone becomes authoritative.
- Canonical Evaluator state binds `Foundation_Model_Version` where applicable.
- Canonical evaluation state also binds `Evaluator_Version`, `Evaluation_Pool_Version`, `Evaluation_Engine_Version`, sampling randomness or sampling state, and applicable evaluation parameters.
- A new Evaluator may replace the current Canonical Evaluator without retraining the Foundation Model.
- The Foundation Model does not determine Canonical Evaluator selection.

B. CONSOLIDATED FORMULA REGISTRY

This appendix consolidates the principal protocol formulas already defined normatively in Part II. It is a reference index, not a replacement for the mechanism-specific conditions surrounding each formula.

B.1 Identity, Evaluation, and Ranking

Mechanism	Formula / rule
Intent identity	$\text{Intent_ID} = H(\text{Registrant_Address} \parallel \text{Schema_Hash} \parallel \text{Block_Number})$
Evaluation contribution	$\text{Evaluation_Commitment} = H(\text{Intent_ID} \parallel \text{Material_Hash} \parallel \text{Metadata_Hash} \parallel \text{Version})$
Contribution influence	$\text{MaxShare}_i = \max(0.02, 1 / C_i)$
Verification subset	$\text{VerificationSubsetSize} = \text{ceil}(0.10 \times V)$
Verification seed	$\text{Verification_Seed} = H(\text{Protocol_Randomness} \parallel \text{Intent_ID} \parallel \text{Evaluator_Round_ID})$
Eligible Miner set	$\text{Eligible_Miner_Set_Root} = \text{MerkleRoot}(\text{Eligible_Miners})$
Ranking Leader	$\text{LeaderIndex} = \text{Round_ID} \bmod V $
Miner response set	$\text{Miner_Response_Root} = \text{MerkleRoot}(\text{Valid_Miner_Responses})$
Local score	$\text{Local_Score}_{(v,m)} = E_i * (\text{Evaluation_Input}_m)$
Canonical score	$\text{Canonical_Score}_m = \text{StakeWeightedMedian}(\text{Local_Scores}_m)$
Miner ordering	$R_i = \text{Sort}(M_i, \text{Canonical_Score DESC}, \text{Miner_ID ASC})$

B.2 Consensus and State

Mechanism	Formula / rule
Byzantine quorum	$\tau = \text{floor}(2 \times V / 3) + 1$
Consensus signing domain	$\text{Consensus_Message} = H(\text{Chain_ID} \parallel \text{Protocol_Version} \parallel \text{Validator_Set_ID} \parallel \text{State_Sequence} \parallel \text{Protocol_State_Root})$
Block Builder	$\text{BlockBuilderIndex} = H(\text{Protocol_Randomness} \parallel \text{"block"} \parallel \text{Period_ID}) \bmod V $
Successor state	$\text{State_Sequence_new} = \text{State_Sequence_previous} + 1$
Claimable entitlement	$\text{Claimable}_p = \text{ProvenLifetimeEntitlement}_p - \text{WithdrawnAmount}_p$

B.3 Payments and Direct Jobs

Mechanism	Formula / rule
Escrow effective balance	$\text{EffectiveBalance} = \text{EscrowBalance} - \text{AccountedSpend} - \text{ReservedInFlightExposure}$
Streaming acknowledgement	$\text{ACK}_k = \text{Sign}(\text{Session_ID} \parallel \text{Sequence}_k \parallel \text{CumulativeTab}_k \parallel \text{Context})$
Session exposure bound	$\text{AcknowledgedSpend} + \text{MaximumInFlightExposure} \leq \text{AuthorizedSessionLimit}$
Aggregate exposure bound	$\text{AggregateInFlightExposure} \leq \text{AvailableEscrow}$
Job result hash	$\text{outputHash} = H(\text{Miner_Response})$
Job resolution	$\text{JobResolutionRoot} = H(\text{Job_ID} \parallel \text{Intent_ID} \parallel \text{Miner_ID} \parallel \text{Request_Commitment} \parallel \text{outputHash} \parallel \text{Payment_State} \parallel \text{Protocol_Context})$

B.4 Economics

Mechanism	Formula / rule
Daily emission	$E(d) = E_0 \times 2^{(-\text{floor}(d / 1461))}$
Protocol fee	$\text{ProtocolFee} = 0.02 \times \text{GrossPaidUSDC at genesis}$
Miner settlement USDC	$\text{MinerSettlementUSDC} = 0.98 \times \text{GrossPaidUSDC at genesis}$
Signal price	$\text{SignalPrice}_{(m,i)} = \text{min_price_usdc}_{(m,i)} \times \text{DemandMultiplier}_i$
Validator pool	$\text{ValidatorEmissionPool}(d) = 0.60 \times E(d)$
Evaluator pool	$\text{EvaluatorEmissionPool}(d) = 0.20 \times E(d)$
Qualified demand total	$D_{\text{total}}(d) = \sum D_i(d)$
Raw Evaluator allocation	$\text{RawEvaluatorAllocation}_i(d) = \text{floor}(\text{EvaluatorEmissionPool}(d) \times D_i(d) / D_{\text{total}}(d))$
Fee backing	$\text{RewardBackingUSDC}_i(d) = \text{beta} \times 0.02 \times D_i(d)$
Fee-backed cap	$\text{FeeBackedCap}_i(d) = \text{floor}(\text{RewardBackingUSDC}_i(d) / P_{\text{ref}}(d))$
Final Evaluator allocation	$\text{FinalEvaluatorAllocation}_i(d) = \text{min}(\text{RawEvaluatorAllocation}_i(d), \text{FeeBackedCap}_i(d))$
Miner period entitlement	$\text{PeriodMinerEntitlement}_{(m,N)} = \text{floor}(\text{MinerUSDC}_{(m,N)} \times M_N / U_N)$

C. STATE AND COMMITMENT REGISTRY

This appendix lists the principal cryptographic roots and commitments used to bind Telegraph state. Full field requirements and transition rules remain defined in the corresponding Part II sections.

C.1 Top-Level Protocol State

- `Protocol_State_Root` - top-level commitment to consensus-relevant protocol state for the applicable `State_Sequence`.
- `Active_Validator_Set_Root` - commitment to the Validator set authorized for the applicable consensus context.
- `Ranking_State_Root` - commitment to finalized Miner Rankings and associated ranking state.
- `Canonical_Evaluator_Root` - commitment to each active Intent's current Canonical Evaluator and versioned execution state.
- `Evaluation_State_Root` - commitment to ranking/evaluation data required for deterministic audit.
- `Routing_State_Root` - commitment to routing eligibility, `Routing_Revocation` state, and other consensus-relevant routing state.
- `Penalty_Status_Root` - commitment to Validator penalty and participation status where represented by protocol state.

C.2 Economic Entitlement State

- `Miner_Entitlement_Root` - cumulative lifetime Miner MACHINA entitlements.
- `Validator_Entitlement_Root` - cumulative lifetime Validator emission entitlements after finalized assignment and penalty accounting.
- `Evaluator_Entitlement_Root` - cumulative lifetime Evaluator emission entitlements after Canonical eligibility, fee-backed allocation, and Treasury redirection rules.

Entitlement roots represent lifetime gross entitlement. Withdrawals are tracked separately and do not reduce the cumulative value committed by later roots.

C.3 Ranking and Evaluation Commitments

- `Eligible_Miner_Set_Root` - commits the eligible Miner set fixed for a ranking round before results are known.
- `Miner_Response_Root` - commits the valid Miner responses received within the ranking collection window.
- `Evaluation_Package_Root` - binds the Intent, round, eligible set, request or batch, response root, provenance, Canonical Evaluator state, evaluation configuration/material commitment, protocol version, and deterministic execution inputs.
- `Score_Commitment(v,m)` - binds Validator identity, Intent, round, Miner, evaluation package, Canonical Evaluator executable hash, `Local_Score`, and nonce before reveal.
- `Evaluation_Commitment` - binds Intent, material hash, metadata hash, and contribution version.

C.4 Payment, Job, and Settlement Commitments

- `ACK_k` - signed cumulative WebSocket/session acknowledgement binding session, sequence, cumulative tab, and context.
- `outputHash` - hash commitment to a direct-job Miner response.
- `JobResolutionRoot` - commitment to the job identity, Intent, Miner, request, output, payment state, and protocol context authorized by Validators.
- `Consensus_Message` - hash binding chain, protocol version, Validator set, state sequence, and `Protocol_State_Root`.
- Canonical Ethereum state submission - commits the `State_Sequence`, `Settlement_Period_ID`, Validator-set identifier/root, signer set or bitmap, `Protocol_State_Root`, entitlement roots, aggregate Validator signature, and required economic metadata.

C.5 Version and Context Binding

Consensus-critical evidence is valid only in its committed context. Depending on the mechanism, commitments bind `Intent_ID`, `Round_ID` or `Job_ID`, protocol version, Canonical Evaluator executable hash and version, `Foundation_Model_Version` where applicable, `Evaluation_Pool_Version`, `Evaluation_Engine_Version`, Validator-set state, settlement period, `State_Sequence`, and chain domain.

A material change to an Intent interface, Evaluator artifact, Foundation Model version, evaluation pool, evaluation engine, or another compatibility-sensitive protocol input follows the applicable versioning and revalidation rule. Evidence or authorization valid in one context cannot be reused to authorize an unrelated protocol event.

D. REFERENCE BOUNDARY

Part III is a consolidated reference layer. The mechanism-specific requirements in Part II remain authoritative for eligibility, timing, sequencing, verification, failure handling, and transition validity. The Genesis Parameter Registry fixes initial values where this specification defines them and preserves deployment-finalized values where it does not.

The protocol uses strict round-robin rotation for ranking-round Leader coordination. This rule does not modify other randomness-dependent mechanisms. Hidden Evaluator sampling, Validator verification-subset selection, spot checks, and Block Builder selection continue to use finalized protocol randomness as defined in their respective sections.

No registry entry changes the protocol invariants established elsewhere in this specification: no principal slashing, no retroactive clawback of finalized cumulative entitlements, no reduced Byzantine quorum, no direct governance appointment of competitive protocol winners, zero Miner protocol emissions, Ethereum as the canonical economic settlement domain, and verified rankings as the protocol security boundary.